

分类号: TP309
密 级: 公 开

单位代码: 10422
学 号: 202117047



山东大学
SHANDONG UNIVERSITY

博士学位论文

Dissertation for Doctoral Degree

论文题目: 安全多方计算中的多方隐私集合运算研究

Research on Multi-Party Private Set Operations in Secure Multi-Party
Computation

作者姓名	董明朗
培养单位	网络空间安全学院
专业名称	网络空间安全
指导教师	陈宇

2026 年 5 月 29 日

原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师指导下，独立进行研究所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的科研成果。对本论文的研究作出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律责任由本人承担。

论文作者签名：_____ 日 期：_____

关于学位论文使用授权的声明

本人完全了解山东大学有关保留、使用学位论文的规定，同意学校保留或向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅；本人授权山东大学可以将本学位论文全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其他复制手段保存论文和汇编本学位论文。

（保密的论文在解密后应遵守此规定）

论文作者签名：_____ 导师签名：_____ 日 期：_____

摘 要

随着大数据与人工智能技术的飞速发展，跨机构的数据融合与协同计算已成为释放数据价值的关键途径。然而，数据孤岛现象与日益严格的隐私保护法律法规之间的矛盾，使得如何在保护数据隐私的前提下实现数据价值的安全流通成为亟待解决的问题。作为安全多方计算（Secure Multi-party Computation, MPC）的重要分支，隐私集合运算（Private Set Operations, PSO）允许参与方在不泄露私有数据的前提下协同计算集合的交集、并集等信息。尽管两方场景下的技术已趋于成熟，但多方场景（Multi-Party Private Set Operations, MPSO）在安全性、实用性、功能性和统一性方面都存在严重不足，仍面临严峻挑战：一方面，现有的多方隐私集合求并（Multi-party Private Set Union, MPSU）协议或依赖不切实际的“非合谋假设”，难以抵御现实世界存在的任意共谋攻击，或性能低下，难以满足实际应用的需求；另一方面，现有 MPSO 协议功能单一，难以满足现实场景中的复杂需求，且技术异构，开发部署维护成本高。目前学术界缺乏能够支持任意集合公式计算的 MPSO 统一框架。

针对上述挑战，本文深入研究了 MPSO 的核心理论与关键技术，从具体协议的突破到通用框架的构建，取得了一系列创新性成果：

首先，针对 MPSU 协议安全性弱与效率低下的痛点，本文提出了一种新的密码学组件——批量秘密分享隐私成员测试（batch secret-shared private membership test, batch ssPMT）协议，并以此为基础分别基于对称密钥技术和公钥密码技术提出了两种安全高效的 MPSU 协议。其中在对称技术路线中，本文构造了首个在标准半诚实模型下证明安全的基于对称密钥的 MPSU 协议。该协议不仅成功消除了之前最先进的（State-of-the-Art, SOTA）协议 [1] 对非共谋假设的依赖，显著增强了安全性，还表现出更加优异的具体性能。在局域网（LAN）环境下，其在线阶段的运行效率提升了 3.9 ~ 10.0 倍，整体运行效率提升了 1.2 ~ 7.8 倍。在公钥技术路线中，本文构造了首个同时实现线性计算复杂度和线性通信复杂度的 MPSU 协议，其总通信量相比于之前 SOTA 协议 [1] 降低了 3.0 ~ 36.5 倍，在带宽受限的广域网（WAN）环境下具有显著优势。

其次，为了解决现有 MPSO 协议功能局限以及缺乏统一性的难题，本文通过引入新的密码学原语——谓词零分享（Predicative Zero-Sharing），基于对称密钥技术构建了首个实用的 MPSO 统一框架。该框架不仅能够计算由交、并、差运算任意组合构成的集合公式，还可扩展至支持更为复杂的任意集合公式结果求势（MPSO-card）及基于电路的通用 MPSO（Circuit-MPSO）功能。

最后，基于该 MPSO 框架，本文实例化了一系列安全高效的具体协议，填补了 MPSO 各个子领域的多项研究空白。其中，实例化的多方隐私集合求交（Multi-party Private Set Intersection, MPSI）协议是首个在标准半诚实模型下实现最优渐进复杂度（与明文传输方案相同量级）的基于对称密钥的 MPSI 方案，同时也是目前在线效率最高的 MPSI 协议，在 LAN 环境下比之前 SOTA 协议 [2] 快了 2.4 ~ 5.2 倍；实例化的

多方隐私交集求势 (MPSI-card) 协议和多方隐私交集求势与和 (MPSI-card-sum) 协议是首个在标准半诚实模型下实现最优渐进复杂度的同类方案, 其中 MPSI-card 协议具有目前同类协议中在线阶段的最佳性能, 其在线通信量比 SOTA [3] 降低了 14.0 ~ 20.3 倍, 而 MPSI-card 协议是唯一拥有具体实现的同类方案; 实例化的基于电路的 MPSI 协议 (Circuit-MPSI) 协议是首个在不诚实大多数设定下安全的同类方案, 突破了 Circuit-MPSI 仅限于诚实大多数安全的局限; 实例化的 MPSU 协议是标准半诚实模型下基于对称密钥的又一高效 MPSU 方案, 在理论上实现了更优的渐进复杂度; 实例化的多方隐私并集求势协议 (MPSU-card) 协议和基于电路的 MPSU 协议 (Circuit-MPSU) 是目前唯一可用的同类构造。此外, 本文还探索了基于公钥体制的 MPSO 框架构造, 完善了 MPSO 的理论技术体系。

关键词: 安全多方计算; 隐私集合运算; 多方隐私集合运算; 多方隐私集合求交; 多方隐私集合求并

ABSTRACT

With the rapid development of big data and artificial intelligence, cross-organization data fusion and collaborative computation have become essential pathways to unlocking the value of data. However, the conflict between "data silos" and increasingly strict privacy protection laws and regulations makes it an urgent problem to achieve the secure circulation of data value while preserving data privacy. As an important branch of Secure Multi-party Computation (MPC), Private Set Operations (PSO) allow parties to collaboratively compute information such as the intersection and union of their sets without disclosing private data. Although technologies in the two-party setting have matured, research of Multi-Party Private Set Operations (MPSO) still suffers from severe deficiencies in security, practicality, functionality, and uniformity, facing significant challenges. On the one hand, existing Multi-party Private Set Union (MPSU) protocols either rely on an unrealistic non-collusion assumptions, rendering them vulnerable to arbitrary collusion attacks in the real world, or fail to achieve the performance adequate for practical applications. On the other hand, current MPSO protocols are functionally limited and struggle to meet the complex demands of real-world scenarios. Moreover, their heterogeneous techniques lead to high costs in development, deployment, and maintenance. Currently, the academic community lacks a unified MPSO framework capable of evaluating arbitrary set formulas.

Addressing the aforementioned challenges, this dissertation deeply investigates the core theories and key technologies of MPSO. Spanning from breakthroughs in specific protocols to the construction of a generic framework, this work yields a series of innovative results:

First, targeting the weak security and inefficiency of existing MPSU protocols, this dissertation proposes a novel cryptographic component — batch secret-shared private membership test (batch ssPMT) protocol. Building upon this, two secure and efficient MPSU protocols are proposed, utilizing symmetric-key and public-key techniques respectively. In the symmetric-key route, this dissertation constructs the first symmetric-key based MPSU protocol proven secure in the standard semi-honest model. This protocol successfully eliminates the reliance on the non-collusion assumption present in the prior State-of-the-Art (SOTA) protocol [1], significantly enhancing security while exhibiting superior concrete performance. In the LAN setting, its online execution efficiency is improved by $3.9 \sim 10.0$ times, and the overall execution efficiency is improved by $1.2 \sim 7.8$ times. In the public-key route, this dissertation constructs the first MPSU protocol that achieves linear computation and linear communication complexity. Compared to the previous SOTA protocol [1], its total communication overhead is reduced by $3.0 \sim 36.5$ times, demonstrating significant advantages in bandwidth-constrained WAN settings.

Second, to resolve the dilemma of functional limitations and the lack of uniformity in existing MPSO protocols, this dissertation introduces a novel cryptographic primitive — Predicative Zero-Sharing. Based on symmetric-key techniques, the first practical and unified MPSO framework is constructed. This framework can compute not only set formulas composed of arbitrary combinations of intersection, union, and difference operations, but also be extended to support more complex functionalities, including computing the cardinality of arbitrary set formula results (MPSO-card) and generic circuit-based MPSO (Circuit-MPSO).

Finally, based on this MPSO framework, this dissertation instantiates a series of secure and efficient concrete protocols, filling multiple research gaps across various MPSO subfields. Among them, the instantiated Multi-party Private Set Intersection (MPSI) protocol is the first symmetric-key MPSI protocol to achieve optimal asymptotic complexity (on par with the plaintext transmission scheme) in the standard semi-honest model. It is also the most online-efficient MPSI protocol to date, operating $2.4 \sim 5.2$ times faster than the prior SOTA protocol [2] in the LAN setting. The instantiated multi-party private set intersection cardinality (MPSI-card) and multi-party private set intersection cardinality and sum (MPSI-card-sum) protocols are the first of their kinds to achieve optimal asymptotic complexity in the standard semi-honest model. The MPSI-card protocol offers the best online performance among existing schemes, reducing online communication by $14.0 \sim 20.3$ times compared to the SOTA [3]; meanwhile, the MPSI-card-sum protocol is the only scheme of its kind with a concrete implementation. The instantiated Circuit-MPSI protocol is the first of its kind secure in the dishonest majority setting, breaking the limitation of previous Circuit-MPSI works that require to assume honest majority. The instantiated MPSU protocol is another efficient symmetric-key MPSU provides in the standard semi-honest model, achieving better asymptotic complexity. The instantiated multi-party private set union cardinality (MPSU-card) and Circuit-MPSU protocols are currently the only available practical constructions of their kind. Furthermore, this dissertation explores the construction of the MPSO framework based on public-key cryptosystems, completing the theoretical and technical system of MPSO.

Keywords: Secure Multi-party Computation; Private Set Operations; Multi-Party Private Set Operations; Multi-Party Private Set Intersection; Multi-Party Private Set Union

目 录

摘 要	I
ABSTRACT	III
主要符号对照表	IX
插图目录	X
表格目录	XI
1 绪论	1
1.1 研究背景与意义	1
1.2 研究现状	4
1.2.1 多方隐私集合求交	4
1.2.2 多方隐私交集计算	5
1.2.3 多方隐私集合求并	6
1.2.4 多方隐私集合运算框架	6
1.3 本文主要贡献	7
1.3.1 多方隐私集合求并协议的研究	7
1.3.2 多方隐私集合运算框架的研究	8
1.4 本文组织结构	11
2 预备知识	12
2.1 符号说明	12
2.2 安全模型	12
2.3 多方隐私集合运算	13
2.4 基本组件	13
2.4.1 随机不经意传输	13
2.4.2 不经意可编程伪随机函数	14
2.4.3 秘密分享隐私等值测试	14
2.4.4 多方秘密分享洗牌	14
2.4.5 哈希分桶	16
2.4.6 多密钥重随机化公钥加密	16
3 基于对称密钥的高效多方隐私集合求并协议	18
3.1 引言	18
3.2 技术路线概述	19
3.2.1 LG 协议的回顾与局限性分析	19

3.2.2	高效的 batch ssPMT 原语	20
3.2.3	基于 batch ssPMT 和 mss-ROT 的 SK-MPSU 协议	21
3.3	批量秘密分享隐私成员测试	23
3.3.1	功能定义	23
3.3.2	协议构造与安全性证明	23
3.3.3	复杂度分析及对比	24
3.3.4	batch ssPMT 的复杂度细节	25
3.3.5	与 mq-ssPMT 的对比	26
3.4	多方秘密分享随机不经意传输	26
3.4.1	功能定义	27
3.4.2	协议构造与安全性证明	27
3.5	SK-MPSU 协议构造及安全性证明	30
3.6	复杂度分析与对比	34
3.6.1	SK-MPSU 的复杂度细节	34
3.6.2	与 LG 协议的对比	35
3.7	本章小结	35
4	基于公钥体制的线性多方隐私集合求并协议	36
4.1	引言	36
4.2	技术路线概述	37
4.2.1	GNT 协议中的 MPSU 框架凝练	38
4.2.2	基于 batch ssPMT 和 MKR-PKE 的 PK-MPSU 协议	39
4.3	协议构造及安全性证明	39
4.4	复杂度分析与对比	44
4.4.1	PK-MPSU 的复杂度细节	44
4.4.2	与 GNT 协议的对比	45
4.5	MPSU 协议的实现与性能评估	45
4.5.1	实现细节	45
4.5.2	实验结果分析	46
4.5.3	实验结果与分析	46
4.6	本章小结	49
5	基于对称密钥的通用多方隐私集合运算框架	50
5.1	引言	50
5.2	集合公式的谓词公式表示	52
5.2.1	可构造集合与集合谓词公式	52

5.2.2	规范集合谓词公式表示	54
5.3	谓词零分享	57
5.3.1	功能定义	57
5.3.2	安全性松弛	57
5.3.3	从松弛到标准的转化技术	61
5.3.4	从简单到复合的组合技术	62
5.4	成员零分享	64
5.4.1	功能定义	65
5.4.2	针对集合成员谓词的松弛成员零分享	65
5.4.3	针对集合非成员谓词的松弛成员零分享	66
5.4.4	针对任意集合谓词公式的成员零分享	66
5.5	实现通用功能及其扩展的 MPSO 框架	68
5.5.1	技术路线概述	68
5.5.2	框架构造及安全性证明	70
5.5.3	复杂度分析	73
5.6	本章小结	74
6	基于对称密钥的实例化协议与综合性能评估	75
6.1	引言	75
6.2	全成员零分享及其变体	76
6.2.1	全成员零分享	76
6.2.2	带载荷的全成员零分享	77
6.3	全非成员零分享	79
6.4	多方隐私集合求交与交集计算	80
6.4.1	多方隐私集合求交、交集求势和基于电路的 MPSI	80
6.4.2	多方隐私交集求势与和	80
6.5	多方隐私集合求并与并集计算	80
6.5.1	多方隐私集合求并、并集求势和基于电路的 MPSU	83
6.5.2	多方隐私集合求并、并集求势和基于电路的 MPSU	83
6.6	各类协议的复杂度分析与对比	83
6.6.1	MPSI 及其变体的复杂度分析	83
6.6.2	MPSU 及其变体的复杂度分析	85
6.6.3	与现有工作的理论对比	85
6.7	各类协议的实现与综合性能评估	88
6.7.1	实现细节与参数设置	88

6.7.2	实验环境与对比方案	89
6.7.3	实验结果与分析	90
6.8	本章小结	90
7	基于公钥体制的通用多方隐私集合运算框架	92
7.1	引言	92
7.2	谓词零加密	92
7.3	成员零加密	92
7.4	框架构造	92
7.5	实例化协议	92
7.6	理论分析与对比	92
7.7	本章小结	92
8	总结与展望	93
8.1	全文总结	93
8.2	未来工作展望	93
	参考文献	94
	致 谢	102
	攻读博士学位期间发表的学术论文	103
	攻读博士学位期间所获奖项	104

主要符号对照表

m	参与方的数量
P_i	第 i 个参与方 ($1 \leq i \leq m$)
X_i	第 i 个参与方持有的私有集合
n	参与方持有集合中的元素个数
l	集合中元素的比特长度
$[x]$	x 在 m 个参与方之间的加法秘密分享
λ	计算安全参数
σ	统计安全参数
$\approx^c$	两个分布在计算上不可区分
$\approx^s$	两个分布在统计上不可区分
$\parallel$	字符串拼接符
$\oplus$	两个向量按分量异或

插图目录

图 2.1	MPSO 中的典型功能	13
图 2.2	二选一随机不经意传输的理想功能 $\mathcal{F}_{\text{ROT}}$	14
图 2.3	批量不经意可编程伪随机函数的理想功能 $\mathcal{F}_{\text{bOPRF}}$	15
图 2.4	秘密分享隐私等值测试的理想功能 $\mathcal{F}_{\text{ssPEQT}}$	15
图 2.5	多方秘密分享洗牌的理想功能 $\mathcal{F}_{\text{shuffle}}$	15
图 3.1	批量秘密分享隐私成员测试的理想功能 $\mathcal{F}_{\text{bssPMT}}$	23
图 3.2	批量秘密分享隐私成员测试协议 Π_{bssPMT}	24
图 3.3	多方秘密分享随机不经意传输的理想功能 $\mathcal{F}_{\text{mss-rot}}$	28
图 3.4	多方秘密分享随机不经意传输协议 $\Pi_{\text{mss-rot}}$	28
图 3.5	SK-MPSU 协议 $\Pi_{\text{SK-MPSU}}$	32
图 4.1	PK-MPSU 协议 $\Pi_{\text{PK-MPSU}}$	41
图 5.1	谓词零分享功能 $\mathcal{F}_{\text{PZS}}^Q$	57
图 5.2	谓词零分享协议 Π_{PZS}^Q	63
图 5.3	批量成员零分享功能 $\mathcal{F}_{\text{bMZS}}^Q$	65
图 5.4	批量成员零分享协议 Π_{bMZS}^Q	67
图 5.5	MPSO / MPSO-card / Circuit-MPSO 协议	71
图 6.1	批量全成员零分享功能 $\mathcal{F}_{\text{bpMZS}}$	76
图 6.2	批量全成员零分享协议 Π_{bpMZS}	77
图 6.3	带载荷的批量全成员零分享功能 $\mathcal{F}_{\text{bpMZSp}}$	78
图 6.4	带载荷的批量全成员零分享协议 Π_{bpMZSp}	78
图 6.5	批量全非成员零分享功能 $\mathcal{F}_{\text{bpNMZS}}$	79
图 6.6	批量全非成员零分享协议 Π_{bpNMZS}	79
图 6.7	MPSI / MPSI-card / Circuit-MPSI 协议	81
图 6.8	MPSI-card-sum 协议	82
图 6.9	MPSU / MPSU-card / Circuit-MPSU 协议	84

表格目录

表 3.1 LG 协议与 SK-MPSU 协议在离线和在线阶段的理论计算与通信复杂度对比	35
表 4.1 标准半诚实模型下 MPSU 协议的理论与通信复杂度对比	37
表 4.2 GNT 协议与 PK-MPSU 协议在离线和在线阶段的理论与通信复杂度对比	45
表 4.3 SK-MPSU、PK-MPSU 和 LG 协议在 LAN 与 WAN 环境下的在线 / 总运行时间	47
表 4.4 SK-MPSU、PK-MPSU 和 LG 协议在 LAN 与 WAN 环境下的在线 / 总通信开销	48
表 6.1 半诚实模型下 MPSI 协议的渐近在线通信与计算复杂度	86
表 6.2 半诚实模型下 MPSI 协议的渐近离线通信与计算复杂度	86
表 6.3 半诚实模型下 MPSI-card / MPSI-card-sum 协议的渐近在线通信与计算复杂度	87
表 6.4 半诚实模型下 MPSI-card / MPSI-card-sum 协议的渐近离线通信与计算复杂度	87
表 6.5 半诚实模型下 MPSU 协议的渐近在线通信与计算复杂度	87
表 6.6 半诚实模型下 MPSU 协议的渐近离线通信与计算复杂度	87
表 6.7 各 MPSI 协议的运行时间与通信开销	91
表 6.8 各 MPSI-card 协议的运行时间与通信开销对比	91
表 6.9 本文的 MPSI-card-sum 协议的运行时间与通信开销	91
表 6.10 各 MPSU 协议的运行时间与通信开销对比	91

1 绪论

1.1 研究背景与意义

随着大数据、人工智能和云计算技术的飞速发展，数据已演变为驱动社会创新与经济增长的关键生产要素。政府部门、金融机构、医疗系统以及互联网企业积累了海量的高价值数据，通过跨机构的数据融合与联合分析，可以挖掘出更深层次的数据价值，赋能精准医疗、金融风控、联合营销等应用场景。然而，在数据共享的过程中，“数据孤岛”与“隐私保护”之间的矛盾日益凸显。

一方面，数据孤岛现象严重阻碍了数据价值的释放。出于商业机密保护或行政壁垒的考量以及对数据流失的担忧，各数据持有方往往不愿或不能将其私有数据直接对外开放。另一方面，日益严格的隐私保护法律法规对数据流通提出了极高的合规要求。欧盟的《通用数据保护条例》(GDPR)、中国的《中华人民共和国数据安全法》和《中华人民共和国个人信息保护法》等均明确规定，个人信息处理必须遵循合法、正当、必要的原则，且严厉限制了未经授权的数据共享行为。如何在严格保护各方数据隐私及主权的前提下，实现数据的“可用不可见”，已成为学术界和产业界共同关注的焦点。

在这一背景下，安全多方计算 (Secure Multi-party Computation, MPC) 技术应运而生。作为密码学的一个重要分支，MPC 允许互不信任的多个参与方在不泄露各自私有输入的前提下，协同计算某个约定函数的输出。姚期智院士在 1982 年提出的“百万富翁问题”奠定了 MPC 的理论基础 [4]，随后 GMW 协议 [5] 和 BGW 协议 [6] 进一步完善了其通用框架。

尽管通用 MPC 协议在理论上能够计算任意函数，但在处理大规模数据集的实际应用中，将其编译为布尔电路或算术电路往往会带来巨大的计算和通信开销，难以满足高频、海量数据场景下的实时性需求。因此，针对特定功能设计高效的**专用 MPC 协议**成为了该领域的重要研究方向。在众多专用协议中，**隐私集合运算 (Private Set Operations, PSO)** 因其在现实场景中的广泛适用性而备受瞩目，以 Google、Facebook、Microsoft 等为代表的科技公司投入了大量资源研究 PSO 技术。PSO 允许参与方在各自持有私有集合的前提下，协同完成对私有集合的计算，且不泄露任何额外信息。根据参与方的数量，PSO 可划分为**两方隐私集合运算**和**多方隐私集合运算 (Multi-party Private Set Operations, MPSO)**。

两方 PSO 因只涉及两个输入集合 X, Y ，通常只考虑计算交集 $X \cap Y$ 和并集 $X \cup Y$ ，功能相对局限。具体地，两方 PSO 主要包括以下三类功能：

- **两方隐私集合求交 (Private Set Intersection, PSI)**：允许持有隐私集合的双方共同计算交集 $X \cap Y$ 。该技术已广泛应用于隐私保护下的联系人发现 [7]、广告转化率归因 [8] 等场景。

- **两方隐私交集计算 (Private Computation on Set Intersection, PCSI)** . 在不泄露交集的前提下, 允许持有隐私集合的双方共同计算交集 $X \cap Y$ 的部分或汇总信息 (如基数或关联数据之和)。这在需要统计分析但严禁泄露个体身份的场景 (如打击儿童性虐待材料传播 [9]、疫情流调统计 [10]) 中至关重要。
- **两方隐私集合求并 (Private Set Union, PSU)** . 允许持有隐私集合的双方共同计算并集 $X \cup Y$ 。常用于两家机构联合构建全量的 IP 黑名单库或漏洞数据库 [11]。

目前, 两方 PSO 技术已相对成熟。特别是两方 PSI 取得了巨大进展 [12, 13, 14, 15, 16, 17, 18], 现有最先进的协议 [18] 已能实现与不安全的朴素哈希 PSI 方案相当的性能。近年来, 两方 PCSI 和两方 PSU 也迅猛发展, 出现了一系列高效且安全的两方 PCSI [19, 20] 和两方 PSU [21, 19, 22, 23, 20, 24] 协议构造, 并形成了统一的两方 PSO 框架 [20], 开始在实际业务中逐步大规模落地。

与两方场景不同, MPSO 涉及多个输入集合 $X_1, \dots, X_m$ ($m > 2$)。由于集合运算的组合方式呈指数级增长, 理想条件下的 MPSO 协议应该能够计算由有限次二元集合运算 (包括交集、并集和差集) 组成的任意集合公式 $f(X_1, \dots, X_m)$ 。¹这使得 MPSO 涵盖了更加丰富的功能。然而, 目前 MPSO 的研究主要聚焦于以下四类特定功能:

- **多方隐私集合求交 (Multi-party Private Set Intersection, MPSI)** . 允许 m ($m > 2$) 个参与方分别持有隐私集合 X_i ($1 \leq i \leq m$), 最终准确地共同计算出所有集合的交集 $\bigcap_{i=1}^m X_i$ 。其典型应用包括隐私保护下的位置信息共享 [7]、隐私通讯录的公共联系人发现 [25]、DNA 检测和模式匹配 [26]、以及联合僵尸网络检测等。
- **多方隐私交集计算 (Multi-party Private Computation on Set Intersection, MPCSI)** . 允许 m 个参与方分别持有隐私集合 X_i , 最终输出交集 $\bigcap_{i=1}^m X_i$ 的部分或汇总信息。MPCSI 主要包括基于电路的 MPSI 协议 (Circuit-MPSI)、多方隐私交集求势协议 (MPSI-card) 和多方隐私交集求势与和协议 (MPSI-card-sum)。其中, Circuit-MPSI 允许参与方在交集上计算任意函数 $f(\bigcap_{i=1}^m X_i)$; MPSI-card 用于计算交集的基数; MPSI-card-sum 用于计算交集基数及对应权值的总和。这类功能被广泛应用于在线广告效果评估 [8]、打击儿童性虐待材料 (CSAM) 传播 [9] 以及 COVID-19 隐私接触者追踪 [10, 27, 28]。
- **多方隐私集合求并 (Multi-party Private Set Union, MPSU)** . 允许 m 个参与方分别持有隐私集合 X_i , 最终准确地共同计算出所有集合的并集 $\bigcup_{i=1}^m X_i$ 。该功能是信息安全风险评估 [29]、IP 黑名单和漏洞数据聚合 [11]、联合图计算 [30]、分布式网络监控 [31] 的核心技术, 也是构建支持全连接的隐私数据库 [21] 和 Private ID 协议 [19] 的核心组件。

¹在 MPSO 的设定中, 通常指定一名领导者 (Leader) 获得结果集合, 而其他参与方 (Client) 无法获取除自身输入外的任何信息。

• **多方隐私并集计算(Multi-party Private Computation on Set Union, MPCSU)**

· 允许 m 个参与方分别持有隐私集合 X_i ，最终输出并集 $\bigcup_{i=1}^m X_i$ 的部分或汇总信息。MPCSU 主要包括基于电路的 MPSU 协议 (Circuit-MPSU) 和多方隐私并集求势协议 (MPSU-card)。该类功能同样存在许多潜在应用，例如，政府卫生部门可以通过 MPSU-card 协议统计某月内所有医院确诊某类传染病的总人数 (即并集基数)，而在统计过程中不泄露任何患者的具体身份。

虽然多方场景赋予了 MPSU 更加广阔的应用前景，但其复杂性导致了两方场景下的成熟技术无法直接迁移，进一步增加了 MPSU 协议设计的难度与挑战。具体地，MPSU 领域面临着以下的重大挑战：

1. **安全性弱**. 许多现有的 MPSU 协议 (甚至是部分最新的研究成果, 如 [1, 32, 33, 34]) 的安全性依赖于“非共谋假设” (Non-collusion Assumption)，即假设某些参与方之间不会共谋。该假设在现实世界的开放网络环境或存在利益冲突的商业合作中难以成立，导致这些协议难以抵御实际场景中的任意共谋攻击，一旦落地存在严重的安全风险。
2. **效率低下**. 现有的 MPSU 协议的构造主要遵循两类技术路线，包括基于公钥密码技术的路线和基于不经意传输 (Oblivious Transfer, OT) 与对称密钥技术的路线。前者由于涉及大量昂贵的公钥操作，实际运行效率极低；后者虽然执行速度快，但渐进复杂度较差。目前的各类 MPSU 协议尚未能做到统筹兼顾，难以在保证理论最优性的同时实现实际可用的性能。
3. **功能局限**. 现有的 MPSU 协议功能单一，缺乏对计算任意集合公式的支持，难以满足现实场景中的复杂需求。例如，公共卫生部门希望筛选某项健康计划的候选人，条件是“患有特定疾病”且“收入低于特定标准”。这需要各医院首先联合计算患病人员的**并集**，随后与福利部门的低收入名单计算**交集** [31]。这种“先求并、再求交”的复合运算超出了现有仅关注单一功能的 MPSU 协议的能力范畴，缺乏实用的专用解决方案。
4. **缺乏统一性**. 现有的 MPSU 研究呈现出严重的“功能孤岛”特征。研究工作大多聚焦于某一特定功能 (如仅针对 MPSI 或仅针对 MPSU) 的优化，并采用了异构的技术路线，导致产生了大量孤立的协议，缺乏一个统一的理论框架。这种现状不仅增加了系统的开发与维护成本，也使得难以通过模块化组合来应对复杂多变的业务需求。

针对上述背景与挑战，本文致力于研究安全、高效的多方隐私集合运算协议，并构建支持任意集合公式计算的多方隐私集合运算统一框架，从而解决现有 MPSU 领域安全性弱、效率低下、功能局限及缺乏统一性等关键问题，为打破数据孤岛、促进数据要素的安全流通提供坚实的技术支撑。

1.2 研究现状

据粗略估计, 多方隐私集合运算 (MPSO) 领域的相关研究文献已达数百篇, 且正以指数级的速度增长。然而, 在这海量的文献中, 大部分工作并未给出严谨的安全性证明。即便着眼于那些提供了安全性证明的文献, 许多协议在安全性假设或正确性保障上仍存在显著局限。例如, 部分协议 [35, 36, 37, 38, 1, 32, 33, 34] 引入了云服务器辅助 (Server-Aided) 模型或依赖于“特定参与方不共谋”的非共谋假设, 这使其难以抵御现实应用场景中复杂的任意共谋攻击; 另一些协议 [39, 40] 则存在不可忽略的假阳性率 (False Positives), 即以不可忽略的概率错误地将原本不属于结果集合的元素包含在协议输出中, 这在金融风控误报、医疗误诊等高敏感场景下是不可接受的。

鉴于此, 本文将重点关注那些在标准半诚实模型下安全 (即能够抵抗任意参与方的共谋)、且协议输出的错误概率可忽略的 MPSO 协议。同时, 为了梳理技术演进脉络, 本文也会回顾部分虽然未完全达到标准半诚实安全, 但其构造思想对后续工作产生深远影响的经典协议。

目前 MPSO 的研究分布呈现出极端的不均衡性: 多方隐私集合求交 (MPSI) 占据 90% 以上的比例, 得到了广泛的研究 [41, 31, 42, 43, 44, 45, 46, 47, 2]; 多方隐私集合求并 (MPSU) 受到的关注相对稀缺 [31, 48, 49]。不过, 随着近年来 Liu 和 Gao [1] 等高影响力工作的发表, 该方向的研究热度正出现上涨趋势。至于多方隐私交集计算 (MPCSI) 和多方隐私并集计算 (MPCSU), 相关的安全协议寥寥无几 [31, 3, 50]。特别是在不诚实大多数 (Dishonest Majority)² 设定下, 学术界甚至尚未有工作能够实现 Circuit-MPSI、MPSU-card 以及 Circuit-MPSU 协议。

下面我们将分类详细阐述各类协议的研究现状。

1.2.1 多方隐私集合求交

作为 MPSO 中研究最为深入的功能, MPSI 协议主要遵循两条技术路线: 基于公钥密码技术的路线和基于对称密钥技术的路线。

基于公钥密码技术的 MPSI. Freedman 等人 [41] 基于不经意多项式求值 (Oblivious Polynomial Evaluation, OPE) 提出了首个 MPSI 协议, 其中 OPE 利用加法同态加密 (Additively Homomorphic Encryption, AHE) 实现。Kissner 和 Song [31] 同样利用 OPE 技术结合多项式表示提出了一个 MPSI 协议。然而, 这两个协议的计算复杂度对于每个参与方而言均为集合大小 n 的二次方, 导致其实际效率低下, 根本难以应用于大规模数据集。

²不诚实大多数要求敌手腐化参与方数量的阈值 t 满足 $t \geq m/2$, 而标准半诚实安全要求 $t = m - 1$ (即最大腐化阈值)。安全性依赖于非共谋假设的协议不满足半诚实安全定义, 但往往能实现不诚实大多数。

基于对称密钥技术的 MPSI. 近年来, 基于不经意传输 (OT) 和对称密钥操作的 MPSI 协议因其实际可用的效率成为了研究主流。Kolesnikov 等人 [43] 提出了两个 MPSI 协议: 第一个协议实现了最优渐进复杂度,³但仅在增强半诚实模型 (Augmented Semi-Honest Model) 下安全。(增强半诚实模型的安全性要弱于标准半诚实模型, 关于两者的关系详见文献 [51] 的 2.4.4 节)。第二个协议虽然在标准半诚实模型下安全, 但 Client 的复杂度依赖于腐化阈值 t , 未能达到最优。

Garimella 等人 [45] 通过提出并优化不经意键值存储 (Oblivious Key-Value Store, OKVS) [16, 18, 52] 对上述协议的效率进行了改进, 并证明了第一个协议实际上支持恶意安全。在此基础上, Nevo 等人 [46] 进一步优化了 $t < m - 1$ 设定下的恶意 MPSI 协议。Inbar 等人 [44] 基于 OT 和混淆布隆过滤器 (Garbled Bloom Filter), 分别在增强半诚实和标准半诚实模型下各提出了一个 MPSI 协议, 但其每个参与方的计算复杂度均为 $O(mn)$ 。Ben-Efraim 等人 [47] 随后将其扩展至恶意安全模型。

最近, Wu 等人 [2] 基于不经意伪随机函数 (OPRF) 和 OKVS 提出了两个满足标准半诚实安全的 MPSI 协议, 展现出了优于前人工作的性能, 但其 Client 的复杂度依然依赖于腐化阈值 t , 并非最优。

综上所述, 尽管 MPSI 领域的研究已相当成熟, 但目前尚无满足标准半诚实安全的基于对称密钥技术的 MPSI 协议能够达到与 [43] 中增强半诚实协议相同的最优渐进复杂度。

1.2.2 多方隐私交集计算

MPCSI 旨在计算交集的统计信息而非交集本身。目前仅有极少数工作关注了其中的 MPSI-card 和 MPSI-card-sum 这两类功能。

多方隐私交集求势 / 求势与和. Kissner 和 Song [31] 的方案虽然在理论上支持 MPSI-card 功能, 但由于依赖昂贵的同态加密, 其构造效率低下, 仅具备理论价值。Chen 等人 [3] 提出了首个基于 OT 和对称密钥操作的 MPSI-card 和 MPSI-card-sum 协议, 这也是目前在标准半诚实模型下唯一实用的同类协议。然而, 其协议复杂度尚未达到最优, 其中 Leader 的复杂度为 $O(mn + tn \log n)$, Client 的复杂度为 $O(tn)$ (t 为腐化阈值)。

综上所述, 目前尚无满足标准半诚实安全的 MPSI-card 和 MPSI-card-sum 协议能够实现最优渐进复杂度, 且现有方案的实际运行效率仍有较大的提升空间。此外, 目前学术界尚缺乏满足标准半诚实安全性的 Circuit-MPSI 协议。

³在 MPSI 和 MPCSI 中, 最优渐进复杂度定义为 Leader 的计算和通信复杂度均为 $O(mn)$, 且每个 Client 的计算和通信复杂度均为 $O(n)$ (其中 n 为集合大小, m 为参与方数量)。这是因为在不考虑隐私的朴素方案中 (即各 Client 直接发送集合给 Leader, 由 Leader 直接计算出结果集合), 所需的复杂度即为此量级。

1.2.3 多方隐私集合求并

与 MPSI 相比, MPSU 的研究相对滞后且挑战更大。现有的 MPSU 工作同样主要分为基于公钥密码技术和基于对称密钥技术两类。

基于公钥密码技术的 MPSU. Kissner 和 Song [31] 基于多项式表示和加法同态加密 (AHE) 提出了首个 MPSU 协议。但由于涉及大量的 AHE 操作和高次多项式计算, 该协议完全不具备实用性。Frikkén [48] 通过降低多项式次数改进了 [31], 但该协议仍需对加密多项式进行多点求值, 导致 AHE 的操作次数仍高达 $O(n^2)$, 其实际效率难以满足大规模应用需求。

Vos 等人 [40] 提出了一种基于位向量表示和 ElGamal 加密 [53] 的 MPSU 协议, 通过对位向量执行隐私 OR 操作来计算并集。然而, 据 Liu 和 Gao [1] 的实验报告, 该协议的具体效率较差 (例如, 在 10 个参与方、每方集合大小为 2^{10} 的规模下, 运行时间长达 75 秒), 且 Leader 的计算和通信复杂度随参与方数量 m 呈二次增长。

最近, Gao 等人 [49] 基于多密钥重随机化公钥加密 (Multi-Key Rerandomizable Public-Key Encryption, MKR-PKE) 提出了一种渐进复杂度较优的 MPSU 协议。尽管该协议是目前渐进复杂度最佳的 MPSU 协议, 但其仍未能达到线性复杂度。⁴

基于对称密钥技术的 MPSU. Liu 和 Gao [1] 提出了首个 (也是目前唯一一个) 基于 OT 和对称密钥操作的 MPSU 协议。该协议在性能上取得了巨大突破, 是首个实际可用的 MPSU 协议。在 3 方参与, 每个参与方的集合大小为 2^{10} 的设定下, 该协议比 [40] 快了 109 倍。然而, 该协议的安全性依赖于“非共谋假设”, 无法在标准半诚实模型下证明安全。

此外, Blanton 等人 [36] 基于不经意排序和通用 MPC 构造了 MPSU, 但由于严重依赖通用 MPC, 效率极低。

综上所述, 目前尚不存在基于对称密钥技术且满足标准半诚实安全的 MPSU 协议, 现有方案在安全性与实用性之间存在权衡。此外, 在理论层面, 目前也未有 MPSU 协议能够实现严格的线性复杂度。

1.2.4 多方隐私集合运算框架

构建能够支持任意集合公式计算 (后文中我们统称该功能为通用 MPSO 功能) 的 MPSO 框架是该领域长期以来一个开放性问题。Kissner 和 Song [31] 在 MPSO 发展的早期阶段就尝试构建一个 MPSO 框架, 但该框架并未能完全实现通用 MPSO 功能。其方案仅支持计算由交集和并集组成的集合公式, 无法处理差集运算 (例如计算

⁴在 MPSU 的设定中, 线性复杂度意味着每个参与方的复杂度与所有参与方集合大小的总和呈线性关系。本文考虑平衡设定, 即每个参与方持有的集合大小相等, 因此线性复杂度指每个参与方的复杂度与参与方数量 m 和集合大小 n 均呈线性关系。

$X_2 \setminus (X_1 \cap X_3)$), 且严重依赖于加法同态加密, 计算成本极高。Blanton 和 Aguiar [36] 通过重新设计集合运算电路, 并结合通用 MPC 协议, 实现了对交、并、差运算的任意组合的计算支持。尽管该方案依靠通用 MPC 技术在功能上实现了通用性, 但这也带来了巨大的计算开销。例如, 其实验报告中仅针对 3 个参与方、每方 2^{11} 个元素的最基础 MPSI 任务, 耗时就长达 24.8 秒, 且仅支持诚实大多数 (Honest Majority) 设定。

综上所述, 目前学术界尚缺乏一个在标准半诚实模型下, 既能完全实现通用 MPSO 功能 (支持任意集合公式计算), 又具备实际可用效率的统一框架。

1.3 本文主要贡献

针对目前多方隐私集合运算 (MPSO) 领域在安全性、效率、功能完备性及统一性方面存在的显著不足, 以及研究资源在不同功能间分布极度不均衡的现状, 本文开展了系统性的研究工作, 主要研究内容和贡献总结如下:

1.3.1 多方隐私集合求并协议的研究

首先, 本文从应用价值巨大但研究相对匮乏的多方隐私集合求并 (MPSU) 入手, 直面现有 MPSU 协议面临的“安全性弱” (依赖于非共谋假设) 与“效率低下” (实际性能难以支撑应用或理论复杂度未达线性) 两大核心痛点, 分别基于对称密钥和公钥体制提出了安全高效的解决方案: 前者是首个在标准半诚实模型下证明安全的实际可用的 MPSU 协议, 后者则是首个在理论上实现线性复杂度的 MPSU 协议。

具体创新点和贡献如下:

1. **高效的批量秘密分享隐私成员测试 (batch ssPMT) 原语.** 本文深入分析了现有唯一的基于对称密钥技术的 MPSU 协议 [1], 并针对其核心组件——多查询秘密分享隐私成员测试 (multi-query secret-shared private membership test, mq-ssPMT) 协议, 抽象出一种全新的原语——批量秘密分享隐私成员测试 (batch secret-shared private membership test, batch ssPMT)。相比于 mq-ssPMT 的实例化严重依赖于昂贵的通用 MPC 技术所导致的性能瓶颈, 本文提出的 batch ssPMT 能够仅利用轻量级密码学组件进行高效实例化。通过结合哈希分桶 (hashing-to-bins) 技术, 该原语能够在不引入任何额外信息泄露的前提下, 完全替代现有基于对称密钥技术的 MPSU 框架中的 mq-ssPMT 组件。该原语不仅构成了本文两个 MPSU 协议的核心构建模块, 其高效的构造也是协议实际性能获得显著提升的关键所在。此外, 该原语在本文后续构建的 MPSO 框架中也发挥着关键作用, 被用于高效构造其核心共性子协议——成员零分享 (Membership Zero-Sharing) 协议, 从而为任意集合公式的计算提供了底层支撑。

2. **首个标准半诚实模型下基于对称密钥的 MPSU 协议.** 本文将随机不经意传输 (Random Oblivious Transfer, ROT) 的概念推广至多方场景, 定义并构造了多方秘密分享随机不经意传输 (multi-party secret-shared ROT, mss-ROT) 原语。基于 batch ssPMT 和 mss-ROT, 本文提出了首个在标准半诚实模型下证明安全且基于对称密钥技术的 MPSU 协议。该协议不仅成功消除了现有基于对称密钥的方案对非合谋假设的依赖, 显著增强了安全性, 还表现出更加优异的具体性能。特别是在局域网 (LAN) 环境下, 其在线阶段的运行效率比最先进的 (State-of-the-Art, SOTA) 协议 [1] 提升了 3.9 ~ 10.0 倍, 整体运行效率提升了 1.2 ~ 7.8 倍。
3. **首个具有线性复杂度的 MPSU 协议.** 基于 batch ssPMT 和多密钥重随机化公钥加密 (Multi-Key Rerandomizable Public-Key Encryption, MKR-PKE) [54], 本文提出了首个同时实现线性计算复杂度和线性通信复杂度的 MPSU 协议。得益于其优越的渐进复杂度, 该协议极大地降低了通信开销。实验数据显示, 相比于 SOTA 协议 [1], 该协议的总通信量降低了 3.0 ~ 36.5 倍, 使其在带宽受限的广域网 (WAN) 环境下具有显著的应用优势。

1.3.2 多方隐私集合运算框架的研究

为了突破 MPSO 领域“功能局限”与“缺乏统一性”的瓶颈, 本文随后聚焦于支持任意集合公式计算的通用 MPSO 功能, 在标准半诚实模型下, 利用不经意传输 (OT) 与对称密钥技术, 构建了首个能够高效实现通用 MPSO 功能的统一框架, 并将该框架进一步扩展, 实现了更为复杂的任意集合公式结果求势 (MPSO-card) 及基于电路的通用 MPSO (Circuit-MPSO) 功能。

具体创新点和贡献如下:

- **任意集合公式的统一谓词公式表示.** 为了解决现有 MPSO 框架在表示层面的两极化难题——要么将目标集合公式表示为交、并及元素规约 (element reduction) 操作的组合导致通用性受限 [31], 要么直接基于集合代数运算的组合导致协议严重依赖于通用 MPC 从而效率低下 [36], 本文引入了一种名为“规范集合谓词公式” (Canonical Set Predicate Formula, CSPF) 的新型表示方法, 将目标集合公式表示为一类由原子集合谓词 (形如 $x \in X_i$ 或 $x \notin X_i$) 构成的析取范式, 其中每个子公式满足特定的结构, 子公式共同表示了目标集合的一个划分。本文严格证明了任意集合公式均可转化为 CSPF 表示, 且 CSPF 的结构特性 (如子公式数量) 直接决定了协议的性能。
- **谓词零分享 (Predicative Zero-Sharing) 与其松弛定义.** 本文提出了一类新型的密码学原语“谓词零分享” (Predicative Zero-Sharing)。该原语表示一族协议, 其中每个协议均关联一个谓词公式, 如果该公式在所有参与方输入上的真值为真,

那么各方输出 0 的加法秘密分享；否则，输出随机值的加法秘密分享。本文首先给出了一种简化的、基于模拟范式的谓词零分享安全性定义，并严格证明了其与标准半诚实安全定义的等价性。基于此定义，本文进一步提出了谓词零分享的松弛版本 (Relaxed Predicative Zero-Sharing)，该版本具有极强的抽象能力，能够涵盖随机不经意传输 (ROT)、等值条件随机数生成 (ECRG) [24] 等现有原语。在此基础上，本文建立了针对松弛谓词零分享的**组合技术 (Composition Technique)**与**转化技术 (Transformation Technique)**。结合这两项技术，可以以模块化的方式，从关联于原子命题（或其否定）的松弛谓词零分享出发，自底向上地构造出关联于任意复杂谓词公式的、满足标准半诚实安全的谓词零分享协议。这一构造范式是本文 MPSO 框架能够灵活支持任意集合公式计算、并保持协议结构统一性的核心技术支撑。

- **高效的成员零分享 (Membership Zero-Sharing) 原语.** 为了将谓词零分享这一抽象原语应用于具体的集合运算，本文引入了其在 MPSO 场景下的特化实例——“成员零分享” (Membership Zero-Sharing)。具体而言，该协议指定一名参与方（记作 P_{pivot} ）输入单个元素 x ，而其他每个参与方 P_i 输入集合 X_i 。每个协议实例均关联一个由原子集合谓词 $x \in X_i$ 和 $x \notin X_i$ 通过逻辑与 AND、逻辑或 OR 操作符连接而成的集合谓词公式。如果 P_{pivot} 的输入元素 x 使得该公式成立，则各方输出 0 的加法秘密分享；否则，输出随机值的加法秘密分享。基于不经意可编程伪随机函数 (oblivious programmable pseudorandom function, OPPRF)、批量秘密分享隐私成员测试 (batch ssPMT) 和随机不经意传输 (ROT) 等轻量级组件，本文分别针对 $x \in X_i$ 和 $x \notin X_i$ 给出了成员零分享的松弛版本 (Relaxed Membership Zero-Sharing) 的高效实例化。进而，遵循任意谓词零分享的通用构造范式，本文成功实例化了关联于任意集合谓词公式的标准安全成员零分享协议。该协议作为 MPSO 统一框架的核心底层组件，成功弥合了 MPSO 领域中“通用性”与“实用性”之间的鸿沟：其对任意集合谓词公式的支持赋予了框架计算任意集合运算的通用能力，而其基于轻量级组件的高效实例化则保障了框架优异的实际性能。
- **实现通用 MPSO 功能的统一框架及其扩展.** 基于上述集合公式的统一表示和成员零分享协议作为底层原语，本文构建了支持任意集合公式计算的 MPSO 统一框架：首先，利用 CPSF 表示法将目标集合公式转化为若干结构化的子公式；随后，针对每一个子公式，参与方以各自的隐私集合为输入，调用关联于该子公式的标准安全成员零分享协议，生成关于目标集合元素的加法秘密分享序列；为了消除该秘密分享顺序可能泄露的元数据信息（如元素来源），参与方调用多方秘密分享洗牌 (multi-party secret-shared shuffle) 协议对所有分享进行随机置换。为实现标准 MPSO 功能，打乱后的秘密分享将被直接发送给结果接收方进行重构，通过特定的编码格式筛除随机值后即可恢复出目标集合。此外，得益于洗牌后的输出仍保持秘密分享的形式，该框架具有极强的扩展性，首次实现了更为复杂的任意集

合公式结果求势 (MPSO-card) 及基于电路的通用 MPSO (Circuit-MPSO) 功能: 若应用场景仅需输出集合的统计信息或需进行后续保密计算, 参与方可直接对秘密分享进行操作 (如统计有效分享数量以计算基数, 或将秘密分享作为输入馈送至通用 MPC 电路计算关于结果集合的任意函数)。

除了上述对 MPSO 框架的研究本身做出的理论贡献外, 通过对该框架进行实例化, 本文在 MPSO 的各个具体子领域亦取得了突破性进展, 解决了一系列长期存在的开放性问题。具体贡献点如下:

- **多方隐私集合求交.** 基于本文框架实例化的 MPSI 协议是首个在标准半诚实模型下实现**最优渐进复杂度** (即达到与不考虑隐私的明文传输方案相同量级的复杂度——Leader 的计算与通信复杂度为 $O(mn)$, Client 的计算与通信复杂度为 $O(n)$) 的基于对称密钥技术的 MPSI 构造。此前达到该复杂度的协议 [43] 仅在增强半诚实模型 (Augmented Semi-Honest) 下安全, 本文填补了这一空白。同时, 该协议也是目前在线效率最高的 MPSI 协议, 在 LAN / WAN 环境下比 SOTA 协议 [2] 快 $2.4 \sim 5.2 / 1.1 \sim 2.6$ 倍。
- **多方隐私交集计算.** 基于本文框架实例化的 MPSI-card 和 MPSI-card-sum 协议是首个在标准半诚实模型下实现**最优渐进复杂度**的 MPSI-card 和 MPSI-card-sum 协议。其中, MPSI-card 协议也是目前在线阶段最高效的同类协议, 其在线通信量比 SOTA [3] 降低了 $14.0 \sim 20.3$ 倍; MPSI-card-sum 协议是目前唯一一个给出了具体实现的同类协议, 实验表明其计算与通信开销仅约为前述 MPSI 协议的两倍。此外, 基于本框架实例化的 Circuit-MPSI 协议是首个在**不诚实大多数 (Dishonest Majority)** 设定下安全的 Circuit-MPSI 构造, 突破了以往工作仅限于诚实大多数 (Honest Majority) 安全的局限。
- **多方隐私集合求并.** 基于本文框架实例化的 MPSU 协议是标准半诚实模型下基于对称密钥的另一高效 MPSU 方案。尽管其实际计算性能略逊于本文第三章提出的 MPSU 协议, 但其在理论上实现了**更优的渐进复杂度**, 并且其在线通信开销降低了 1.8 倍, 因此在带宽受限网络中具有更强的竞争力。
- **多方隐私并集计算.** 基于本文框架实例化的 MPSU-card 和 Circuit-MPSU 协议是目前**唯一可用**的 MPSU-card 和 Circuit-MPSU 构造, 其性能与该框架下实例化的 MPSU 协议基本相同。

最后, 作为对基于对称密钥的 MPSO 框架的补充, 本文在第六章提出了一个**基于公钥体制的 MPSO 框架**, 同样实现了通用的 MPSO 功能 (该框架的扩展能力略弱于对称密钥版本, 可实现 MPSO-card 功能但无法实现 Circuit-MPSO 功能)。尽管受限于公钥密码操作昂贵的计算开销, 该框架实际运行效率较低, 但其在渐进复杂度上相比对

称密钥版本具有独特的理论优势。这一工作与对称密钥框架互为补充，共同构建了涵盖实用性能与理论边界的完整 MPSO 技术体系。

1.4 本文组织结构

本文共分为七章，各章节的组织结构安排如下：

第一章 绪论：阐述了本文的研究背景与意义，系统地梳理了隐私集合运算领域研究现状，总结了现有技术面临的难题和挑战，并介绍了本文的主要研究内容与创新贡献。

第二章 预备知识：定义了本文使用的符号系统，详细描述了标准半诚实安全模型及基于模拟范式的安全性定义。同时，回顾了不经意传输、秘密分享、伪随机函数、哈希分桶等本文后续章节所需的基础密码学组件。

第三章 基于对称密钥的高效多方隐私集合求并协议：针对现有高效 MPSU 协议依赖非合谋假设的缺陷，提出了一种基于不经意传输和对称密钥操作的 MPSU 协议。本章详细描述了核心原语“批量秘密分享隐私成员测试”与“多方秘密分享随机不经意传输”的构造，给出了协议的具体流程与安全性证明，并通过实验展示了其在局域网环境下的性能优势。

第四章 基于公钥体制的线性多方隐私集合求并协议：针对广域网环境下通信瓶颈问题，提出了一种具有线性通信复杂度的 MPSU 协议。本章详细阐述了基于多密钥重随机化公钥加密的协议构造，重点分析了其在渐进复杂度上的理论突破，并通过实验对比验证了其在低带宽环境下的优越性。

第五章 基于对称密钥的通用多方隐私集合运算框架：为了解决协议功能单一的问题，本章构建了首个支持任意集合公式计算的 MPSO 统一框架。本章引入了“规范集合谓词公式”表示法和“谓词零分享”原语，详细描述了从底层原语到通用框架的自底向上构造过程，并给出了 MPSI、MPCSI 及 MPSU 等多种功能的实例化协议及其性能评估。

第六章 基于公钥体制的通用多方隐私集合运算框架：作为对第五章的理论补充，本章探讨了基于公钥体制的通用框架构造。本章引入了“谓词加密零”原语，构建了相应的通用框架，并重点讨论了其在渐进复杂度上的理论特性，从而与对称密钥框架共同构成了完整的技术体系。

第七章 总结与展望：对全文的研究成果进行了系统总结，并结合当前领域的发展趋势，对多方隐私集合运算未来的研究方向（如恶意模型下的安全性扩展、非平衡集合场景的优化等）进行了展望。

2 预备知识

2.1 符号说明

令 m 表示参与方的数量。本文使用 P_i ($1 \leq i \leq m$) 表示各参与方, X_i 表示其持有的集合, 其中每个集合包含 n 个 l 比特元素。用 $[x] = (x_1, \dots, x_m)$ 表示在 m 个参与方之间的加法秘密分享 (additive secret-sharing), 即每个 P_i 持有一个分享份额 x_i , 满足 $x_1 + \dots + x_m = x$ 。符号 $\parallel$ 表示两个字符串的拼接。 λ 和 σ 分别表示计算安全参数和统计安全参数。符号 $\stackrel{s}{\approx}$ 和 $\stackrel{c}{\approx}$ 分别表示两个分布在统计上不可区分和计算上不可区分。对于向量 $\mathbf{a}$, a_i 表示其第 i 个分量, $\pi(\mathbf{a}) = (a_{\pi(1)}, \dots, a_{\pi(n)})$ 表示对向量元素按照 π 进行置换, 其中 π 是 n 个元素上的一个置换。记号 $\mathbf{a} \oplus \mathbf{b}$ 表示按分量异或, 即 $(a_1 \oplus b_1, \dots, a_n \oplus b_n)$ 。

2.2 安全模型

本文考虑**半诚实 (Semi-honest)** 且**静态 (Static)** 的敌手模型。敌手 $\mathcal{A}$ 可以腐化任意 $t < m$ 个参与方的子集。半诚实敌手 (也称为“诚实但好奇”的敌手) 会严格按照协议规定执行协议步骤, 但在协议执行过程中会记录其视图, 并试图从视图中推断出其他诚实参与方的私有输入信息。

令 $f = (f_1, \dots, f_m)$ 为一个概率多项式时间 (probabilistic polynomial-time, PPT) 的 m 元功能函数 (functionality), Π 是一个用于计算 f 的 m 方协议。在协议 Π 执行过程中, 参与方 P_i ($1 \leq i \leq m$) 的视图记为 $\text{View}_i^\Pi(\mathbf{x})$, 其中 $\mathbf{x} = (x_1, \dots, x_m)$ 为所有参与方的输入。协议结束时 P_i 的输出记为 $\text{Output}_i^\Pi(\mathbf{x})$ 。所有参与方的联合输出记为 $\text{Output}^\Pi(\mathbf{x}) = (\text{Output}_1^\Pi(\mathbf{x}), \dots, \text{Output}_m^\Pi(\mathbf{x}))$ 。为了刻画协议的安全性, 本文采用基于模拟的 (simulation-based) 的定义 [55, 56]:

定义 2.1 (半诚实安全). 若存在一个 PPT 算法 Sim , 使得对于任意被腐化的参与方集合 $P_{\mathcal{A}} = \{P_{i_1}, \dots, P_{i_t}\} \subset \{P_1, \dots, P_m\}$, 以下两个分布簇在计算上不可区分:

$$\{\text{Sim}(P_{\mathcal{A}}, \mathbf{x}_{\mathcal{A}}, f_{\mathcal{A}}(\mathbf{x})), f(\mathbf{x})\}_x \stackrel{c}{\approx} \{\text{View}_{\mathcal{A}}^\Pi(\mathbf{x}), \text{Output}^\Pi(\mathbf{x})\}_x,$$

则称协议 Π 在敌手 $\mathcal{A}$ 存在的情况下安全地计算了功能 f 。

其中, $\mathbf{x}_{\mathcal{A}} = (x_{i_1}, \dots, x_{i_t})$ 表示被腐化方的输入, $f_{\mathcal{A}} = (f_{i_1}, \dots, f_{i_t})$ 表示被腐化方的输出, $\text{View}_{\mathcal{A}}^\Pi(\mathbf{x}) = (\text{View}_{i_1}^\Pi(\mathbf{x}), \dots, \text{View}_{i_t}^\Pi(\mathbf{x}))$ 表示敌手的联合视图。

确定性功能的简化定义: 当功能 f 是确定性 (deterministic) 函数时, 上述定义可以简化为仅比较模拟器模拟的敌手视图和敌手在真实协议执行中的视图的分布 (无需考虑联合输出分布)。具体而言, 安全性定义包含以下两个要求:

- **正确性 (Correctness)**: 存在一个可忽略函数 negl , 使得对于任意输入 $\mathbf{x}$, 满足:

$$\Pr[\text{Output}^\Pi(\mathbf{x}) = f(\mathbf{x})] \geq 1 - \text{negl}(\sigma).$$

- **隐私性 (Privacy)**: 存在一个 PPT 算法 Sim , 使得对于任意 $\mathbf{P}_A \subset \{P_1, \dots, P_m\}$, 满足:

$$\{\text{Sim}(\mathbf{P}_A, \mathbf{x}_A, f_A(\mathbf{x}))\}_{\mathbf{x}} \stackrel{c}{\approx} \{\text{View}_A^\Pi(\mathbf{x})\}_{\mathbf{x}}.$$

2.3 多方隐私集合运算

多方隐私集合运算 (Multi-party Private Set Operations, MPSO) 是多方安全计算 (Multi-Party Computation, MPC) 的一个特例。图 2.1 形式化定义了本文涉及的 MPSO 中的典型理想功能, 包括计算参与方私有集合上的交集、交集基数、带基数的交集求和、并集以及并集基数。

参数: m 个参与方 $P_1, \dots, P_m$, 其中 P_1 为 Leader。输入集合大小为 n 。集合元素的比特长度为 l 。

功能: 收到来自 P_i 的输入 $X_i = \{x_i^1, \dots, x_i^n\} \subseteq \{0, 1\}^l$ 和 $V_i = \{v_i^1, \dots, v_i^n\}$ (假设存在一个映射函数 $\text{payload}_i()$ 将每个元素映射到其关联载荷),

- **MPSI**. 向 P_1 输出交集 $\bigcap_{i=1}^m X_i$ 。
- **MPSI-card**. 向 P_1 输出交集基数 $|\bigcap_{i=1}^m X_i|$ 。
- **MPSI-card-sum**. 对于 $1 \leq i \leq m$, 向每个 P_i 输出交集基数 $|\bigcap_{i=1}^m X_i|$, 并向 P_1 输出交集中负载之和 $\sum_{x \in \bigcap_{i=1}^m X_i, 1 \leq j \leq m} \text{payload}_j(x)$ 。
- **MPSU**. 向 P_1 输出并集 $\bigcup_{i=1}^m X_i$ 。
- **MPSU-card**. 向 P_1 输出并集基数 $|\bigcup_{i=1}^m X_i|$ 。

图 2.1 MPSO 中的典型功能

2.4 基本组件

本节将介绍构建本文协议所需的底层密码学原语。

2.4.1 随机不经意传输

不经意传输 (Oblivious Transfer, OT) [57] 是 MPC 中的基础原语。本文主要使用二选一随机不经意传输 (1-out-of-2 Random OT, ROT), 其理想功能 $\mathcal{F}_{\text{ROT}}$ 如图 2.2 所示。在具体实现中, 我们采用 SoftSpokenOT [58] 协议来实例化 ROT。

参数：发送方 $\mathcal{S}$ 。接收方 $\mathcal{R}$ 。有限域 $\mathbb{F}$ 。

功能：从 $\mathcal{R}$ 接收输入 $e \in \{0, 1\}$ ，采样 $r_0, r_1 \leftarrow \mathbb{F}$ 。向 $\mathcal{S}$ 输出 (r_0, r_1) ，向 $\mathcal{R}$ 输出 r_e 。

图 2.2 二选一随机不经意传输的理想功能 $\mathcal{F}_{\text{ROT}}$

2.4.2 不经意可编程伪随机函数

不经意伪随机函数 (Oblivious Pseudorandom Function, OPRF) [59, 15, 17] 是 PSO 领域的核心原语。Kolesnikov 等人 [13] 引入了批量不经意伪随机函数 (batch oblivious pseudorandom function, batch OPRF)，其提供了一批 OPRF 实例。在第 i 个实例中，发送方 $\mathcal{S}$ 获得一个 PRF 密钥 k_i ，而接收方 $\mathcal{R}$ 输入 x_i 并获得 $\text{PRF}(k_i, x_i)$ 。

不经意可编程伪随机函数 (Oblivious Programmable Pseudorandom Function, OP-PRF) [43, 60, 61, 17, 18] 是 OPRF 的扩展，其允许发送方 $\mathcal{S}$ 首先生成一个均匀随机密钥 k ，并根据 k 和给定的键值对集合生成一个“提示值” hint (提示生成算法记为 $\text{hint} \leftarrow \text{Hint}(k, K, V)$ ，其中 K 为键集合， V 为值集合)。利用 hint 对 PRF F 进行编程，使得 F 对于特定输入 (K 中的键) 输出特定的均匀值 (V 中的值)，而对其他输入则输出伪随机值。这种在特定输入集上输出编程值的 PRF 被称为可编程 PRF (Programmable PRF, PPRF) [60]。接收方 $\mathcal{R}$ 利用其输入和给定的 hint 计算 OP-PRF，但他无法区分自己获得的是编程后的输出还是伪随机值。批量版本的 OP-PRF (batch OP-PRF) 的理想功能 $\mathcal{F}_{\text{bOPPRF}}$ 如图 2.3 所示。

2.4.3 秘密分享隐私等值测试

秘密分享隐私等值测试 (Secret-Shared Private Equality Test, ssPEQT) [60, 61] 是一个两方协议，其中发送方 $\mathcal{S}$ 和接收方 $\mathcal{R}$ 各输入一个元素。若双方输入元素相同，则他们获得 1 的秘密分享，否则获得 0 的秘密分享。图 2.4 定义了该理想功能。

2.4.4 多方秘密分享洗牌

多方秘密分享洗牌是一个 m 方协议，其通过随机置换所有参与方的分享向量并刷新所有分享值，来确保该置换对于任意 $m - 1$ 个参与方的共谋都是保密的。该功能 $\mathcal{F}_{\text{shuffle}}$ 在图 2.5 中给出。Eskandarian 等人 [62] 提出了一种在线高效的协议，参与方在离线阶段生成分享相关性 (share correlations)，使得 Leader 的在线复杂度与 n 和 m 呈线性关系，而 Client 的在线复杂度与 n 呈线性关系且独立于 m 。

参数：发送方 S ，接收方 R 。批量大小 B 。键比特长度 l ，值比特长度 γ 。PPRF $F : \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^l \rightarrow \{0, 1\}^\gamma$ 。

发送方输入： S 输入 B 个键值对集合，包括：

- 不相交的键集合 $\mathbf{K} = (K_1, \dots, K_B)$ 。
- 值集合 $\mathbf{V} = (V_1, \dots, V_B)$ ，其中 $|K_i| = |V_i|, 1 \leq i \leq B$ 。

接收方输入： R 输入 B 个查询 $\mathbf{x} \subseteq (\{0, 1\}^l)^B$ 。

功能：在接收到来自 S 的 $\mathbf{K}, \mathbf{V}$ 和来自 R 的 $\mathbf{x}$ 后：

功能：在从 S 收到 $\mathbf{K}$ 与 $\mathbf{V}$ ，并从 R 收到 $\mathbf{x}$ 后，

- 生成均匀密钥 $\mathbf{k} = (k_1, \dots, k_B)$ ，并计算 $\text{hint} \leftarrow \text{Hint}(\mathbf{k}, \mathbf{K}, \mathbf{V})$ ，满足 $F(k_i, \text{hint}, K_i(j)) = V_i(j)$ ，其中 $1 \leq i \leq B, 1 \leq j \leq |K_i|$ ；
- 定义 $f_i = F(k_i, \text{hint}, x_i)$ ， $1 \leq i \leq B$ ；
- 将 $\mathbf{k}$ 发送给 S ，并将 $\mathbf{f} = (f_1, \dots, f_B)$ 以及 hint 发送给 R 。

图 2.3 批量不经意可编程伪随机函数的理想功能 $\mathcal{F}_{\text{bOPPRF}}$

参数：两个参与方 P_1, P_2 。输入比特长度 γ 。

功能：收到来自 P_1 的输入 x 和来自 P_2 的输入 y 后，采样两个随机比特 a, b ，使得若 $x = y$ ，则 $a \oplus b = 1$ ；否则 $a \oplus b = 0$ 。向 P_1 输出 a ，向 P_2 输出 b 。

图 2.4 秘密分享隐私等值测试的理想功能 $\mathcal{F}_{\text{ssPEQT}}$

参数： m 个参与方 $P_1, \dots, P_m$ 。向量维度 n 。元素长度 l 。

功能：收到来自每个 P_i 的输入 $\mathbf{x}_i = (x_i^1, \dots, x_i^n)$ 后，采样一个随机置换 $\pi : [n] \rightarrow [n]$ 。对于 $1 \leq i \leq m$ ，采样 $\mathbf{x}'_i \leftarrow (\{0, 1\}^l)^n$ ，满足 $\bigoplus_{i=1}^m \mathbf{x}'_i = \pi(\bigoplus_{i=1}^m \mathbf{x}_i)$ 。向 P_i 输出 $\mathbf{x}'_i$ 。

图 2.5 多方秘密分享洗牌的理想功能 $\mathcal{F}_{\text{shuffle}}$

2.4.5 哈希分桶

哈希分桶 (Hashing to Bins) 技术 [12, 63] 被广泛用于 PSO 中, 以确保不同参与方持有的相同元素被分配到具有相同索引的桶 (bin) 中。这种对齐是通过简单哈希 (Simple Hashing) 和布谷鸟哈希 (Cuckoo Hashing) [64] 对输入集合进行预处理来实现的。

本文用以下符号表示简单哈希:

$$\mathcal{T}^1, \dots, \mathcal{T}^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X)$$

该表达式表示使用哈希函数 $h_1, h_2, h_3 : \{0, 1\}^* \rightarrow \{1, \dots, B\}$ 将集合 X 中的元素哈希到 B 个桶中。输出为简单哈希表 $\mathcal{T}^1, \dots, \mathcal{T}^B$, 其中对于任意 $x \in X$, 满足 $\mathcal{T}^{h_i(x)} \supseteq \{x \parallel i \mid i = 1, 2, 3\}$ ¹。

本文用以下符号表示布谷鸟哈希:

$$\mathcal{C}^1, \dots, \mathcal{C}^B \leftarrow \text{Cuckoo}_{h_1, h_2, h_3}^B(X)$$

该表达式表示使用哈希函数 h_1, h_2, h_3 将集合 X 中的元素哈希到 B 个桶中。输出为布谷鸟哈希表 $\mathcal{C}^1, \dots, \mathcal{C}^B$, 其中对于任意 $x \in X$, 存在唯一的 $i \in \{1, 2, 3\}$ 使得 $\mathcal{C}^{h_i(x)} = \{x \parallel i\}$ 。

2.4.6 多密钥重随机化公钥加密

Gao 等人 [54] 引入了多密钥重随机化公钥加密 (Multi-Key Rerandomizable Public-Key Encryption, MKR-PKE) 的概念, 这是 PKE 的一种变体, 具有若干额外的属性。令 $\mathcal{SK}$ 表示私钥空间 (在运算 $+$ 下构成阿贝尔群), $\mathcal{PK}$ 表示公钥空间 (在运算 $\cdot$ 下构成阿贝尔群)。 $\mathcal{M}$ 表示明文空间, $\mathcal{C}$ 表示密文空间。MKR-PKE 是一个 PPT 算法组 (Gen, Enc, ParDec, Dec, ReRand), 满足:

- 密钥生成算法 Gen: 输入安全参数 1^λ , 输出一对密钥 $(pk, sk) \in \mathcal{PK} \times \mathcal{SK}$ 。
- 加密算法 Enc: 输入公钥 $pk \in \mathcal{PK}$ 和明文消息 $x \in \mathcal{M}$, 输出密文 $ct \in \mathcal{C}$ 。
- 部分解密算法 ParDec: 输入私钥分片 $sk \in \mathcal{SK}$ 和密文 $ct \in \mathcal{C}$, 输出另一密文 $ct' \in \mathcal{C}$ 。
- 解密算法 Dec: 输入私钥 $sk \in \mathcal{SK}$ 和密文 $ct \in \mathcal{C}$, 输出消息 $x \in \mathcal{M}$ 或错误符号 $\perp$ 。
- 重随机化算法 ReRand: 输入公钥 $pk \in \mathcal{PK}$ 和密文 $ct \in \mathcal{C}$, 输出另一密文 $ct' \in \mathcal{C}$ 。

MKR-PKE 是一个满足以下性质的 PKE 方案:

¹附加哈希函数的索引 i 有助于处理如 $h_1(x) = h_2(x) = i$ 这样的边缘情况, 这些情况发生的概率是不可忽略的。

多次加密不可区分性. 对于公钥加密方案 $\mathcal{E} = (\text{Gen}, \text{Enc}, \text{Dec})$, 若对于任意概率多项式时间 (PPT) 的敌手 $\mathcal{A}$, 针对其选择的任意两个元组 $(m_1, \dots, m_q)$ 和 $(m'_1, \dots, m'_q)$ (其中 q 是关于 λ 的多项式), 满足以下分布在计算上不可区分:

$$\begin{aligned} & \{\text{Enc}(pk, m_1), \dots, \text{Enc}(pk, m_q) : (pk, sk) \leftarrow \text{Gen}(1^\lambda)\} \stackrel{c}{\approx} \\ & \{\text{Enc}(pk, m'_1), \dots, \text{Enc}(pk, m'_q) : (pk, sk) \leftarrow \text{Gen}(1^\lambda)\} \end{aligned}$$

则称该公钥加密方案具有多次加密不可区分性。选择明文攻击下不可区分性 (indistinguishability under chosen-plaintext attack, IND-CPA) 蕴含多次加密不可区分性。

可部分解密性 (Partially Decryptable). 对于任意两对密钥 $(sk_1, pk_1) \leftarrow \text{Gen}(1^\lambda), (sk_2, pk_2) \leftarrow \text{Gen}(1^\lambda)$ 和任意 $x \in \mathcal{M}$, 满足:

$$\text{ParDec}(sk_1, \text{Enc}(pk_1 \cdot pk_2, x)) \stackrel{s}{\approx} \text{Enc}(pk_2, x)$$

可重随机化性 (Rerandomizable). 对于任意 $pk \in \mathcal{PK}$ 和任意 $x \in \mathcal{M}$, 满足:

$$\text{ReRand}(pk, \text{Enc}(pk, x)) \stackrel{s}{\approx} \text{Enc}(pk, x)$$

Gao 等人 [54] 利用基于椭圆曲线 (Elliptic Curve, EC) 的 ElGamal 加密 [53] 实例化了满足 IND-CPA 安全性的 MKR-PKE。具体的 EC MKR-PKE 构造如下:

- 密钥生成算法 Gen: 输入安全参数 1^λ , 生成 $(\mathbb{G}, g, q)$, 其中 $\mathbb{G}$ 是循环群, g 是生成元, q 是阶。输出 sk 和 $pk = g^{sk}$ 。
- 随机化加密算法 Enc: 输入公钥 pk 和明文消息 $x \in \mathbb{G}$, 采样 $r \leftarrow \mathbb{Z}_q$, 输出 $\text{ct} = (\text{ct}_1, \text{ct}_2) = (g^r, x \cdot pk^r)$ 。
- 部分解密算法 ParDec: 输入私钥分片 sk 和密文 $\text{ct} = (\text{ct}_1, \text{ct}_2)$, 输出 $\text{ct}' = (\text{ct}_1, \text{ct}_2 \cdot \text{ct}_1^{-sk})$ 。
- 解密算法 Dec: 输入私钥 sk 和密文 $\text{ct} = (\text{ct}_1, \text{ct}_2)$, 输出 $x = \text{ct}_2 \cdot \text{ct}_1^{-sk}$ 。
- 重随机化算法 ReRand: 输入 pk 和密文 $\text{ct} = (\text{ct}_1, \text{ct}_2)$, 采样 $r' \leftarrow \mathbb{Z}_q$, 输出 $\text{ct}' = (\text{ct}_1 \cdot g^{r'}, \text{ct}_2 \cdot pk^{r'})$ 。

3 基于对称密钥的高效多方隐私集合求并协议

3.1 引言

隐私集合求并 (Private Set Union, PSU) 是隐私集合运算领域的一个重要研究分支, 它允许一组互不信任的参与方在不泄露除并集外任何额外信息的前提下, 协同计算各自私有集合的并集。PSU 在现实世界的协作计算和数据共享场景中具有广泛的应用价值。例如, 在信息安全风险评估与管理中 [29], 多个组织可以利用 PSU 联合各自持有的 IP 黑名单或漏洞数据库 [11], 从而构建更全面的威胁情报视图; 在政务数据开放中, 不同部门可以通过 PSU 聚合分布式的统计数据 [31]。此外, PSU 也是构建支持全连接 (Full Join) 的隐私数据库 [21] 以及 Private ID 协议 [19] 的核心底层组件。

与 PSO 中的另一研究分支——隐私集合求交 (Private Set Intersection, PSI) 相比, PSU 的研究起步较晚且发展相对缓慢: 目前的 PSI 技术已经相对成熟, 尤其是两方 PSI 在效率和实用性方面已经达到较高水平 [12, 13, 14, 15, 16, 17, 18], 其性能甚至可以接近不安全的朴素哈希方法。同时, 多方 PSI 也在两方协议研究成果的基础上得到了充分发展, 并提出了多种适用于大规模数据处理的高效方案 [43, 46, 65, 47]。然而, 早期由 Kissner 和 Song [31] 提出的两方 PSU 协议及其后续改进工作 [48] 主要依赖于加法同态加密 (AHE) 或复杂的通用电路, 导致其实际效率低下。直到 2019 年, Kolesnikov 等人 [21] 基于 OT 和对称密钥操作提出了首个适用于大规模数据集的高效两方 PSU 协议, 实现了相比基于公钥密码的方案三个数量级的速度提升。随后的研究 [19, 22] 继续在基于对称密钥的技术路线上探索, 利用不经意切换 (Oblivious Switching) 等技术降低了协议的通信与计算开销。近期的研究 [23, 20] 已实现计算复杂度与通信复杂度均为线性的两方 PSU 协议, 标志着两方 PSU 技术也逐渐迈向成熟。

随着跨组织数据协作需求的增长, 多方场景相较两方场景更具实际意义。与两方 PSU 相比, 多方隐私集合求并 (Multi-party Private Set Union, MPSU) 在多组织联合数据分析、分布式威胁情报共享以及跨域数据协同计算等场景中具有更为广泛的应用需求。但 MPSU 领域的研究进展依然滞后, 现有协议的效率与安全性方面仍面临较大挑战。根据底层所采用的密码学技术, 现有的 MPSU 协议构造主要分为以下两类:¹

- **基于公钥密码技术的 MPSU:** 该类协议 [31, 48, 40, 54] 主要依赖于公钥密码技术 (如 AHE), 其共同缺陷在于每个参与方需执行大量耗时的公钥操作, 在多方和大规模数据场景下计算与通信开销较高, 实际效率难以满足应用需求。
- **基于对称密钥技术的 MPSU:** 这一路线目前仅有 Liu 和 Gao [1] 的工作。虽然该协议利用 OT 和对称密钥技术实现了远超公钥路线的性能 (比 [40] 快约两个数量

¹本文不考虑基于通用电路的 MPSU 方案, 因为其性能开销在实际应用中通常难以接受。

级),但其安全性存在严重缺陷:其安全性证明本质上依赖于“非共谋假设”(Non-collusion Assumption),即假设获得结果的领导者(Leader)不会与其他参与方(Client)共谋。在现实世界的开放网络或存在利益竞争的商业合作中,该假设难以成立,导致协议无法抵御任意共谋攻击,存在严重的安全风险。

综上所述,现有 MPSU 研究在效率与安全性之间仍存在明显权衡:基于公钥密码技术的方案安全性强但效率低,而基于对称密钥技术的方案效率高但安全性弱,由此引出了一个关键的研究问题:

能否在标准半诚实模型下,构造出基于 OT 与对称密钥操作、且不依赖任何非共谋假设的高效 MPSU 协议?

本章围绕上述问题展开研究,通过提出两种新型密码学组件——批量秘密分享隐私成员测试(batch secret-shared private membership test, batch ssPMT)和多方秘密分享随机不经意传输(multi-party secret-shared ROT, mss-ROT),分别突破了现有对称密钥 MPSU 方案 [1] 的性能瓶颈和消除了其对非共谋假设的依赖,最终实现首个满足标准半诚实安全的基于对称密钥的高效 MPSU 协议。

3.2 技术路线概述

本节从整体层面对本章的研究思路与技术路线进行概述。本节首先对 Liu 和 Gao [1] 提出的基于对称密钥的 MPSU 协议(以下简称 LG 协议)进行回顾与深度分析,指出其在性能和安全性方面的局限性。随后,本节将阐述本文如何通过引入“批量秘密分享隐私成员测试”(batch ssPMT)和“多方秘密分享随机不经意传输”(mss-ROT)这两个新原语,以逐个突破现有框架的性能和安全性瓶颈,从而构建满足标准半诚实安全且更高效的 MPSU 协议。

3.2.1 LG 协议的回顾与局限性分析

我们首先将 LG 协议抽象为一种基于秘密分享(Secret-Sharing)的 MPSU 框架。该框架的核心思想是将复杂的并集计算转化为一组差集的秘密分享问题。设共有 m 个参与方 $P_1, \dots, P_m$, 各自输入集合 $X_1, \dots, X_m$ 。定义第 j 个参与方的差集为 $Y_j = X_j \setminus (X_1 \cup \dots \cup X_{j-1})$, $2 \leq j \leq m$, 则有 $X_1 \cup \dots \cup X_m = X_1 \cup Y_2 \cup \dots \cup Y_m$, 即所有的差集构成全集的一个划分。因此,只要各参与方能够获得所有差集元素的加法秘密分享,并在最终阶段由 Leader(以下指定为 P_1)重构,即可得到并集结果。该框架具体可分为以下两个阶段:

1. **并集的秘密分享.** 该阶段涉及 $m-1$ 次差集的秘密分享,其中对于第 j 次秘密分享过程($2 \leq j \leq m$), P_j 通过特定的交互协议将其差集 Y_j 的元素以加法秘密分

享的形式分布在所有参与方之间；而对于 X_j 中不属于 Y_j 的元素（即已存在于前面集合中的冗余元素），则让参与方获得随机值的秘密分享。在该阶段结束时，所有参与方共同持有了关于 $(X_1 \cup \dots \cup X_m) \setminus X_1$ 中所有元素的秘密分享，且这些分享按 $Y_2, \dots, Y_m$ 的顺序排列，中间穿插着随机值从而将每个差集的大小填充为 n 。

2. **洗牌与重构**. 如果所有参与方将持有的秘密分享发送给 P_1 ，由 P_1 直接重构出元素，那么它将能够利用秘密分享排布的线性位置特征推断出每个元素的来源方。例如，若 P_1 得到的某个重构出的元素位于第一个长度为 n 的区块（即 Y_2 的位置），则 P_1 可断定该元素来自 P_2 的输入。这种对应关系会导致隐私泄露。为了解决这一问题，各方在重构前需调用“多方秘密分享洗牌”（Multi-party Secret-shared Shuffle）协议，对所有分享进行随机置换。这确保了任何 $m-1$ 方共谋都无法获知洗牌的置换映射，从而掩盖了重构元素与各差集 Y_j 之间的对应关系。最后， P_1 将重构出的差集元素与自身的 X_1 合并，得到最终并集。

LG 协议利用两个核心组件来实现第一阶段：

- **秘密分享隐私成员测试 (secret-shared private membership test, ssPMT)**. 发送方 $\mathcal{S}$ 输入集合 X ，接收方 $\mathcal{R}$ 输入元素 y ；若 $y \in X$ 则双方获得 1 的秘密分享，否则获得 0 的秘密分享。进一步地，Liu 和 Gao 提出了多查询 ssPMT (multi-query ssPMT, mq-ssPMT)，支持接收方同时查询多个元素，其中 $\mathcal{S}$ 输入集合 X ， $\mathcal{R}$ 输入向量 $\mathbf{y} = (y_1, \dots, y_n)$ 。双方获得测试 $\mathbf{y}$ 中元素是否在 X 中的结果比特向量的秘密分享：若 $y_i \in X$ 则第 i 位分享值为 1，否则为 0。
- **双选择比特随机不经意传输 (two-choice-bit ROT)**. 发送方 $\mathcal{S}$ 和 $\mathcal{R}$ 分别持有选择比特 e_0, e_1 ， $\mathcal{S}$ 获得两个随机数 r_0, r_1 ， $\mathcal{R}$ 根据 $e_0 \oplus e_1$ 的结果获得其中之一：若 $e_0 \oplus e_1 = 0$ ， $\mathcal{R}$ 获得随机消息 r_0 ，否则获得 r_1 。

上述 LG 协议的构造存在两个严重的“痛点”：首先，在差集计算过程中，需要反复执行 mq-ssPMT 的现有实例化高度依赖昂贵的通用 MPC 技术，计算与通信开销较大，成为整个协议的效率瓶颈；其次，第一阶段无法抵抗 P_1 与其他参与方之间的共谋攻击，其安全性依赖于非共谋假设，即要求最终获得结果的 P_1 不与其他参与方共谋。基于上述观察，下面从效率与安全两个方面对 LG 进行改进。

3.2.2 高效的 batch ssPMT 原语

针对 mq-ssPMT 的效率瓶颈，本文提出了一种全新的原语——**批量秘密分享隐私成员测试 (batch ssPMT)**。它是 ssPMT 的批量化版本：发送方 $\mathcal{S}$ 输入 n 个不相交集 $X_1, \dots, X_n$ ，接收方 $\mathcal{R}$ 输入 n 个元素 $y_1, \dots, y_n$ ；双方获得比特向量的秘密分享，若 $y_i \in X_i$ 则第 i 位为 1，否则为 0。

与 mq-ssPMT 相比, batch ssPMT 能够测试元素在多个不同集合中的成员关系,而非单一共同集合。两者的关系类似于 batch OPRF [13] 与多点 OPRF [14] 之间的关系。这种“批量化”的特性允许更高效的构造,并能够通过结合哈希分桶 (Hashing-to-bins) 技术,在 MPSU 场景下完全替代 mq-ssPMT,具体构造如下:接收方 $\mathcal{R}$ 利用布谷鸟哈希 (Cuckoo Hashing) 将输入向量 $\mathbf{y}$ 中的元素分配至 B 个桶中,布谷鸟哈希可以确保每个桶至多一个元素;发送方 $\mathcal{S}$ 则利用简单哈希 (Simple Hashing) 将输入集合 X 中的元素分配至对应的所有桶中。随后,双方调用包含 B 个 ssPMT 实例的 batch ssPMT 协议。哈希分桶技术保证了只有当 $\mathcal{R}$ 的第 j 桶中的元素 $y_i \in X$,该元素才会存在于 $\mathcal{S}$ 的第 j 个桶中所有元素组成的集合 X_j 中,从而双方在调用 batch ssPMT 后才能获得 1 的分享。然而,由于该构造最终输出的秘密分享的顺序取决于 $\mathcal{R}$ 使用布谷鸟哈希存放 $\mathbf{y}$ 中各元素的位置,而该位置依赖于 $\mathbf{y}$ 中所有元素,直接重构可能会泄露 $\mathcal{R}$ 的输入向量中的元素信息。本文的关键洞察在于:第二阶段使用的多方秘密分享置换不仅消除了重构顺序与差集的之前对应关系,同时也消除了其与哈希位置的依赖。因此,上述基于哈希分桶和 batch ssPMT 的构造可以无缝嵌入现有的基于秘密分享的 MPSU 框架以取代 mq-ssPMT,而不会引入额外信息泄露。

相比于依赖通用两方计算 (2PC) 的 mq-ssPMT,本文的 batch ssPMT 基于两个轻量级的组件构建:批量不经意可编程伪随机函数 (batch oblivious programmable pseudorandom function, batch OPRF) 和秘密分享隐私等值测试 (secret-shared private equality test, ssPEQT)。其中, batch OPRF 可通过向量不经意线性求值 (textbf{O}blivious Linear Evaluation, VOLE) 获得极其高效的实例化,而 ssPEQT 虽然需利用通用 2PC 实现,但因其仅涉及规模较小的电路,计算和通信开销小,并不会成为性能瓶颈。因此,该构造显著减少了协议对通用 2PC 的依赖,从而大幅提升了协议的整体性能。

3.2.3 基于 batch ssPMT 和 mss-ROT 的 SK-MPSU 协议

本节将在 LG 协议的基础上以递增的方式重新设计 LG 协议的第一阶段,从而构建标准半诚实模型下的首个基于对称密钥的 MPSU 协议 (以下称该协议为 SK-MPSU 协议)。首先给出用上述基于哈希分桶和 batch ssPMT 的构造替代 mq-ssPMT 后的新的秘密分享差集 Y_j 的过程:令 P_j 通过布谷鸟哈希预处理其集合 X_j ,其余 $P_{i < j}$ 通过简单哈希预处理其集合 X_i 。 P_j 作为接收方与每个 P_i 执行 batch ssPMT。对于每个桶, P_j 和 P_i 分别获得成员关系比特的秘密分享 $e_{j,i}$ 和 $e_{i,j}$ 。随后, P_j 作为发送方与每个 P_i 执行双选择比特 ROT,其中 P_j 输入 $e_{j,i}$, P_i 输入 $e_{i,j}$ 。执行结束后, P_j 获得随机对 $(r_{j,i}^0, r_{j,i}^1)$ 。若 $e_{j,i} \oplus e_{i,j} = 0$,则 P_i 获得 $r_{j,i}^0$;否则获得 $r_{j,i}^1$ 。 P_i 将获得的输出 $r_{j,i}^{e_{j,i} \oplus e_{i,j}}$ 设为其秘密分享份额 $s_{j,i}$ 。 P_j 则将其分享份额 $s_{j,j}$ 设置为 $\bigoplus_{i=1}^{j-1} r_{j,i}^0 \oplus x \parallel H(x)$,其中 $x \in X_j$ 是 P_j 桶中的元素,附加的哈希值 $H(x)$ 用于在重构阶段区分真实元素和随机秘密。上述构造的正确性在于:若 $x \in Y_j$,则对于所有 i 都有 $e_{j,i} \oplus e_{i,j} = 0$,此时所有秘密分享份额之和 $\bigoplus_{i=1}^j s_{j,i} = x \parallel H(x)$,即各方持有该元素的秘密分享。反之,若 $x \notin Y_j$,则至少有一个

$r_{j,i}^0 \oplus r_{j,i}^1$ 项无法在求和中抵消, 从而导致各方持有的是一个随机秘密的分享。

尽管上述构造比 LG 协议更高效, 但其同样无法抵御 P_1 和其他参与方的共谋攻击。例如, 在第二阶段中, 若 P_1 重构出秘密 $r_{j,i'}^0 \oplus r_{j,i'}^1 \oplus x \| H(x)$ ($1 \leq i' < j$), 则说明在求和过程中仅有第 i' 个随机项掩码 (记为 $\Delta_{ji'} = r_{j,i'}^0 \oplus r_{j,i'}^1$) 未被抵消。由于 P_j 在与 $P_{i'}$ 的 ROT 中同时获得 $r_{j,i'}^0$ 与 $r_{j,i'}^1$, 若 P_j 与 P_1 共谋, 则 P_1 可以推断出对于所有 $i \neq i'$, $e_{j,i} \oplus e_{i,j} = 0$ 均成立。进而, 他们可以导出 $x \in X_{i'}$ 并且对于所有 $i \neq i'$, $x \notin X_i$, 这泄露了关于 $P_2, \dots, P_{j-1}$ 的输入集合信息。因此, 该协议与 LG 一样, 需要假设 P_1 不与其他参与方共谋才能证明安全性。

该共谋攻击成立的根本原因在于: 当 $x \notin Y_j$ 时, 随机秘密的值在 P_1 与其共谋方的视图中并非均匀随机; 并且第二阶段中多方秘密分享置换的置换与重新分享并不会改变这些随机秘密的值。为了抵御该攻击, 本章的解决方案是将每个随机项掩码 $\Delta_{ji} = r_{j,i}^0 \oplus r_{j,i}^1$ 在参与方子集 $\{P_2, \dots, P_j\}$ 之间进行加法秘密分享。这样, 只要该子集中存在至少一个诚实方, 求和式中任何潜在的剩余分量 (包含至少一个随机项掩码) 都仍是均匀随机的。为了实现该功能, 将双选择比特 ROT 推广到多方场景, 抽象出多方秘密分享随机不经意传输 (mss-ROT) 的概念: 在 mss-ROT 中, 选择比特由 P_{ch_0} 和 P_{ch_1} 分别持有, 掩码 Δ 由参与方子集 J 提供, 最终各方输出零或随机值的秘密分享, 其秘密是否为零由选择比特共同决定。具体地, 设共有 j 个参与方参与一次 mss-ROT, 其中两方 P_{ch_0} 和 P_{ch_1} 分别持有选择比特 b_0 和 b_1 。设存在一个参与方子集 J , 其中每个 $P_{j'} \in J$ 输入一个值 $\Delta_{j'}$ 。协议输出随机值 $r_{i'}$ 给每个参与方 $P_{i'}$ 作为秘密分享份额, 满足: 若 $b_0 \oplus b_1 = 0$, 则 $\bigoplus r_{i'} = 0$; 否则 $\bigoplus r_{i'} = \bigoplus_{j' \in J} \Delta_{j'}$ 。

在引入 mss-ROT 后, 对前述秘密分享差集 Y_j 的过程进行如下修改: 对于每个桶, 在 P_j 与 P_i 获得 $e_{j,i}$ 和 $e_{i,j}$ 后, 不再执行双选择比特 ROT, 而是调用 mss-ROT。其中设 $J = \{P_2, \dots, P_j\}$, 并由每个 $P_{j'} \in J$ 输入一个均匀随机值 $\Delta_{j',ji}$ 。根据 mss-ROT 的功能, 若 $e_{j,i} \oplus e_{i,j} = 0$, 则所有输出异或和为 0; 否则所有输出异或和为 $\bigoplus_{j'=2}^j \Delta_{j',ji}$ 。在完成所有 mss-ROT 调用后, 每个参与方将从多次 mss-ROT 调用中获得的输出进行异或, 作为其对应该桶的最终分享份额, P_j 额外异或 $x \| H(x)$ 。此时, 若 $x \in Y_j$, 则对所有 i 均有 $e_{j,i} \oplus e_{i,j} = 0$, 因此所有 mss-ROT 输出的异或和为 0, 最终各方得到 $x \| H(x)$ 的秘密分享; 若 $x \notin Y_j$, 则异或和中至少存在一个随机项掩码 $\bigoplus_{j'=2}^j \Delta_{j',ji}$, 并且该值以加法秘密分享形式分布在 $\{P_2, \dots, P_j\}$ 中。因此, 即使 P_1 与 $\{P_2, \dots, P_j\}$ 其中最多 $m-2$ 方共谋, 该异或和仍保持均匀分布, 各方持有的仍是一个随机秘密。由于不包含 P_1 的任何共谋子集都无法获知重构出的秘密, 最终协议可以抵抗任意 $m-1$ 方共谋, 从而在标准半诚实模型下证明安全。

3.3 批量秘密分享隐私成员测试

批量秘密分享隐私成员测试 (batch secret-shared private membership test, batch ssPMT) 是本章提出的 SK-MPSU 协议以及第四章提出的 PK-MPSU 协议的核心构建组件。作为一个两方协议, batch ssPMT 旨在发送方 $\mathcal{S}$ 和接收方 $\mathcal{R}$ 之间批量地实现多个 ssPMT 实例。

3.3.1 功能定义

给定批量大小 (batch size) 为 B , $\mathcal{S}$ 输入 B 个不相交的集合 $X_1, \dots, X_B$, $\mathcal{R}$ 输入 B 个元素组成的向量 $\mathbf{x} = (x_1, \dots, x_B)$ 。协议执行结束后, 双方分别获得分享份额向量 $\mathbf{e}_S = (e_S^1, \dots, e_S^B)$ 和 $\mathbf{e}_R = (e_R^1, \dots, e_R^B)$ 。其中对于每个 $i \in [B]$, 若 $y_i \in X_i$, 则 $e_S^i \oplus e_R^i = 1$; 否则 $e_S^i \oplus e_R^i = 0$ 。图 3.1 形式化描述了 batch ssPMT 的理想功能 $\mathcal{F}_{\text{bssPMT}}$ 。

参数: 发送方 $\mathcal{S}$, 接收方 $\mathcal{R}$; 批量大小 B ; 集合元素的比特长度 l ; OPPrf 输出的比特长度 γ 。

输入: $\mathcal{S}$ 输入 B 个不相交集 $X_1, \dots, X_B$; $\mathcal{R}$ 输入向量 $\mathbf{x} \in (\{0, 1\}^l)^B$ 。

功能: 收到双方输入后, 对于每个 $i \in [B]$, 随机采样两个比特 $e_S^i, e_R^i \in \{0, 1\}$, 满足:

- 若 $x_i \in X_i$, 则 $e_S^i \oplus e_R^i = 1$;
- 否则, $e_S^i \oplus e_R^i = 0$ 。

最后, 将 $\mathbf{e}_S = (e_S^1, \dots, e_S^B)$ 发送给 $\mathcal{S}$, 将 $\mathbf{e}_R = (e_R^1, \dots, e_R^B)$ 发送给 $\mathcal{R}$ 。

图 3.1 批量秘密分享隐私成员测试的理想功能 $\mathcal{F}_{\text{bssPMT}}$

3.3.2 协议构造与安全性证明

本节基于批量不经意可编程伪随机函数 (batch OPPrf) 和秘密分享隐私等值测试 (ssPEQT) 给出 batch ssPMT 协议的高效构造, 具体协议 Π_{bssPMT} 如图 3.2 所示。

定理 3.1. 在 $(\mathcal{F}_{\text{bOPPrf}}, \mathcal{F}_{\text{ssPEQT}})$ -混合模型下, 协议 Π_{bssPMT} 安全地实现了功能 $\mathcal{F}_{\text{bssPMT}}$ 。

正确性分析. 根据 batch OPPrf 的理想功能, 若接收方的元素 $x_i \in X_i$, 则接收方必然获得对应的预定义标签 $t_i = s_i$ 。随后在第 i 个 ssPEQT 实例中, 由于双方输入相等, 其输出的比特秘密分享之和 $e_S^i \oplus e_R^i = 1$ 。反之, 若 $x_i \notin X_i$, 接收方获得的 t_i 是一个伪

1. **初始化**: 对于每个 $1 \leq i \leq B$, $\mathcal{S}$ 随机选取一个标签 $s_i \leftarrow \{0, 1\}^\gamma$, 并构造一个多重集 (multiset) S_i , 该集合包含 $|X_i|$ 个重复的元素, 且所有元素均等于 s_i 。
2. **执行 batch OPPRF**: 双方调用批量大小为 B 的理想功能 $\mathcal{F}_{\text{batchOPPRF}}$, 其中:
 - $\mathcal{S}$ 作为发送方, 输入 B 个键集合 (Key Set) $X_1, \dots, X_B$ 和 B 个值集合 (Value Set) $S_1, \dots, S_B$ 。
 - $\mathcal{R}$ 作为接收方, 输入向量 $\mathbf{x}$, 并获得输出向量 $\mathbf{t} = (t_1, \dots, t_B)$ 。
3. **执行 ssPEQT**: 双方调用 B 个独立的 $\mathcal{F}_{\text{ssPEQT}}$ 实例。在第 i 个实例中, $\mathcal{S}$ 输入 s_i , $\mathcal{R}$ 输入 t_i 。最终, 双方分别获得比特 $e_S^i, e_R^i \in \{0, 1\}$ 。

 图 3.2 批量秘密分享隐私成员测试协议 Π_{bssPMT}

随机值。此时 $t_i = s_i$ 发生碰撞的概率仅为 $2^{-\gamma}$ 。对于整个向量而言, 发生至少一次错误碰撞的概率上限为 $B \cdot 2^{-\gamma}$ 。通过设置安全参数 $\gamma \geq \sigma + \log B$, 可以使该碰撞概率忽略不计 (negligible)。因此, 若 $x_i \notin X_i$, ssPEQT 的输出以压倒性概率 (overwhelming probability) 满足 $e_S^i \oplus e_R^i = 0$ 。

安全性分析. 协议 Π_{bssPMT} 的安全性直接继承自底层组件 batch OPPRF 与 ssPEQT 协议的安全性。具体地, 在理想世界中, 给定 $\mathcal{F}_{\text{bssPMT}}$ 输出的前提下, 模拟器依次调用 ssPEQT 与 batch OPPRF 协议的子模拟器。只要 batch OPPRF 与 ssPEQT 协议安全地实现了理想功能 $\mathcal{F}_{\text{ssPEQT}}$ 和 $\mathcal{F}_{\text{batchOPPRF}}$, 则模拟器在理想世界中模拟的敌手视图 (即 batch OPPRF 与 ssPEQT 协议的子模拟器在理想世界中模拟的敌手视图的联合分布) 与敌手在真实世界中的视图 (敌手分别在 batch OPPRF 与 ssPEQT 协议的调用过程中视图的联合分布) 是计算不可区分的。

3.3.3 复杂度分析及对比

在本章提出的 MPSU 协议以及后续章节 batch ssPMT 的使用中, batch ssPMT 始终与哈希分桶技术相结合, 其中 $\mathcal{R}$ 利用布谷鸟哈希将输入向量 $\mathbf{y}$ 中的元素分配至 B 个桶中, $\mathcal{S}$ 则利用简单哈希将输入集合 X 中的元素分配至对应的所有桶中。双方调用包含 B 个 ssPMT 实例的 batch ssPMT 协议。本节将上述过程看作一个整体进行复杂度分析。为了使得调用包含 B 个 ssPMT 实例的 batch ssPMT 协议的计算与通信开销相对于集合规模 n 呈线性增长, 本文采用无 stash 的三哈希布谷鸟哈希技术 [60], 在该参数设置下, 为保证哈希失败概率 (即元素无法存入哈希表而必须存入 stash 的概率) 小于 2^{-40} , 本文设置桶数 $B = 1.27n = O(n)$ 。本文实现 batch ssPMT 相对于 n 的线

性复杂度，主要依靠两个关键技术：首先，本文遵循文献 [60] 的范式，利用批量不经意随机函数（batch oblivious pseudorandom function, batch OPRF）与不经意键值存储（oblivious key-value store, OKVS） [16, 45, 18, 52] 构造批量不经意可编程 PRF（batch OPPRF）。通过通信摊销技术，计算 $B = O(n)$ 个 OPPRF 实例的总通信开销仅与元素总数 $3n$ 相关。其次，本文采用子域向量不经意线性求值（subfield-VOLE） [66, 67, 68] 实例化 batch OPRF，并结合文献 [18] 中的 OKVS 构造，确保了规模为 $O(n)$ 的 batch OPPRF 的计算复杂度与 n 线性相关。

3.3.4 batch ssPMT 的复杂度细节

Π_{bssPMT} 协议中各阶段的具体开销分析如下：

1. **执行 batch OPPRF**：该部分的开销由 batch OPRF 和 OKVS 两部分组成。

- **batch OPRF**：本文使用 subfield-VOLE 实例化 batch OPRF 功能 [69]。对于 $1 \leq k \leq B$ 且 $B = O(n)$ 的 OPRF 实例，解随机化消息。对于 $B = O(n)$ 个 OPRF 实例，协议在离线阶段执行规模为 B 的 subfield-VOLE，在线阶段发送 B 个长度为 l 的去随机化消息。本文采用基于带固定权重和正则噪声的对偶 LPN（Dual LPN with Fixed Weight and Regular Noise） [66, 67]，并使用 Expand-Convolute 码 [68] 以实现近线性的伴随式计算（syndrome computation）。

设 OPPRF 的输出长度为 γ ，其下界由 batch OPPRF 的总调用次数决定。在本章提出的 SK-MPSU 协议与后续章节提出的 MPSU 协议中，对于所有 $1 \leq i < j \leq m$ ，参与方 P_i 与 P_j 均需调用一次 batch OPPRF，总调用次数为 $(m^2 - m)/2$ 。为保证当 $\mathbf{x} \notin X_i$ 时， $t_i = s_i$ 发生的概率不大于 $2^{-\sigma}$ ，需满足 $((m^2 - m)/2)B \cdot 2^{-\gamma} \leq 2^{-\sigma}$ ，即 $\gamma \geq \sigma + \log((m^2 - m)/2) + \log_2 B$ 。

- **OKVS**： $\mathcal{S}$ 将简单哈希的桶中的 $3n$ 个元素编码至 OKVS，因此 OKVS 中键值对的规模为 $3n$ 。本文采用文献 [18] 的 OKVS 构造，其 Encode_H 与 Decode_H 算法的计算复杂度均为 $O(n)$ 。在使用 $w = 3$ 且簇大小为 2^{14} 的参数设置下，OKVS 的存储开销约为 $1.28\gamma \cdot (3n)$ 比特。

综上，batch OPPRF 的阶段性和开销如下：

- **离线阶段**：单方计算复杂度为 $O(n \log n)$ ，通信复杂度为 $O(t\lambda \log(n/t))$ ，轮数复杂度为 $O(1)$ （其中 $t \approx 128$ ）。
- **在线阶段**：单方计算复杂度为 $O(n)$ ，通信复杂度为 $O(\gamma n)$ ，轮数复杂度为 $O(1)$ 。

2. 执行 ssPEQT: 对于每个桶, 需要执行一次 ssPEQT 实例。本文利用通用 MPC 技术实例化 ssPEQT [60]。具体构造为用 GMW 协议 [5] 实现由 $\gamma - 1$ 个 AND 门组成的电路。为降低轮复杂度, 本文采用分治策略, 将 AND 门递归地组织为二叉树结构, 使得轮数缩减至 $O(\log \gamma)$ 。总计 B 个 ssPEQT 实例涉及 $(\gamma - 1)B$ 个 AND 门。

通过在离线阶段利用 Silent OT 扩展 [66] 生成 Beaver 三元组, 在线阶段每个 AND 门仅需 4 比特通信和 $O(1)$ 计算。因此, 每次 ssPEQT 调用的在线通信为 4γ 比特, 计算复杂度为 $O(1)$ 。

- **离线阶段:** 单方计算复杂度为 $O(\gamma n \log n)$, 通信复杂度为 $O(t\lambda \log(\gamma n/t))$, 轮数复杂度为 $O(1)$ 。
- **在线阶段:** 单方计算复杂度为 $O(n)$, 通信复杂度为 $O(\gamma n)$, 轮数复杂度为 $O(\log \gamma)$ 。

3.3.5 与 mq-ssPMT 的对比

Liu 和 Gao 将 [23] 中基于对称密钥加密 (SKE) 的 mq-RPMT 构造改造为 mq-ssPMT, 其核心开销来自其中的秘密分享不经意解密匹配 (secret-shared oblivious decryption-then-matching, ssVODM)。该协议需通过 GMW 协议实现解密与比较电路, 总计需要 $(T + \gamma' - \log n - 1)n$ 个 AND 门, 其中 T 为解密电路中的 AND 门数量, 典型参数下 $T \approx 600$; γ' 为 SKE 密文比特长度, 其下界与 mq-ssPMT 的总调用次数有关: 在 LG 协议中, 对于 $1 \leq i < j \leq m$, 总共存在 $1 + 2 + \dots + (m - 1) = (m^2 - m)/2$ 次 mq-ssPMT 调用。为了控制错误概率可忽略不计, 需保证 $((m^2 - m)/2)n \cdot 2^{-\gamma'} \leq 2^{-\sigma}$, 因此 $\gamma' \geq \sigma + \log((m^2 - m)/2) + \log_2 n$ 。²

相比之下, 规模为 B 的 batch ssPMT 由规模为 B 的批量 OPPRF 和 B 个 ssPEQT 实例构成。其中, batch OPPRF 采用了最先进的线性构造 [61, 17, 18], 该部分可实现相对于 n 的线性计算与通信复杂度。在 ssPEQT 环节, 各参与方仅需协同处理 $1.27(\gamma - 1)n$ 个 AND 门, 其中 γ 为 OPPRF 的输出长度。在典型的参数设置下, $(T + \gamma' - \log n - 1)n$ 的规模远大于 $1.27(\gamma - 1)n$ 。这意味着 batch ssPMT 实际所需的 AND 门数量远少于 mq-ssPMT。因此, 基于 batch ssPMT 的构造不仅极大地降低了对通用 2PC 组件的依赖, 相较于 mq-ssPMT, 在计算与通信复杂度上均实现了大幅度的性能优化。

3.4 多方秘密分享随机不经意传输

在基于秘密分享的 MPSU 框架中, 为了在标准半诚实模型下消除对非共谋假设的依赖, 本节引入一种新的密码学原语——多方秘密分享随机不经意传输 (Multi-Party

²LG 原文中错误地将 γ' 设置为了输入元素的比特长度, 这一参数设置可能导致其安全性在多方环境下无法得到保证。

Secret-Shared Random Oblivious Transfer, mss-ROT)。该原语是对传统两方随机不经意传输 (ROT) 功能的扩展, 将选择比特进行秘密分享, 并允许参与方的任意非空子集共同提供随机掩码的秘密分享。

3.4.1 功能定义

在 MPC 中, 基于秘密分享的设计方法被广泛采用。例如, ROT 的输出满足关系 $r_0 \oplus r_b = b \cdot \Delta$, 其中 $\Delta = r_0 \oplus r_1$ 。因此 ROT 的功能可以看作将选择比特 b 和掩码 Δ 标量乘后在两方之间进行秘密分享。对于前述的双选择比特 ROT, 它可以看作是将 b 秘密分享的版本, 输出满足 $r_0 \oplus r_b = (b_0 \oplus b_1) \cdot \Delta$ 。为解决 LG 协议中 P_1 与其他参与方共谋后重构出的 Δ 泄露诚实方信息的问题, 本节进一步扩展 ROT 中秘密分享的思想, 将 Δ 进行秘密分享, 同时满足以下特定设计要求:

1. **掩码分享灵活分布:** 允许参与方中的任何非空子集对 Δ 进行秘密分享, 从而保证分享可以分布在任意潜在的共谋方集合 J 中。这样, 只要 J 中存在一名诚实方, 重构出的 Δ 在敌手的视图中就是随机的。
2. **两方分享选择比特:** 选择比特仅在两方之间进行秘密分享, 以便嵌入到本文的 MPSU 框架中。
3. **掩码不可复用:** 由于所有分享份额都在最终阶段重构, 因此 mss-ROT 中的掩码 Δ 不能在多次调用之间复用。

具体地, mss-ROT 定义为: 将选择比特 $b = b_0 \oplus b_1$ 在两方 P_{ch_0} 和 P_{ch_1} 之间秘密分享。同时, 掩码 Δ 同样以秘密分享的形式由参与方子集 J 提供, 即 $\Delta = \bigoplus_{j \in J} \Delta_j$ 。最终所有参与方获得随机输出 r_i , 所有输出满足关系 $\bigoplus_{i=1}^m r_i = (b_0 \oplus b_1) \cdot \Delta = (b_0 \oplus b_1) \cdot (\bigoplus_{j \in J} \Delta_j)$, 也就是说, 若选择比特 $b = 0$, 异或和等于 0; 否则, 异或和等于掩码 Δ 。图 3.3 给出了 mss-ROT 的理想功能 $\mathcal{F}_{\text{mss-rot}}$ 的形式化描述。

3.4.2 协议构造与安全性证明

本节基于 ROT 原语给出 mss-ROT 协议的高效构造, 核心设计思路是将 $(b_0 \oplus b_1) \cdot (\bigoplus_{j \in J} \Delta_j)$ 化简为选择比特和掩码之间的两两乘积, 对于每一对选择比特和掩码, 通过调用 ROT 协议将乘积在两方之间秘密分享, 最后每个参与方将从所有 ROT 的调用中得到的输出求和, 作为自己的最终输出 r_i 。具体协议 $\Pi_{\text{mss-rot}}$ 如图 3.4 所示。

定理 3.2. 在 $\mathcal{F}_{\text{ROT}}$ -混合模型下, 协议 $\Pi_{\text{mss-rot}}$ 在任意半诚实敌手腐化不超过 $t < m$ 个参与方的情况下, 安全地实现了功能 $\mathcal{F}_{\text{mss-rot}}$ 。

参数：共有 m 个参与方 $P_1, \dots, P_m$ ，其中 P_{ch_0} 和 P_{ch_1} 输入选择比特的秘密分享， $\text{ch}_0, \text{ch}_1 \in \{1, \dots, m\}$ 。令 $J = \{j_1, \dots, j_d\}$ 表示输入掩码 Δ 秘密分享的参与方索引集合，其中 $d \leq m$ 。消息长度为 l 。

输入： P_{ch_0} 输入 $b_0 \in \{0, 1\}$ ； P_{ch_1} 输入 $b_1 \in \{0, 1\}$ ；对于每个 $j \in J$ ， P_j 输入掩码的分享份额 $\Delta_j \in \{0, 1\}^l$ 。

功能：收到来自 P_{ch_0} 和 P_{ch_1} 的选择比特秘密分享，以及来自每个 P_j 的掩码分享后：

- 随机采样 $r_i \leftarrow \{0, 1\}^l$ ，并将 r_i 发送给 P_i ($2 \leq i \leq m$)。
- 若 $b_0 \oplus b_1 = 0$ ，计算 $r_1 = \bigoplus_{i=2}^m r_i$ 。
- 若 $b_0 \oplus b_1 = 1$ ，计算 $r_1 = \bigoplus_{i=2}^m r_i \oplus (\bigoplus_{j \in J} \Delta_j)$ 。
- 将 r_1 发送给 P_1 。

图 3.3 多方秘密分享随机不经意传输的理想功能 $\mathcal{F}_{\text{mss-rot}}$

1. 每个 P_i ($1 \leq i \leq m$) 初始化 $r_i = 0$ 。若 $\text{ch}_j \in \{0, 1\} \in J$ ，则 P_{ch_j} 设置 $r_{\text{ch}_j} = b_j \cdot \Delta_{\text{ch}_j}$ ($\cdot$ 表示按位与)。
2. 对于每个 $j \in J$ ： P_{ch_0} 与 P_j 调用 $\mathcal{F}_{\text{ROT}}$ ， P_{ch_0} 作为接收方输入 b_0 ， P_j 作为发送方。 P_{ch_0} 获得输出 $u_{j,\text{ch}_0}^{b_0} \in \{0, 1\}^l$ ， P_j 获得输出 $u_{j,\text{ch}_0}^0, u_{j,\text{ch}_0}^1 \in \{0, 1\}^l$ 。 P_j 更新 $r_j = r_j \oplus u_{j,\text{ch}_0}^0$ ，并向 P_{ch_0} 发送消息 $\Delta'_{j,\text{ch}_0} = u_{j,\text{ch}_0}^0 \oplus u_{j,\text{ch}_0}^1 \oplus \Delta_j$ 。 P_{ch_0} 更新 $r_{\text{ch}_0} = r_{\text{ch}_0} \oplus u_{j,\text{ch}_0}^{b_0} \oplus b_0 \cdot \Delta'_{j,\text{ch}_0}$ 。
3. 对于每个 $j \in J$ ： P_{ch_1} 与 P_j 调用 $\mathcal{F}_{\text{ROT}}$ ， P_{ch_1} 作为接收方输入 b_1 ， P_j 作为发送方。 P_{ch_1} 获得输出 $u_{j,\text{ch}_1}^{b_1} \in \{0, 1\}^l$ ， P_j 获得输出 $u_{j,\text{ch}_1}^0, u_{j,\text{ch}_1}^1 \in \{0, 1\}^l$ 。 P_j 更新 $r_j = r_j \oplus u_{j,\text{ch}_1}^0$ ，并向 P_{ch_1} 发送消息 $\Delta'_{j,\text{ch}_1} = u_{j,\text{ch}_1}^0 \oplus u_{j,\text{ch}_1}^1 \oplus \Delta_j$ 。 P_{ch_1} 更新 $r_{\text{ch}_1} = r_{\text{ch}_1} \oplus u_{j,\text{ch}_1}^{b_1} \oplus b_1 \cdot \Delta'_{j,\text{ch}_1}$ 。
4. 若集合 $I = \{\text{ch}_0, \text{ch}_1\} \cup J$ 的大小满足 $|I| < m$ ，则每个 P_i ($i \in I$) 采样随机数 $r'_{i,i'}$ 发送给每个 $P_{i'}$ ($i' \in \{1, \dots, m\} \setminus I$)，并更新 $r_i = r_i \oplus r'_{i,i'}$ 。 $P_{i'}$ 更新 $r_{i'} = r_{i'} \oplus r'_{i,i'}$ 。
5. 每个 P_i ($1 \leq i \leq m$) 输出 r_i 的最终值。

图 3.4 多方秘密分享随机不经意传输协议 $\Pi_{\text{mss-rot}}$

为证明该定理，考虑如下特定情形：设 $\text{ch}_0 = 1$, $\text{ch}_1 = m$, 且 $J = \{1, 2, \dots, m\}$ 。即参与方 P_1 与 P_m 分别输入选择比特秘密分享 $b_1, b_m \in \{0, 1\}$, 并且每个参与方 P_i ($1 \leq i \leq m$) 输入 Δ_i 。其他情形的证明与此类似。

正确性分析. 根据协议描述，可以得出以下等式：

$$r_1 = \bigoplus_{j=2}^m (r_{j,1}^{b_1} \oplus b_1 \cdot u_{j,1}) \oplus r_{1,m}^0 \oplus b_1 \cdot \Delta_1, \quad (3.1)$$

$$r_i = r_{i,1}^0 \oplus r_{i,m}^0, 1 < i < m, \quad (3.2)$$

$$r_m = \bigoplus_{j=1}^{m-1} (r_{j,m}^{b_m} \oplus b_m \cdot u_{j,m}) \oplus r_{m,1}^0 \oplus b_m \cdot \Delta_m, \quad (3.3)$$

$$u_{j,1} = \Delta_j \oplus r_{j,1}^0 \oplus r_{j,1}^1, u_{j,m} = \Delta_j \oplus r_{j,m}^0 \oplus r_{j,m}^1 \quad (3.4)$$

根据 ROT 的功能定义，存在以下关系：

$$r_{j,1}^{b_1} = r_{j,1}^0 \oplus b_1 \cdot (r_{j,1}^0 \oplus r_{j,1}^1), \quad (3.5)$$

$$r_{j,m}^{b_m} = r_{j,m}^0 \oplus b_m \cdot (r_{j,m}^0 \oplus r_{j,m}^1), \quad (3.6)$$

将等式 (3.4)、(3.5)、(3.6) 代入等式 (3.1)、(3.2) 和 (3.3) 中消去相同项，可得：

$$r_1 = \bigoplus_{j=2}^m (r_{j,1}^{b_1} \oplus b_1 \cdot \Delta_j) \oplus r_{1,m}^0 \oplus b_1 \cdot \Delta_1, \quad (3.7)$$

$$r_m = \bigoplus_{j=1}^{m-1} (r_{j,m}^{b_m} \oplus b_m \cdot \Delta_j) \oplus r_{m,1}^0 \oplus b_m \cdot \Delta_m, \quad (3.8)$$

将等式 (3.2)、(3.7)、(3.8) 代入计算所有参与方输出的异或和 $r_1 \oplus (\bigoplus_{j=2}^{m-1} r_j) \oplus r_m$, 可得 $r_1 \oplus (\bigoplus_{j=2}^{m-1} r_j) \oplus r_m = \bigoplus_{i=1}^m (b_1 \oplus b_m) \cdot \Delta_i$ 。由此可以总结出：若 $b_1 \oplus b_m = 0$, 则 $r_1 = \bigoplus_{j=2}^m r_j$; 否则 $r_1 = \Delta_1 \oplus (\bigoplus_{j=2}^m (r_j \oplus \Delta_j))$ 。这与理想功能 $\mathcal{F}_{\text{mss-rot}}$ 的定义完全一致，从而证明协议的正确性。

安全性分析. 设被敌手腐化的参与方集合为 Corr , 诚实方集合为 $\mathcal{H}$, 其中 $|\text{Corr}| = t$ 。

直觉上，各参与方的操作均仅限于调用 $\mathcal{F}_{\text{ROT}}$ 和接收随机消息，因此模拟器可以通过生成与诚实方输入无关的随机值来模拟 $\mathcal{F}_{\text{ROT}}$ 的输出和协议消息，然后通过调用 ROT 协议的子模拟器来模拟敌手的完整视图。具体地，模拟器接收所有被腐化方 $P_c \in \text{Corr}$ 的输出 r_c , 并需要模拟每个 P_c 的视图。我们根据持有选择比特秘密分享的 P_1 和 P_m 是否被腐化，分为以下三种情况证明所有参与方的输出和敌手视图的联合分布在理想世界和真实协议中不可区分：

情况 1: P_1 和 P_m 均未被腐化. 每个 P_c 的视图包括其输入的掩码分享 Δ_c 以及 $\mathcal{F}_{\text{ROT}}$ 的输出 $(r_{c,1}^0, r_{c,1}^1)$ 和 $(r_{c,m}^0, r_{c,m}^1)$ 。模拟器先随机生成 Δ_c 和上述 $\mathcal{F}_{\text{ROT}}$ 的输出，同时保证约束条件 $r_{c,1}^0 \oplus r_{c,m}^0 = r_c$ 。然后，模拟器扮演诚实方的角色按照协议与 P_c 交互。

由于这些值在真实执行中同样是均匀随机分布的，因此所有被腐化方的输出 r_c 及其由模拟器生成的视图组成的联合分布，与真实协议中所有输出和敌手视图的联合分布计算不可区分。

情况 2: 只有 P_1 被腐化或只有 P_m 被腐化. 利用协议在 P_1 和 P_m 角色上的对称性，该情况只需要考虑 P_1 被腐化的情形。 P_1 的视图包含选择比特分享份额 b_1 、输入的掩码分享 Δ_1 、 $\mathcal{F}_{\text{ROT}}$ 的输出 $(r_{1,m}^0, r_{1,m}^1)$ 和 $\{r_{j,1}^{b_1}\}_{1 \leq j \leq m}$ ，以及来自 P_j 的协议消息 $\{u_{j,1}\}_{1 \leq j \leq m}$ ；对于每个 $P_c (c \neq 1)$ ，其视图包含其输入的掩码分享 Δ_c 以及 $\mathcal{F}_{\text{ROT}}$ 的输出 $(r_{c,1}^0, r_{c,1}^1)$ 和 $(r_{c,m}^0, r_{c,m}^1)$ 。针对 P_1 的视图，模拟器扮演诚实方的角色按照协议与 P_1 交互，同时在保证 $\bigoplus_{j=2}^m (r_{j,1}^{b_1} \oplus b_1 \cdot u_{j,1}) \oplus r_{1,m}^0 \oplus b_1 \cdot \Delta_1 = r_1$ 成立的前提下，随机生成 $\mathcal{F}_{\text{ROT}}$ 的均匀输出 $(r_{1,m}^0, r_{1,m}^1)$ 、 $r_{i,1}^{b_1}$ 以及来自诚实方 $P_i \in \mathcal{H}$ 的均匀随机消息 $u_{i,1}$ ；对于其他被腐化方的视图，模拟器扮演诚实方的角色按照协议执行，同时设置 $r_{c,m}^0 = r_{c,1}^0 \oplus r_c$ 并随机生成 $\mathcal{F}_{\text{ROT}}$ 的均匀输出 $r_{c,m}^1$ 。在真实协议的执行中， P_1 接收到的消息为 $u_{i,1} = \Delta_i \oplus r_{i,1}^0 \oplus r_{i,1}^1$ 。根据 ROT 功能定义， $r_{i,1}^0$ 和 $r_{i,1}^1$ 都是均匀分布且独立于 P_1 视图的。因此，消息 $u_{j,1}$ 对 P_1 来说是均匀随机的，所有输出和敌手视图的联合分布在理想世界和真实协议中不可区分。

情况 3: P_1 和 P_m 均被腐化. P_1 的视图由选择比特分享份额 b_1 、输入的掩码分享 Δ_1 、 $\mathcal{F}_{\text{ROT}}$ 的输出 $(r_{1,m}^0, r_{1,m}^1)$ 和 $\{r_{j,1}^{b_1}\}_{1 \leq j \leq m}$ ，以及来自 P_j 的协议消息 $\{u_{j,1}\}_{1 \leq j \leq m}$ 组成；对于每个 $P_c (c \neq 1, c \neq m)$ ，其视图由其输入的掩码分享 Δ_c 以及 $\mathcal{F}_{\text{ROT}}$ 的输出 $(r_{c,1}^0, r_{c,1}^1)$ 和 $(r_{c,m}^0, r_{c,m}^1)$ 组成； P_m 的视图由输入的掩码分享 Δ_m 、 $\mathcal{F}_{\text{ROT}}$ 的输出 $(r_{m,1}^0, r_{m,1}^1)$ 和 $\{r_{j,m}^{b_m}\}_{1 \leq j \leq m}$ ，以及来自 P_j 的协议消息 $\{u_{j,m}\}_{1 \leq j \leq m}$ 组成。针对 P_1 的视图，模拟器扮演诚实方的角色按照协议与 P_1 交互，同时在保证 $\bigoplus_{j=2}^m (r_{j,1}^{b_1} \oplus b_1 \cdot u_{j,1}) \oplus r_{1,m}^0 \oplus b_1 \cdot \Delta_1 = r_1$ 成立的前提下，随机生成 $\mathcal{F}_{\text{ROT}}$ 的均匀输出 $r_{i,1}^{b_1}$ 以及来自诚实方 $P_i \in \mathcal{H}$ 的均匀随机消息 $u_{i,1}$ ；针对 $P_c (c \neq 1, c \neq m)$ 的视图，模拟器扮演诚实方的角色按照协议与 P_c 交互，同时设置 $r_{c,m}^0 = r_{c,1}^0 \oplus r_c$ 并随机生成 $\mathcal{F}_{\text{ROT}}$ 的均匀输出 $r_{c,m}^1$ ；针对 P_m 的视图，模拟器扮演诚实方的角色按照协议与 P_m 交互，除了对协议进行如下修改：

- 在保证约束条件 $\bigoplus_{j=1}^{m-1} (r_{j,m}^{b_m} \oplus b_m \cdot u_{j,m}) \oplus r_{m,1}^0 \oplus b_m \cdot \Delta_m = r_m$ 成立的前提下，随机生成 $\mathcal{F}_{\text{ROT}}$ 的均匀输出 $r_{i,m}^{b_m}$ 以及来自诚实方 $P_i \in \mathcal{H}$ 的均匀消息 $u_{i,m}$ 。
- 设置 $\mathcal{F}_{\text{ROT}}$ 的输出 $r_{c,m}^{b_m}$ 的值，使其与前述模拟中每个 $P_c (c \neq 1, c \neq m)$ 的部分视图 $(r_{c,m}^0, r_{c,m}^1)$ 保持一致。

可以看出，所有被腐化方的输出 r_c 及其由模拟器生成的视图组成的联合分布，与真实协议中所有输出和敌手视图的联合分布计算不可区分。

3.5 SK-MPSU 协议构造及安全性证明

本节将介绍如何利用 batch ssPMT 和 mss-ROT 构造 SK-MPSU 协议。该构造遵循引言中概述的基于秘密分享的 MPSU 框架：

在第一阶段中, 每个 P_j ($2 \leq j \leq m$) 执行差集 Y_j 的秘密分享过程。 P_j 通过布谷鸟哈希将 X_j 分配到 B 个桶中; 每个 P_i ($1 \leq i < j$) 通过简单哈希将 X_i 分配到 B 个桶中。 P_j 作为接收方 $\mathcal{R}$ 与每个 P_i 执行 batch ssPMT。对于每个桶, P_j 和 P_i 获得成员关系比特的秘密分享 $e_{j,i}$ 和 $e_{i,j}$ 。接着, $\{P_{\min(2,i)}, \dots, P_j\}$ 调用 mss-ROT, 其中 P_j 和 P_i 分别作为持有 $e_{j,i}$ 和 $e_{i,j}$ 的 P_{cho} 和 P_{ch1} , 而每个 $P_{j'} \in \{P_2, \dots, P_j\}$ 均匀采样 $\Delta_{j',ji}$ 作为输入。每个 $P_{j'} \in \{P_{\min(2,i)}, \dots, P_j\}$ 获得输出 $r_{j',ji}$ 。若 $e_{j,i} \oplus e_{i,j} = 0$, 则 $\bigoplus_{j'=\min(2,i)}^j r_{j',ji} = 0$; 否则 $\bigoplus_{j'=\min(2,i)}^j r_{j',ji} = \bigoplus_{j'=2}^j \Delta_{j',ji}$ 。在该过程结束时, P_1 设置自己的秘密分享份额 $s_{1,j} = r_{1,j1}$; 每个 $P_{j'} \in \{P_2, \dots, P_{j-1}\}$ 设置自己的秘密分享份额 $s_{j',j} = \bigoplus_{i=1}^{j-1} r_{j',ji}$; P_j 设置自己的秘密分享份额 $s_{j,j} = \bigoplus_{i=1}^{j-1} r_{j,ji} \oplus x \| \mathbf{H}(x)$, 其中 $x \in X_j$ 是 P_j 桶中的元素; 对于未参与 mss-ROT 调用的参与方, 将自己的秘密分享份额设为 0。此时有秘密分享之和为 $\bigoplus_{i=1}^j s_{i,j} = (\bigoplus_{i=1}^{j-1} (\bigoplus_{j'=\min(2,i)}^j r_{j',ji})) \oplus x \| \mathbf{H}(x)$ 。若 $x \in X_j \setminus (X_1 \cup \dots \cup X_{j-1})$, 则对于所有 i , $e_{j,i} \oplus e_{i,j} = 0$ 且 $\bigoplus_{j'=\min(2,i)}^j r_{j',ji} = 0$ 成立, 因此 $\bigoplus_{i=1}^j s_{i,j} = x \| \mathbf{H}(x)$ 。这使得参与方能够持有 $Y_j = X_j \setminus (X_1 \cup \dots \cup X_{j-1})$ 中每个元素的秘密分享。否则, 至少有一个随机项 $\bigoplus_{j'=\min(2,i)}^j r_{j',ji}$ 无法被消去, 并以秘密分享的形式分布在 $\{P_2, \dots, P_j\}$ 之间。因此, 对于 $X_j \setminus Y_j$ 中的每个元素, 参与方持有一个随机秘密的分享, 即使 P_1 与 $\{P_2, \dots, P_j\}$ 中最多 $m-2$ 个参与方共谋, 该秘密也不会泄露任何信息。

在第二阶段中, 所有参与方调用多方秘密分享洗牌协议, 对第一阶段生成的所有秘密分享进行随机置换和重新分享。随后, 每个 P_j 将其置换后的分享发送给 P_1 , 由 P_1 重构 $(X_1 \cup \dots \cup X_m) \setminus X_1$, 再与 X_1 合并, 从而得到最终并集。

图 3.5 形式化地描述了 SK-MPSU 协议的具体执行步骤。

定理 3.3. 在 $(\mathcal{F}_{\text{bssPMT}}, \mathcal{F}_{\text{mss-rot}}, \mathcal{F}_{\text{shuffle}})$ -混合模型下, 协议 $\Pi_{\text{SK-MPSU}}$ 在任意半诚实敌手腐化不超过 $t < m$ 个参与方的情况下, 安全地实现了功能 $\mathcal{F}_{\text{mpsu}}$ 。

证明. 根据 P_1 是否被腐化, 本证明分为两种情况。模拟器接收被腐化方 $P_c \in \text{Corr}$ 的输入 X_c ; 若 $P_1 \in \text{Corr}$, 模拟器还接收 MPSU 协议的输出 $\bigcup_{i=1}^m X_i$ 。模拟器需要根据其接受的输出模拟被腐化方的视图。然而, $P_1 \in \text{Corr}$ 和 $P_1 \notin \text{Corr}$ 这两种情况下的模拟过程仅在输出的重构阶段存在差异, 因此为了简化描述, 我们将对之前阶段的模拟过程统一处理。

对于每个被腐化方 P_c , 其视图由其输入 X_c 、 $\mathcal{F}_{\text{bssPMT}}$ 和 $\mathcal{F}_{\text{mss-rot}}$ 的输出、 $\mathcal{F}_{\text{shuffle}}$ 输出的最终秘密分享份额 $\vec{s}h'_c$ 、随机采样的 Δ 秘密分享份额组成; 若 $P_1 \in \text{Corr}$, P_1 的视图还包括来自其他参与方 P_i 的 $m-1$ 个分享份额的集合 $\{\vec{s}h'_i\}_{1 \leq i \leq m}$ 。模拟器扮演诚实方的角色按照协议与 P_c 交互来模拟其视图, 除了对协议进行如下修改:

- **步骤 2.** 若 $P_i, P_j \in \mathcal{H}$, 随机生成 $\mathcal{F}_{\text{bssPMT}}$ 的输出比特 $\{e_{c,j}^b\}_{c < j \leq m}$ 和 $\{e_{c,i}^b\}_{1 \leq i < c}$ 。
- **步骤 3.** 若 $\min(2, i) \leq d \leq j$ 中包含诚实方 $P_d \in \mathcal{H}$, 随机生成 $\mathcal{F}_{\text{mss-rot}}$ 的均匀输出 $\{r_{c,ji}^b\}_{c < j \leq m, 1 \leq i < j}$ 。若 $c \neq 1$, 模拟器还需随机生成均匀的 $\{\Delta_{c,ji}^b\}_{c < j \leq m, 1 \leq i < j}$ 作为 P_c 的随机带。

参数. m 个参与方 $P_1, \dots, P_m$ 。输入集合大小为 n 。集合元素的比特长度为 l 。布谷鸟哈希参数：哈希函数 h_1, h_2, h_3 以及桶数 B 。抗碰撞哈希函数 $H(x) : \{0, 1\}^l \rightarrow \{0, 1\}^\kappa$ 。Elem($\mathcal{C}^b$) 表示 $\mathcal{C}^b$ 中的元素。

输入. 每个参与方 P_i 输入集合 $X_i = \{x_i^1, \dots, x_i^n\} \subseteq \{0, 1\}^l$ 。

1. P_1 执行 $\mathcal{T}_1^1, \dots, \mathcal{T}_1^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_1)$ 。对于 $1 < j \leq m$, P_j 执行 $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B \leftarrow \text{Cuckoo}_{h_1, h_2, h_3}^B(X_j)$ 以及 $\mathcal{T}_j^1, \dots, \mathcal{T}_j^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_j)$ 。
2. 对于每个 $1 < j \leq m$:
 - 对于 $1 \leq i < j$: P_i 和 P_j 调用批量大小为 B 的 $\mathcal{F}_{\text{bssPMT}}$, 其中 P_i 作为发送方输入 $\mathcal{T}_i^1, \dots, \mathcal{T}_i^B$, P_j 作为接收方输入 $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B$ 。对于实例 $b \in [B]$, P_i 获得 $e_{i,j}^b \in \{0, 1\}$, P_j 获得 $e_{j,i}^b \in \{0, 1\}$ 。
3. 对于每个 $1 < j \leq m$:
 - 对于 $1 \leq i < j$, 针对每个桶 $b \in [B]$: $P_{\min(2,i)}, \dots, P_j$ 调用 $\mathcal{F}_{\text{mss-rot}}$, 其中 P_j 作为 P_{ch_0} 输入 $e_{j,i}^b$, P_i 作为 P_{ch_1} 输入 $e_{i,j}^b$ 。对于 $1 < j' \leq j$, $P_{j'}$ 采样 $\Delta_{j',ji}^b \leftarrow \{0, 1\}^{l+\kappa}$ 并将其作为 Δ 的份额输入。对于 $\min(2,i) \leq i' \leq j$, $P_{i'}$ 获得输出 $r_{i',ji}^b \in \{0, 1\}^{l+\kappa}$ 。
 - P_1 设置 $s_{j,1}^b = r_{1,j1}^b$ 。对于 $1 < j' < j$, $P_{j'}$ 计算 $s_{j',j}^b = \bigoplus_{i=1}^{j'-1} r_{j',ji}^b$ 。若 $\mathcal{C}_j^b$ 不对应于空桶, 则 P_j 计算 $s_{j,j}^b = \bigoplus_{i=1}^{j-1} r_{j,ji}^b \oplus (\text{Elem}(\mathcal{C}_j^b) \| H(\text{Elem}(\mathcal{C}_j^b)))$; 否则, 随机选择 $s_{j,j}^b$ 。
4. 对于每个 $1 \leq i \leq m$, P_i 计算向量 $\vec{s}h_i \in (\{0, 1\}^{l+\kappa})^{(m-1)B}$ 如下: 对于 $\max(2,i) \leq j \leq m, 1 \leq b \leq B$, 令 $sh_{i,(j-2)B+b} = s_{i,j}^b$; 将其他所有位置设为 0。
5. 对于每个 $1 \leq i \leq m$, 所有参与方 P_i 以 $\vec{s}h_i$ 为输入调用 $\mathcal{F}_{\text{shuffle}}$ 。 P_i 获得输出 $\vec{s}h'_i$ 。
6. 对于每个 $1 < j \leq m$, P_j 将 $\vec{s}h'_j$ 发送给 P_1 。
7. P_1 恢复 $\vec{v} = \bigoplus_{i=1}^m \vec{s}h'_i$ 并初始化 $Y = \emptyset$ 。对于 $1 \leq i \leq (m-1)B$, 若存在 $x \in \{0, 1\}^l$ 使得 $v_i = x \| H(x)$ 成立, 则将 x 加入 Y 。最终输出 $X_1 \cup Y$ 。

图 3.5 SK-MPSU 协议 $\Pi_{\text{SK-MPSU}}$

- **步骤 4.** 随机生成 $\mathcal{F}_{\text{shuffle}}$ 的均匀输出 $\vec{s}h'_c$ 。

现在先讨论 $P_1 \in \text{Corr}$ 的情况。在步骤 4 和 5 中，模拟器计算 $Y = \bigcup_{i=1}^m X_i \setminus X_1$ ，并按照如下步骤构造向量 $\vec{v} \in (\{0, 1\}^{l+\kappa})^{(m-1)B}$ ：

- 对于所有属于 Y 的元素 $x_i \in Y$ ，令 $v_i = x_i \| H(x)$ ，其中 $1 \leq i \leq |Y|$ 。
- 对于 $|Y| < i \leq (m-1)B$ ，采样随机值 $v_i \leftarrow \{0, 1\}^{l+\kappa}$ 。

然后，模拟器采样一个随机置换 $\pi : [(m-1)B] \rightarrow [(m-1)B]$ 并计算 $\vec{v}' = \pi(\vec{v})$ 。对于 $1 \leq i \leq m$ ，模拟器采样份额 $\vec{s}h'_i \leftarrow (\{0, 1\}^{l+\kappa})^{(m-1)B}$ ，使其满足 $\bigoplus_{i=1}^m \vec{s}h'_i = \vec{v}'$ 且与之前为每个被腐化方 P_c 采样的 $\vec{s}h'_c$ 保持一致。最后，分别将所有 $\vec{s}h'_i$ 加入 P_1 的视图，并将 $\vec{s}h'_c$ 作为其来自 $\mathcal{F}_{\text{shuffle}}$ 的输出加入每个被腐化方 $P_{c'} (c' \neq 1)$ 的视图中。

以上对 $\mathcal{F}_{\text{bssPMT}}$ 和 $\mathcal{F}_{\text{mss-rot}}$ 的输出修改不会影响被腐化方 P_c 的视图：根据 $\mathcal{F}_{\text{bssPMT}}$ 的定义，当 $P_i, P_j \in \mathcal{H}$ ，输出 $e_{c,j}^b$ 和 $e_{c,i}^b$ 在敌手视角下作为 P_c 与 P_j 之间和 P_i 与 P_c 之间的秘密分享份额而呈均匀随机分布。根据 $\mathcal{F}_{\text{mss-rot}}$ 的定义，若 $e_{i,j}^b \oplus e_{j,i}^b = 0$ ，则来自 $\mathcal{F}_{\text{mss-rot}}$ 的每个输出 $r_{c,ji}^b$ 是 0 在 $P_{\min(2,i)}, \dots, P_j$ 之间的秘密分享；若 $e_{i,j}^b \oplus e_{j,i}^b = 1$ ，则是 $\bigoplus_{d=2}^j \Delta_{d,ji}^b$ 的秘密分享。因此，即使 P_c 与他人共谋，从敌手的角度来看， $r_{c,ji}^b$ 仍然呈均匀随机分布，因为总存在一个持有其中一份份额的诚实方 $P_d \in \mathcal{H} (\min(2, i) \leq d \leq j)$ 。

接下来只需证明 $\mathcal{F}_{\text{shuffle}}$ 的输出除了并集之外不会泄露任何其他信息，该输出的取值分为两种情况：当 $P_1 \notin \text{Corr}$ 时为 $\vec{s}h'_c$ ，当 $P_1 \in \text{Corr}$ 时为所有输出 $\{\vec{s}h'_i\}_{1 \leq i \leq m}$ 。在前一种情况较易处理下，由于输出 $\vec{s}h'_c$ 是所有参与方之间的秘密分享份额，因此从敌手的角度来看均匀随机分布的；在后一种情况下，对于所有 $1 < j \leq m$ ，考虑一个元素 $x \in X_j$ ，且 x 被 P_j 放入第 b 个桶中。在真实协议中，若不存在 $X_i (1 \leq i < j)$ 使得 $x \in X_i$ ，则对于所有 $1 \leq i < j$ ，有 $e_{i,j}^b \oplus e_{j,i}^b = 0$ 。根据图 3.3 中的 $\mathcal{F}_{\text{mss-rot}}$ 功能定义，在满足 $\bigoplus_{d=\min(2,i)}^j r_{d,ji}^b = 0$ 的条件下，每个 $r_{d,ji}^b$ 在 $\{0, 1\}^{l+\kappa}$ 中均匀随机分布。根据协议描述，可以推导出在满足 $\bigoplus_{d=\min(2,i)}^j u_{d,j}^b = x \| H(x)$ 的条件下，每个 $u_{d,j}^b$ 在 $\{0, 1\}^{l+\kappa}$ 中也是均匀随机分布的，也就是说，所有 $u_{d,j}^b$ 组成了 $x \| H(x)$ 在所有参与方之间的秘密分享。这与模拟过程完全一致。

若存在某个 $X_i (1 \leq i < j)$ 使得 $x \in X_i$ ，则 $e_{i,j}^b \oplus e_{j,i}^b = 1$ 。根据图 3.3 中的 $\mathcal{F}_{\text{mss-rot}}$ 功能定义，在满足 $\bigoplus_{d=\min(2,i)}^j r_{d,ji}^b = \bigoplus_{j'=2}^j \Delta_{j',ji}^b$ 的条件下，每个 $r_{d,ji}^b$ 是均匀随机分布的，其中每个 $\Delta_{j',ji}^b$ 由 $P_{j'} (1 < j' \leq j)$ 随机生成。根据协议描述，可以推导出在满足 $\bigoplus_{d=\min(2,i)}^j u_{d,j}^b = x \| H(x) \oplus \bigoplus_{j'=2}^j \Delta_{j',ji}^b \oplus r$ （其中 r 表示剩余项之和）的条件下，每个 $u_{d,j}^b$ 均匀随机分布。那么，即使 P_1 与他人共谋，从敌手的角度来看， $\bigoplus_{d=\min(2,i)}^j u_{d,j}^b$ 仍然是均匀随机分布的，因为总存在一个参与方 $P_{j'} \in \mathcal{H} (1 < j' \leq j)$ 随机且独立于诚实方输入地生成 $\Delta_{j',ji}^b$ 。对于所有空桶， $u_{d,j}^b$ 是均匀随机选择的，因此相应的 $\bigoplus_{d=1}^j u_{d,j}^b$ 也是均匀随机的，这与模拟一致。根据 $\mathcal{F}_{\text{shuffle}}$ 的定义，所有参与方在随机置换下对 $\bigoplus_{d=1}^j u_{d,j}^b$ 进行加法秘密分享，该置换能够抵抗任意被腐化方共谋攻击，并分别接收到 $\{\vec{s}h'_i\}$ 。综上所述， $\mathcal{F}_{\text{shuffle}}$ 的所有输出 $\{\vec{s}h'_i\}_{1 \leq i \leq m}$ 在真实执行和理想执行中的分布是相同的。 $\square$

3.6 复杂度分析与对比

3.6.1 SK-MPSU 的复杂度细节

本节对本章提出的 SK-MPSU 协议（图 3.5）中各阶段的具体开销进行详细分析。

1. 执行 batch ssPMT

对于所有 $1 \leq i < j \leq m$ ，参与方 P_i 与 P_j 均需调用一次 batch ssPMT。总体而言，每个参与方 P_j 共涉及 $m - 1$ 次 batch ssPMT 调用，其中在前 $j - 1$ 次调用中担任接收方 $\mathcal{R}$ ，在后 $m - j$ 次调用中担任发送方 $\mathcal{S}$ 。每次调用 batch ssPMT 的批量大小为 $B = 1.27n = O(n)$ ，其开销参考 3.3.4 节中的分析。

- **离线阶段：**单方计算复杂度为 $O(\gamma mn \log n)$ ，通信复杂度为 $O(t\lambda m \log(\gamma n/t))$ ，轮数复杂度为 $O(1)$ 。
- **在线阶段：**单方计算复杂度为 $O(mn)$ ，通信复杂度为 $O(\gamma mn)$ ，轮数复杂度为 $O(\log \gamma)$ 。

2. 执行 mss-ROT

每次调用 mss-ROT 相当于多个两方 OT 的执行，每个 OT 的执行包含两部分：在离线阶段，参与方调用随机选择比特 ROT，随后 $\mathcal{S}$ 向 $\mathcal{R}$ 发送一条长度为 $l + \kappa$ 的消息（其中 $\kappa = \sigma + \log(m - 1) + \log n$ ）；在在线阶段， $\mathcal{R}$ 发送 1 比特的去随机化消息以将 ROT 转换为指定输入版本。若 P_i 和 P_j 持有选择比特，则 P_i 和 P_j 与 $\{P_2, \dots, P_j\} \setminus \{P_i, P_j\}$ 之外的成员调用 OT。对于 $1 \leq i < j \leq m$ ，各方针对 B 个桶调用 mss-ROT，其中 P_i 和 P_j 持有两个选择比特。综合考虑 mss-ROT 的所有调用： P_1 需调用 $[\sum_{j=2}^m (j - 1)]B = \frac{m^2 B - mB}{2}$ 次两方 OT；对于 $1 < i \leq m$ ，每个参与方 P_i 需调用 $[(i - 1) + \sum_{j=i+1}^m 3(j - 2)]B = \frac{3m^2 B - 3i^2 B - 9mB + 11iB - 2B}{2}$ 次两方 OT。

- **离线阶段：**单方计算复杂度为 $O(m^2 n \log n)$ ，通信复杂度为 $O(t\lambda m^2 \log(n/t))$ ，轮数复杂度为 $O(1)$ 。
- **在线阶段：**单方计算复杂度为 $O(m^2 n)$ ，通信复杂度为 $O(m^2 n)$ ，轮数复杂度为 $O(1)$ 。

3. 执行多方秘密分享洗牌

本文采用文献 [62] 提出的多方秘密分享洗牌协议。在离线阶段，每个参与方生成一个随机置换并与其他所有参与方执行规模为 $(m - 1)B$ 、长度为 $l + \kappa$ 的 Share Translation 协议 [70]，以最终生成一组满足分享相关性 (share correlations) 的相关联向量。在在线阶段，各参与方利用这些相关联向量以相对于数据规模 n 线性的开销对输入的股份额向量实现高效置换和重分享。

	计算复杂度			通信复杂度		
	离线阶段	在线阶段		离线阶段	在线阶段	
		Leader	Client		Leader	Client
[1]	$(T + \gamma' + m)mn \log n$	$(T + \gamma' + m)mn$	$(T + \gamma')mn$	$\lambda m^2 n \log n$	$(T + \gamma')mn + lm^2 n$	$(T + \gamma')mn$
本文 SK-MPSU	$(\gamma + m)mn \log n$	$m^2 n$		$\lambda m^2 n \log n$	$\gamma mn + lm^2 n$	$(\gamma + l + m)mn$

表 3.1 LG 协议与 SK-MPSU 协议在离线和在线阶段的理论计算与通信复杂度对比

- **离线阶段：**单方计算复杂度为 $O(m^2 n \log(mn))$ ，通信复杂度为 $O(\lambda m^2 n \log(mn))$ ，轮数复杂度为 $O(1)$ 。
- **在线阶段：**对于 Leader P_1 ，计算复杂度为 $O(m^2 n)$ ，通信复杂度为 $O((l + \kappa)m^2 n)$ ；对于普通参与方 P_j ，计算复杂度为 $O(mn)$ ，通信复杂度为 $O((l + \kappa)mn)$ 。轮数复杂度为 $O(m)$ 。

4. 输出重构

对于每个 $1 < j \leq m$ ， P_j 在在线阶段将 $\vec{s}h'_j \in (\{0, 1\}^{l+\kappa})^{(m-1)B}$ 发送给 P_1 。 P_1 重构 $(m-1)B$ 个秘密，其中每个秘密拥有 m 个分享份额。

- **在线阶段：** P_1 的计算复杂度为 $O(m^2 n)$ ，通信复杂度为 $O((l + \kappa)m^2 n)$ ； P_j 的计算复杂度为 $O((l + \kappa)mn)$ ；轮数复杂度为 $O(1)$ 。

3.6.2 与 LG 协议的对比

表 3.1 中展示了 LG 协议与本章提出的 SK-MPSU 协议在离线阶段和在线阶段中各参与方理论计算与通信复杂度的对比，其中 n 为集合大小， m 为参与方数量。 λ 和 σ 分别表示计算和统计安全参数，且 $\lambda = 128, \sigma = 40$ 。 T 表示 SKE 解密电路中的 AND 门数量， $T \approx 600$ 。 l 表示输入元素的比特长度。 γ 为 OPRF 的输出长度。 γ' 为 SKE 密文比特长度，且 $\gamma' \approx \gamma$ 。 t 为对偶 LPN 中的噪声权重， $t \approx 128$ 。

3.7 本章小结

4 基于公钥体制的线性多方隐私集合求并协议

4.1 引言

在第三章中，我们回顾并分析了现有的多方隐私集合求并（Multi-party Private Set Union, MPSU）协议，并将其主要归纳为两类：基于公钥密码技术的 MPSU 和基于对称密钥技术的 MPSU。针对现有对称密钥 MPSU 协议在安全性方面依赖“非共谋假设”（Non-collusion Assumption）这一开放性问题，本文提出了批量秘密分享隐私成员测试（batch secret-shared private membership test, batch ssPMT）和多方秘密分享随机不经意传输（multi-party secret-shared ROT, mss-ROT）两个新组件，并在此基础上逐步构建了满足标准半诚实安全的基于对称密钥的 SK-MPSU 协议，解决了 MPSU 领域在效率与安全性之间的权衡问题。

然而，MPSU 领域仍存在另一个关键性的科学问题：目前无论是基于公钥技术还是对称密钥技术的 MPSU 协议，均无法同时实现线性的计算复杂度和通信复杂度。在 MPSU 的设定中，所谓“线性复杂度”，是指每个参与方的复杂度与所有参与方输入规模的总和呈线性关系。本文主要考虑平衡设定，即每个参与方持有的集合大小相同，记为 n 。设参与方数量为 m ，因此线性复杂度意味着每个参与方的复杂度同时随参与方数量 m 和集合大小 n 呈线性增长（即满足 $O(mn)$ 量级）。实现线性复杂度具有重要的实际意义：一方面，计算与通信复杂度与参与方数量 m 线性相关，意味着协议能够高效地扩展到包含大量参与方的多方协作场景；另一方面，复杂度与集合大小 n 线性相关，则是协议能够处理大规模数据集、具备实用性的前提。

现有 MPSU 协议在复杂度方面仍与理想的线性目标存在较大差距：早期的基于公钥密码技术的 MPSU 协议 [31, 48]，其计算复杂度往往随集合大小 n 呈高次增长（例如 $O(n^2)$ 或 $O(n^3)$ 量级），随着数据规模增大将带来巨大的计算开销，实际执行效率难以满足大规模应用需求。虽然文献 [40] 实现了相对于 n 线性的计算和通信复杂度，但方案中获得并集结果的 Leader 的计算和通信复杂度随参与方数量 m 呈二次增长。[54] 首次以相对于 n 拟线性（即 $O(n \log n / \log \log n)$ ）的代价实现了所有参与方的计算和通信复杂度随 m 线性增长的目标。另一方面，基于对称密钥技术的 MPSU 协议（包括第三章提出的 SK-MPSU）在复杂度上普遍存在随参与方数量 m 二次增长的问题。其根本原因在于：在该类协议底层的基于秘密分享（secret-sharing）的 MPSU 框架设计下，Leader 最终需要重构 $(m-1)n$ 个秘密，而每个秘密的重构过程都涉及 m 个参与方的分享份额，这种结构决定了其处理每个元素时都必须与所有参与方进行交互，因此整体计算与通信开销不可避免地包含 m^2n 量级的项。这一结构性限制表明，在基于秘密分享的 MPSU 框架下，实现相对于 m 的线性复杂度在理论上是不可行的。

协议	计算复杂度		通信复杂度		轮复杂度	半诚实安全性
	Leader	Client	Leader	Client		
[31]	m^2n^3 pub	m^2n^3 pub	λm^3n^2	λm^3n^2	m	✓
[48]	mn^2 pub	mn^2 pub	λmn	λmn	m	✓
[40]	lm^2n pub	lmn pub	λlm^2n	λlmn	l	✓
[54]	$mn(\log n / \log \log n)$ pub		$(\gamma + \lambda)mn(\log n / \log \log n)$		$\log \gamma + m$	✗
[1]	$(T + \gamma' + m)mn$ sym	$(T + \gamma')mn$ sym	$(T + \gamma')mn + lm^2n$	$(T + \gamma')mn$	$\log(l - \log n) + m$	✗
SK-MPSU-MPSU	m^2n sym	m^2n sym	$\gamma mn + lm^2n$	$(\gamma + l + m)mn$	$\log \gamma + m$	✓
PK-MPSU-MPSU	mn pub	mn pub	$(\gamma + \lambda)mn$	$(\gamma + \lambda)mn$	$\log \gamma + m$	✓

表 4.1 标准半诚实模型下 MPSU 协议的理论与通信复杂度对比

表 4.1 给出了现有 MPSU 协议与本文提出的方案在渐近复杂度上的理论对比。¹ 可以看出, 现有 MPSU 研究在理论上存在明显空白: 无论在标准半诚实模型还是依赖非共谋假设的非标准模型下, 设计一个同时具备线性计算复杂度和线性通信复杂度的 MPSU 协议, 仍然是一个尚未解决的挑战。本文仅关注标准半诚实安全, 因此本章提出以下关键的研究问题:

能否在标准半诚实模型下, 构造出同时实现线性计算复杂度和线性通信复杂度的 MPSU 协议?

本章围绕上述问题展开研究, 为了突破基于秘密分享的 MPSU 框架的结构性复杂度限制, 本章引入公钥加密机制, 在第三章提出的 batch ssPMT 的基础上, 利用多密钥重随机化公钥加密 (Multi-Key Rerandomizable Public-Key Encryption, MKR-PKE) 方案 [54], 在标准半诚实模型下提出了首个同时实现线性计算复杂度和通信复杂度的 MPSU 协议 (以下称该协议为 PK-MPSU 协议)。

在给出第三章提出的 SK-MPSU 协议和本章提出的 PK-MPSU 协议之后, 本章进行了全面的性能评估与基准测试。实验在不同网络环境下开展, 并覆盖多种参与方数量与集合规模的组合。通过与 SOTA 协议 [1] 进行多维度的综合对比 (包括在线与总运行时间、在线与总通信开销), 本文系统分析了两种协议在不同参数规模和网络条件下的性能表现与适用场景, 为实际部署提供了科学的参考依据。

4.2 技术路线概述

本节从整体层面对本章的研究思路与技术路线进行概述。本章将采用不同于第三章中基于秘密分享的 MPSU 框架的技术路线, 以目前渐近复杂度较优 ($O(mn \log n / \log \log n)$) 的基于公钥密码的 MPSU 方案 [54] (以下简称 GNT 协议) 为设计起点。首先, 本节将该协议抽象为一种基于多密钥重随机化公钥加密 (MKR-PKE) 的 MPSU 框架, 并以此

¹为了便于对比, 我们省略了大 O 符号, 并仅保留主要项。✓ 表示标准半诚实模型下的协议, ✗ 表示依赖非共谋假设的协议。pub: 公钥操作; sym: 对称密钥操作。 n 是集合大小, m 是参与方数量, λ 是计算安全参数。 l 是输入元素的比特长度, γ 是 OPPRF 的输出长度, γ' 是对称加密密文长度且 $\gamma' \approx \gamma$ 。

剖析 GNT 协议在复杂度上的局限性。随后，本节将重点论述如何利用第三章提出的批量秘密分享隐私成员测试 (batch ssPMT) 原语，在标准半诚实安全模型下重新设计差集的密文计算机制，从而突破现有技术的瓶颈，实现计算与通信复杂度均与 m 和 n 线性相关的“双线性”目标。

4.2.1 GNT 协议中的 MPSU 框架凝练

我们首先将 GNT 协议的核心逻辑凝练为一种基于 MKR-PKE 方案的 MPSU 框架。该框架的基本思想是将复杂的并集计算转化为一组差集的 MKR-PKE 密文计算过程，其中所有参与方持有 MKR-PKE 的公钥 pk ，而私钥 sk 的秘密分享份额分布在各参与方手中。这种设计确保了每个参与方均能对私有数据进行加密，且任何获得密文的个体即使与其余 $m - 2$ 个参与方共谋，也无法获取关于明文的任何有效信息。设共有 m 个参与方 $P_1, \dots, P_m$ ，各自输入集合 $X_1, \dots, X_m$ 。定义第 j 个参与方的差集为 $Y_j = X_j \setminus (X_1 \cup \dots \cup X_{j-1})$ ， $2 \leq j \leq m$ ，则有 $X_1 \cup \dots \cup X_m = X_1 \cup Y_2 \cup \dots \cup Y_m$ ，即所有的差集构成全集的一个划分。因此，只要 Leader（以下指定为 P_1 ）能够获得所有差集元素的 MKR-PKE 密文，并在最终阶段由所有参与方共同协作解密，即可得到并集结果。该框架具体可分为以下两个阶段：

1. **并集的密文计算**。该阶段涉及 $m - 1$ 次差集的密文计算，其中对于第 j 次密文计算过程 ($2 \leq j \leq m$)， P_j 先加密其集合 X_j 中的元素，随后依次与每个前序参与方 P_i ($1 < i < j$) 进行交互，使得：若元素 $x \in X_j$ 亦属于 X_i ，其密文将被“不经意地”替换为无意义消息 (dummy message)² 的密文。经过与所有前序参与方的交互后， P_j 获得加密后的差集 $Y_j = X_j \setminus (X_1 \cup \dots \cup X_{j-1})$ 并将其发送给 P_1 。
2. **协作解密与洗牌**。如果 P_1 能够直接按照原始顺序获得所有密文解密后的明文，它将能够利用密文排布的线性位置特征推断出每个元素的来源方。例如，若 P_1 得到的某个解密元素位于第一个长度为 n 的区块（即 Y_2 的位置），则 P_1 可断定该元素来自 P_2 的输入。这种对应关系会导致隐私泄露。为了解决这一问题，各参与方共同完成多方协作解密与洗牌 (collaborative decryption and shuffle) 过程，使得 P_1 仅能获得随机置换后的解密结果，且任何 $m - 1$ 方共谋都无法获知洗牌的置换映射，从而掩盖了明文中的元素与各差集 Y_j 之间的对应关系。最后， P_1 将所有明文中的无意义消息剔除，得到最终并集。

通过将 GNT 协议分为上述两个阶段，我们识别出其相对于 n 的超线性复杂度来源于其第一阶段中差集的密文计算过程的构造，而第二阶段的计算和通信复杂度已经满足了与 m 和 n 线性相关的“双线性”特征。因此，只要在保证标准半诚实安全的前提下，使得该过程中 P_j 每次与前序参与方 P_i ($1 < i < j$) 的交互过程的计算和通信开销随 n 线性增长，那么即可构建具备线性计算复杂度和通信复杂度的 MPSU 协议。

² 无意义消息是指取值范围在集合定义域之外的特殊标识符，通常预设为零字符串 $\perp$ 。

4.2.2 基于 batch ssPMT 和 MKR-PKE 的 PK-MPSU 协议

为实现上述目标,本节利用第三章介绍的批量秘密分享隐私成员测试(batch ssPMT)和双选择比特不经意传输 (two-choice-bit OT) 对基于 MKR-PKE 的 MPSU 框架中的第一阶段进行重新设计,其中差集 Y_j 的密文计算过程的具体构造如下: 令 P_j 通过布谷鸟哈希预处理其集合 X_j , 其余 $P_{i < j}$ 通过简单哈希预处理其集合 X_i 。 P_j 作为接收方与每个 P_i 执行 batch ssPMT。对于每个桶, P_j 和 P_i 分别获得桶中元素 x 关于输入集合的成员关系比特的秘密分享 $e_{j,i}$ 和 $e_{i,j}$ 。当迭代开始时 ($i = 2$), P_j 构造候选消息对 (m_0, m_1) , 其中 $m_0 = \text{Enc}(pk, x)$ 为元素密文, $m_1 = \text{Enc}(pk, \perp)$ 为无意义消息密文。 P_j 和 P_i 执行双选择比特 OT, 其中 P_j 与 P_i 分别以 $e_{j,i}$ 与 $e_{i,j}$ 作为选择比特, P_j 作为发送方输入 (m_0, m_1) , 而 P_i 作为接收方获得密文 $\text{ct} = m_{e_{j,i} \oplus e_{i,j}}$ 。根据双选择比特 OT 的功能: 若 $x \notin X_i$, 则 ct 是 x 的密文; 反之则为无意义消息的密文。交互结束后, P_i 对 ct 执行重随机化 (rerandomize) 得到 ct' 并返还给 P_j 。随后, P_j 同样重随机化 ct' 并更新 $m_0 \leftarrow \text{ct}'$, 同时重随机化 m_1 , 然后继续与下一参与方 P_{i+1} 重复上述交互过程。

在完成与所有 P_i ($1 < i < j$) 的交互后, 当且仅当对于所有 i 均满足 $x \notin X_i$ 时, P_j 最终持有的 m_0 才保留 x 的加密信息。由此, P_j 获得集合 $X_j \setminus (X_2 \cup \dots \cup X_{j-1})$ 中全部元素的密文。最后, P_j 与 P_1 执行类似的交互过程替换掉该集合中属于 X_1 的元素的密文, P_j 即可获得差集 $Y_j = X_j \setminus (X_1 \cup \dots \cup X_{j-1})$ 的密文。 P_j 将获得的差集密文发送给 P_1 。上述构造能够实现线性复杂度, 主要得益于两个关键因素:

1. **交互拓扑的优化.** 每个差集 Y_j 的密文计算过程设计为以 P_j 为中心的点对点的星型拓扑结构, 确保 P_j 与 P_i 之间的每一次交互仅涉及双方自身的输入集合, 而无需访问其他参与方的数据。
2. **底层组件的线性复杂度支撑.** 第三章提出的 batch ssPMT 协议本身具有线性的计算与通信复杂度。

由此, 上述设计在标准半诚实安全模型下实现了第一阶段的“双线性”构造, 通过与后续的协作解密与洗牌阶段相结合, 本文成功构建了首个同时满足线性计算复杂度和通信复杂度的 MPSU 协议。

4.3 协议构造及安全性证明

本节将介绍如何利用 batch ssPMT 和 MKR-PKE 构造 PK-MPSU 协议。该构造遵循引言中概述的基于公钥加密的 MPSU 框架:

在第一阶段, 在第一阶段中, 每个 P_j ($2 \leq j \leq m$) 执行差集 Y_j 的密文计算过程。具体地, P_j 首先与每个 P_i ($1 < i < j$) 交互实现密文的条件“不经意”替换: (1) P_j 通过布谷鸟哈希将 X_j 分配至 B 个桶中, P_i 通过简单哈希将 X_i 分配至 B 个桶中。 P_j 作

为接收方 $\mathcal{R}$ 与 P_i 执行 batch ssPMT, 双方获得份额 $e_{j,i}$ 和 $e_{i,j}$ 。(2) 若 $i = 2$, P_j 将桶中元素 $x \in X_j$ 加密作为 m_0 的初始值, 并将无意义消息加密作为 m_1 的初始值。(3) P_j 作为发送方 $\mathcal{S}$ 与 P_i 执行双选择比特 OT (该 OT 等价于标准的 1-out-of-2 ROT, 其中 $\mathcal{S}$ 的选择比特用于决定是否交换 m_0 和 m_1 的顺序)。 P_i 获得 $\text{ct} = m_{e_{j,i} \oplus e_{i,j}}$: 若 $x \notin X_i$, 则 ct 为 x 的密文; 否则为无意义消息的密文。(4) P_i 重随机化 ct 并返回给 P_j 。(5) P_j 再次重随机化并更新 m_0 , 重复此过程。与所有前序参与方交互后, 仅当 x 排除在所有 X_i 之外时, P_j 才保留其加密信息。由此, P_j 持有了 $X_j \setminus (X_2 \cup \dots \cup X_{j-1})$ 的加密形式。为了让 Leader P_1 获得 $X_j \setminus (X_1 \cup \dots \cup X_{j-1})$ 的密文, 双方执行类似过程。这从加密集合中剔除了属于 X_1 的元素, 并使 P_1 获得最终密文。

第二阶段的协作解密与洗牌运作如下: P_1 对密文进行重随机化与随机置换 π_1 , 随后发给 P_2 。 P_2 执行部分解密、重随机化与置换 π_2 后传给 P_3 。此过程循环至最后一名参与方 P_m 。最后, P_1 执行最终解密, 过滤掉无意义元素, 得到并集结果。

图 4.1 形式化地描述了 PK-MPSU 协议的具体执行步骤。

定理 4.1. 在 $(\mathcal{F}_{\text{ssPMT}}, \mathcal{F}_{\text{ROT}})$ 混合模型下, 若 MKR-PKE 方案满足多次加密不可区分性和可重随机化性, 协议 $\Pi_{\text{PK-MPSU}}$ 在任意半诚实敌手腐化不超过 $t < m$ 个参与方的情况下, 安全地实现了功能 $\mathcal{F}_{\text{mps}}^m$ 。

证明. 根据 P_1 是否被腐化, 本证明分为两种情况。模拟器接收被腐化方 $P_c \in \text{Corr}$ 的输入 X_c ; 若 $P_1 \in \text{Corr}$, 模拟器还接收 MPSU 协议的输出 $\bigcup_{i=1}^m X_i$ 。模拟器需要根据其接受的输出模拟被腐化方的视图。

对于每个被腐化方 P_c , 其视图由其输入 X_c 、 $\mathcal{F}_{\text{ssPMT}}$ 和 $\mathcal{F}_{\text{ROT}}$ 的输出、协议消息 $\{u_{j,c,0}^b\}_{c < j \leq m}$, $\{u_{j,c,1}^b\}_{c < j \leq m}$ 、重随机化消息 $\{v_{c,i}^b\}_{1 < i < c}$ 、随机置换 π_c 以及来自 P_{c-1} 置换后的部分解密消息 $\vec{\text{ct}}_{c-1}''$ (若 $c > 1$) 或来自 P_m 的消息 $\vec{\text{ct}}_m''$ (若 $c = 1$) 组成。模拟器扮演诚实方的角色按照协议与 P_c 交互来模拟其视图, 除了对协议进行如下修改:

- **步骤 2.** 若 $P_i, P_j \in \mathcal{H}$, 随机生成 $\mathcal{F}_{\text{ssPMT}}$ 的均匀输出 $\{e_{c,j}^b\}_{c < j \leq m}$ 和 $\{e_{c,i}^b\}_{1 \leq i < c}$ 。
- **步骤 3.** 若 $P_i, P_j \in \mathcal{H}$, 随机生成 $\mathcal{F}_{\text{ROT}}$ 的均匀输出 $\{r_{c,i,0}^b, r_{c,i,1}^b\}_{1 \leq i < c}$ 以及 $\{r_{j,c,e_{c,j}^b}^b\}_{c < j \leq m}$ 。对于 $c < j \leq m$, 若 $P_j \in \mathcal{H}$, 模拟器计算 $u_{j,c,e_{c,j}^b}^b = r_{j,c,e_{c,j}^b}^b \oplus \text{Enc}(pk, \perp)$ 并随机生成 $u_{j,c,e_{c,j}^b \oplus 1}^b$ 。对于 $1 < i < c$, 若 $P_i \in \mathcal{H}$, 模拟器计算 $v_{c,i}^b = \text{Enc}(pk, \perp)$ 。
- **步骤 6.** (在 $P_1 \notin \text{Corr}$ 的情况下) 若 $P_{c-1} \in \mathcal{H}$, 对于 $1 \leq i \leq (m-1)B$, 计算 $\text{ct}_{c-1}^i = \text{Enc}(pk, \perp)$, 然后使用随机采样的置换 π 打乱来自 P_{c-1} 的向量 $\vec{\text{ct}}_{c-1}'$, 即计算 $\vec{\text{ct}}_{c-1}'' = \pi(\vec{\text{ct}}_{c-1}')$ 。

现在先讨论 $P_1 \in \text{Corr}$ 的情况。在步骤 6 中, 假设 d 是诚实方中最大的编号, 即 $P_{d+1}, \dots, P_m \in \text{Corr}$ 。模拟器按照如下步骤构造部分解密消息 $\vec{\text{ct}}_d''$:

- 对于所有属于并集 $Y = \bigcup_{j=1}^m X_j$ 的元素 $x_i \in Y$, 令 $\text{ct}_d^i = \text{Enc}(pk_A, x_i)$, 其中 $1 \leq i \leq |Y|$ 。

参数. m 个参与方 $P_1, \dots, P_m$ 。输入集合的大小为 n 。集合元素的比特长度为 l 。布谷鸟哈希参数：哈希函数 h_1, h_2, h_3 和桶数 B 。一个 MKR-PKE 方案 $\mathcal{E} = (\text{Gen}, \text{Enc}, \text{ParDec}, \text{Dec}, \text{ReRand})$ 。

输入. 每个参与方 P_i 拥有输入 $X_i = \{x_i^1, \dots, x_i^n\} \subseteq \{0, 1\}^l$ 。

0. 对于 $1 \leq i \leq m$, P_i 运行 $(pk_i, sk_i) \leftarrow \text{Gen}(1^\lambda)$, 并将其公钥 pk_i 分发给其他参与方。定义 $sk = sk_1 + \dots + sk_m$, 所有参与方均可计算关联公钥 $pk = \prod_{i=1}^m pk_i$ 。
1. P_1 执行 $\mathcal{T}_1^1, \dots, \mathcal{T}_1^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_1)$ 。对于 $1 < j \leq m$, P_j 执行 $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B \leftarrow \text{Cuckoo}_{h_1, h_2, h_3}^B(X_j)$ 和 $\mathcal{T}_j^1, \dots, \mathcal{T}_j^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_j)$ 。
2. 对于 $1 < j \leq m$:
 - 对于 $1 \leq i < j$: P_i 和 P_j 调用批量大小为 B 的 $\mathcal{F}_{\text{bssPMT}}$, 其中 P_i 作为发送方, 输入为 $\mathcal{T}_i^1, \dots, \mathcal{T}_i^B$, P_j 作为接收方, 输入为 $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B$ 。对于实例 $b \in [B]$, P_i 收到 $e_{i,j}^b \in \{0, 1\}$, P_j 收到 $e_{j,i}^b \in \{0, 1\}$ 。
3. 对于 $1 < j \leq m$, 针对每个桶 $b \in [B]$:
 - P_j 定义 $\vec{c}_j$ 并设置 $c_j^b = \text{Enc}(pk, \text{Elem}(\mathcal{C}_j^b))$ 。Elem($\mathcal{C}_j^b$) 表示 $\mathcal{C}_j^b$ 中的元素。
 - 对于 $1 < i < j$:
 - P_i 和 P_j 调用 $\mathcal{F}_{\text{ROT}}$, 其中 P_i 作为接收方, 输入为 $e_{i,j}^b$, P_j 作为发送方。 P_i 收到 $r_{i,j}^b = r_{j,i,e_{i,j}^b}^b \in \{0, 1\}^\lambda$ 。 P_j 收到 $r_{j,i,0}^b, r_{j,i,1}^b \in \{0, 1\}^\lambda$ 。 P_j 计算 $u_{j,i,e_{j,i}^b}^b = r_{j,i,e_{j,i}^b}^b \oplus c_j^b$, $u_{j,i,e_{j,i}^b \oplus 1}^b = r_{j,i,e_{j,i}^b \oplus 1}^b \oplus \text{Enc}(pk, \perp)$, 然后发送 $u_{j,i,0}^b, u_{j,i,1}^b$ 给 P_i 。
 - P_i 定义 $v_{j,i}^b = u_{j,i,e_{i,j}^b}^b \oplus r_{i,j}^b$ 并将 $v_{j,i}^b = \text{ReRand}(pk, v_{j,i}^b)$ 发送给 P_j 。 P_j 更新 $c_j^b = \text{ReRand}(pk, v_{j,i}^b)$ 。
4. 对于 $1 < j \leq m$, 针对每个桶 $b \in [B]$:
 - P_1 和 P_j 调用 $\mathcal{F}_{\text{ROT}}$, 其中 P_1 作为接收方, 输入为 $e_{1,j}^b$, P_j 作为发送方。 P_1 收到 $r_{1,j}^b = r_{j,1,e_{1,j}^b}^b \in \{0, 1\}^\lambda$ 。 P_j 收到 $r_{j,1,0}^b, r_{j,1,1}^b \in \{0, 1\}^\lambda$ 。 P_j 计算 $u_{j,1,e_{j,1}^b}^b = r_{j,1,e_{j,1}^b}^b \oplus c_j^b$, $u_{j,1,e_{j,1}^b \oplus 1}^b = r_{j,1,e_{j,1}^b \oplus 1}^b \oplus \text{Enc}(pk, \perp)$, 然后将 $u_{j,1,0}^b, u_{j,1,1}^b$ 发送给 P_1 。
 - P_1 定义 $\vec{ct}_1' \in (\{0, 1\}^\lambda)^{(m-1)B}$, 并设置 $ct_1'^{(j-2)B+b} = \text{ReRand}(pk, u_{j,1,e_{1,j}^b}^b \oplus r_{1,j}^b)$ 。
5. P_1 采样 $\pi_1 : [(m-1)B] \rightarrow [(m-1)B]$, 计算 $\vec{ct}_1'' = \pi_1(\vec{ct}_1')$ 并发送给 P_2 。
6. 对于 $1 < j \leq m$:
 - 对于 $1 \leq i \leq (m-1)B$: P_j 计算 $ct_j^i = \text{ParDec}(sk_j, ct_{j-1}^i)$, $pk_{A_j} = pk_1 \cdot \prod_{d=j+1}^m pk_d$, 以及 $ct_j^i = \text{ReRand}(pk_{A_j}, ct_j^i)$ 。
 - P_j 采样 $\pi_j : [(m-1)B] \rightarrow [(m-1)B]$ 。令 $\vec{ct}_j' = (ct_j^1, \dots, ct_j^{(m-1)B})$, 计算 $\vec{ct}_j'' = \pi_j(\vec{ct}_j')$ 。如果 $j \neq m$, P_j 发送 $\vec{ct}_j''$ 给 P_{j+1} ; 否则, P_m 发送 $\vec{ct}_m''$ 给 P_1 。
7. P_1 设置 $Y = \emptyset$ 。对于 $1 \leq i \leq (m-1)B$, P_1 计算 $pt_i = \text{Dec}(sk_1, ct_m^i)$ 。如果 $pt_i \neq \perp$, 则更新 $Y = Y \cup \{pt_i\}$ 。输出 Y 。

 图 4.1 PK-MPSU 协议 $\Pi_{\text{PK-MPSU}}$

- 对于 $|Y| < i \leq (m-1)B$, 设置 $\text{ct}_d^i = \text{Enc}(pk_A, \perp)$ 。

其中 $pk_{A_d} = pk_1 \cdot \prod_{j=d+1}^m pk_j$ 。然后采样随机置换 π 并计算 $\vec{\text{ct}}_d'' = \pi(\vec{\text{ct}}_d')$ 。

对于其他被腐化方 $P_{d'+1} \in \{P_2, \dots, P_{d-1}\}$, 若 $P_{d'} \in \mathcal{H}$, 模拟器为 $1 \leq i \leq (m-1)B$ 模拟每个部分解密消息 $\text{ct}_{d'}^i = \text{Enc}(pk, \perp)$, 并采样均匀随机的 $\pi_{d'}$ 计算来自 $P_{d'}$ 的向量 $\vec{\text{ct}}_{d'}'' = \pi(\vec{\text{ct}}_{d'}')$ 。最后将 $\vec{\text{ct}}_{d'}''$ 添加至 $P_{d'+1}$ 的视图中。

上述对 $\mathcal{F}_{\text{bssPMT}}$ 和 $\mathcal{F}_{\text{ROT}}$ 输出的修改不会影响被腐化方 P_c 的视图, 其原因与定理 3.3 的证明类似; 而模拟器随机生成的 $u_{j,c,e_{c,j}^b \oplus 1}^b$ 与真实协议的敌手视图中相同: $u_{j,c,e_{c,j}^b \oplus 1}^b$ 在真实执行中是均匀随机分布的, 因为其中用于掩码加密消息的份额 $r_{j,c,e_{c,j}^b \oplus 1}^b$ (即 P_j 来自 $\mathcal{F}_{\text{ROT}}$ 的输出之一) 对 P_c 来说是均匀随机分布的, 并且独立于 $r_{j,c,e_{c,j}^b}^b$ 。

根据协议规范和模拟过程, 在满足 $e_{c,j}^b \oplus e_{j,c}^b = 1$ 的条件下, 模拟器生成的 $u_{j,c,e_{c,j}^b}^b$ 与真实协议的敌手视图中的分布完全一致。而对于 $e_{c,j}^b \oplus e_{j,c}^b = 0$ 的情况, 分析可进一步分为 $c \neq 1$ 和 $c = 1$ 两个子情况。首先论证当 $c \neq 1$ 时, 模拟器构造的 $u_{j,c,e_{c,j}^b}^b$ 与真实过程中的消息是不可区分的: 在真实执行中, 对于 $1 < c < j \leq m$ 且 $1 \leq b \leq B$: 若 $\text{Elem}(C_j^b) \in X_j \setminus (X_2 \cup \dots \cup X_{c-1})$, 则 $c_j^b = \text{Enc}(pk, \text{Elem}(C_j^b))$ 且 $u_{j,c,e_{c,j}^b}^b = r_{j,c,e_{c,j}^b}^b \oplus \text{Enc}(pk, \text{Elem}(C_j^b))$; 否则, $c_j^b = \text{Enc}(pk, \perp)$ 且 $u_{j,c,e_{c,j}^b}^b = r_{j,c,e_{c,j}^b}^b \oplus \text{Enc}(pk, \perp)$ 。而在模拟过程中, 对于 $1 < c < j \leq m, 1 \leq b \leq B$, $u_{j,c,e_{c,j}^b}^b = r_{j,c,e_{c,j}^b}^b \oplus \text{Enc}(pk, \perp)$ 。如果存在能够区分这两个过程的算法, 那么存在一个算法可以在不知道 sk 的情况下区分两组加密消息列表 (由于 sk 在 m 个参与方之间进行秘密分享, 对于任何由 $m-1$ 个参与方组成的联盟, 其分布都是均匀随机的)。因此, 这将导致存在一个敌手可以攻破 $\mathcal{E}$ 的多次加密不可区分性 (其中 $q = (m-1)B$); 当 $c = 1$ 时, 基于与上述 $c > 1$ 情况相似的理由, 模拟器构造的 $u_{j,e_{1,j}^b}^b$ 与真实过程也是不可区分的。

接下来, 通过一系列混合实验证明模拟器生成的 $v_{c,i}^b = \text{Enc}(pk, \perp)$ 与真实协议中的密文不可区分:

- Hyb_0 : 真实协议中的交互。对于 $1 < i < c$ 且 $1 \leq b \leq B$: 若 $\text{Elem}(C_c^b) \in X_c \setminus (X_1 \cup \dots \cup X_i)$, 则 $v_{c,i}^b = \text{Enc}(pk, \text{Elem}(C_c^b))$; 否则 $v_{c,i}^b = \text{Enc}(pk, \perp)$ 。随后计算 $v_{c,i}^b = \text{ReRand}(pk, v_{c,i}^b)$ 。
- Hyb_1 : 对于 $1 < i < c$ 且 $1 \leq b \leq B$: 若 $\text{Elem}(C_c^b) \in X_c \setminus (X_1 \cup \dots \cup X_i)$, 则 $v_{c,i}^b = \text{Enc}(pk, \text{Elem}(C_c^b))$; 否则 $v_{c,i}^b = \text{Enc}(pk, \perp)$ 。根据 $\mathcal{E}$ 的可重随机化性质, 这一改变相对于 Hyb_0 是不可区分的。
- Hyb_2 : 对于 $1 < i < c$ 且 $1 \leq b \leq B$, 令 $v_{c,i}^b = \text{Enc}(pk, \perp)$ 。根据 $\mathcal{E}$ 的多次加密不可区分性, 这一改变相对于 Hyb_1 是不可区分的。

当 $P_1 \in \text{Corr}$ 时, 通过以下混合实验证明, 模拟器生成的 $\vec{\text{ct}}_d''$ 与真实协议的敌手视图中的分布不可区分:

- Hyb_0 : 真实协议中的交互。 $\vec{ct}_1'' = \pi_1(\vec{ct}_1')$ 。对于 $2 \leq j \leq d$ 且 $1 \leq i \leq (m-1)B$: 计算 $ct_j^i = \text{ParDec}(sk_j, ct_{j-1}''^i)$, $ct_j^i = \text{ReRand}(pk_{A_j}, ct_j^i)$, 以及 $\vec{ct}_j'' = \pi_j(\vec{ct}_j')$ 。
- Hyb_1 : 对于 $2 \leq j \leq d$ 且 $1 \leq i \leq (m-1)B$, 令 $ct_j^i = \text{ParDec}(sk_j, ct_{j-1}^i)$ 。计算 $\vec{ct}_d' = \text{ReRand}(pk_{A_d}, \vec{ct}_d)$ 以及 $\vec{ct}_d'' = \pi(\vec{ct}_d')$, 其中 $\pi = \pi_1 \circ \dots \circ \pi_d$ 。显然, Hyb_1 与 Hyb_0 的分布是相同的。
- Hyb_2 : 将 $\vec{ct}_1$ 替换为如下向量:
 - 对于 $\forall x_i \in Y = \bigcup_{j=1}^m X_j$, $ct_1^i = \text{Enc}(pk, x_i)$, 其中 $1 \leq i \leq |Y|$ 。
 - 对于 $|Y| < i \leq (m-1)B$, 设置 $ct_1^i = \text{Enc}(pk, \perp)$ 。

由于敌手并不知道置换 π_d , $\vec{ct}_1$ 中元素的顺序对 $\vec{ct}_d''$ 的结果没有影响, 故 Hyb_2 与 Hyb_1 在视图上是相同的。

- Hyb_3 : 将 $\vec{ct}_d$ 替换为如下向量:
 - 对于 $\forall x_i \in Y = \bigcup_{j=1}^m X_j$, $ct_d^i = \text{Enc}(pk_{A_d}, x_i)$, 其中 $1 \leq i \leq |Y|$ 。
 - 对于 $|Y| < i \leq (m-1)B$, 设置 $ct_d^i = \text{Enc}(pk_{A_d}, \perp)$ 。

Hyb_3 与 Hyb_2 的不可区分性由 $\mathcal{E}$ 的部分解密性质所保证。

- Hyb_4 : 将 $\vec{ct}_d'$ 替换为如下向量:
 - 对于 $\forall x_i \in Y = \bigcup_{j=1}^m X_j$, $ct_d^i = \text{Enc}(pk_{A_d}, x_i)$, 其中 $1 \leq i \leq |Y|$ 。
 - 对于 $|Y| < i \leq (m-1)B$, 设置 $ct_d^i = \text{Enc}(pk_{A_d}, \perp)$ 。

基于 $\mathcal{E}$ 的可重随机化性质, Hyb_4 与 Hyb_3 是不可区分的。

- Hyb_5 : 唯一的改变是由模拟器均匀随机采样 π 。由于 π_d 在敌手视角下是均匀随机分布的, π 同样满足均匀随机分布, 因此 Hyb_5 与模拟过程生成的 $\vec{ct}_d''$ 是相同的。

当 $P_1 \notin \text{Corr}$ 时, 由于模拟器不知道最终并集, 因此必须在 P_c 的视图将部分解密消息 $\vec{ct}_{c-1}''$ 修改为置换后的 $\text{Enc}(pk, \perp)$ 。对比上述混合实验, 我们只需在 Hyb_4 之后增加一个额外的混合实验, 将所有重随机化后的部分解密消息 $\vec{ct}_{c-1}'$ 替换为 $\text{Enc}(pk, \perp)$ 。由于敌手不知道诚实方 P_1 的私钥 sk_1 , 因此无法获得部分私钥 $sk_{A_{c-1}} = sk_1 + sk_c + \dots + sk_m$ 的任何信息, 基于 $\mathcal{E}$ 的多次加密不可区分性, 其无法区分两组不同的密文列表, 综上, 该修改是不可区分的。

上述分析同样适用于 $P_1 \in \text{Corr}$ 时, 被腐化方 $P_{d'+1}$ ($d' \neq d$) 视图中部分解密消息 $\vec{ct}_{d'}''$ 的模拟, 即 $\vec{ct}_{d'}''$ 也利用置换后的 $\text{Enc}(pk, \perp)$ 进行类似的计算。为避免冗余, 在此省略具体分析。 $\square$

4.4 复杂度分析与对比

4.4.1 PK-MPSU 的复杂度细节

本节对本章提出的 PK-MPSU 协议（图 4.1）中各阶段的具体开销进行详细分析。

1. 执行 batch ssPMT

该阶段的开销与 $\Pi_{\text{SK-MPSU}}$ 协议中该阶段的开销（参见 3.6.1 节）完全一致。

- **离线阶段：**单方计算复杂度为 $O(\gamma mn \log n)$ ，通信复杂度为 $O(t\lambda m \log(\gamma n/t))$ ，轮数复杂度为 $O(1)$ 。
- **在线阶段：**单方计算复杂度为 $O(mn)$ ，通信复杂度为 $O(\gamma mn)$ ，轮数复杂度为 $O(\log \gamma)$ 。

2. 执行 ROT 与消息重随机化

对于 $1 \leq i < j \leq m$ ，在离线阶段， P_i 和 P_j 调用 B 个具有随机输入的 Silent ROT 实例。为了将每个 ROT 转化为 OT，在在线阶段， P_i 先发送 1 比特的去随机化消息，将每个 ROT 转换为指定输入版本；然后对于每个 OT 实例， P_j 执行两次加密，并向 P_i 发送两条 4λ 比特的消息。 P_i 执行一次重随机化。若 $i \neq 1$ ， P_i 向 P_j 发送一个密文，且 P_j 同样执行一次重随机化。本文基于椭圆曲线的 ElGamal 加密实例化 MKR-PKE 方案。因此，每次加密、部分解密和重随机化操作均相当于一次椭圆曲线上的点标量乘法运算。密文长度为 4λ 。

- **离线阶段：**单方计算复杂度为 $O(mn \log n)$ ，通信复杂度为 $O(t\lambda m \log(n/t))$ ，轮数复杂度为 $O(1)$ 。
- **在线阶段：**单方计算复杂度为 $O(mn)$ 次公钥操作，通信复杂度为 $O(\lambda mn)$ ，轮数复杂度为 $O(1)$ 。

3. 协作解密与洗牌

每个参与方在在线阶段将密文发送给下一方之前，对 $(m-1)B$ 个密文执行置换、部分解密及重随机化操作。

- **在线阶段：**单方计算复杂度为 $O(mn)$ 次公钥操作，通信复杂度为 $O(\lambda mn)$ ，轮数复杂度为 $O(m)$ 。

	计算复杂度		通信复杂度	
	离线阶段	在线阶段	离线阶段	在线阶段
[54]	$\gamma mn \log n (\frac{\log n}{\log \log n})$	$mn (\frac{\log n}{\log \log n})$	$t \lambda m \log n (\frac{\log n}{\log \log n})$	$(\gamma + \lambda) mn (\frac{\log n}{\log \log n})$
本文 PK-MPSU	$\gamma mn \log n$	mn	$t \lambda m \log n$	$(\gamma + \lambda) mn$

表 4.2 GNT 协议与 PK-MPSU 协议在离线和在线阶段的理论计算与通信复杂度对比

4.4.2 与 GNT 协议的对比

在表 4.2 中，我们展示了 GNT 协议与本章 PK-MPSU 协议在离线阶段和在线阶段中，每个参与方的理论计算和通信复杂度的对比情况，其中在离线阶段，计算由对称密钥操作组成；在在线阶段，计算主要由公钥操作组成。 n 为集合大小， m 为参与方数量， $\lambda = 128, \sigma = 40$ 。

我们可以得出结论：如表 4.2 所示，本章 PK-MPSU 协议的复杂度在所有方面，包括 Leader 和 Client 在离线与在线阶段的计算及通信开销，均优于 GNT 协议，并且实现了计算和通信线性复杂度。

4.5 MPSU 协议的实现与性能评估

为了全面评估本章提出的 PK-MPSU 协议以及第三章提出的 SK-MPSU 协议的实际表现，本文对这两个协议进行了实现，并与目前实际性能最好的 LG 协议 [1] 进行了多维度的性能对比与基准测试。需要指出的是，大多数早期 MPSU 工作 [31, 48, 36, 54] 均未提供公开代码实现，LG 协议的公开实现可从 <https://github.com/lx-1234/MPSU> 获取，而本文实现的代码已开源于 <https://github.com/real-world-cryptography/MPSU>。本节首先介绍协议实现的具体细节，然后给出实验环境与参数设置，最后分析实验结果。

4.5.1 实现细节

本文基于 C++ 实现了完整的 SK-MPSU 协议和 PK-MPSU 协议。实现过程中严格遵循协议设计，并对各个核心组件进行了模块化实现。在底层库的选择上：

- **VOLE.** 本文使用 libOTe [71] 提供的 VOLE 实现，并采用 Expand-Convolute codes [68] 实例化相关编码族。
- **OKVS 与 GMW.** 本文使用文献 [18] 中提出的优化 OKVS 构造，并复用了 [72] 中提供的 OKVS 实现。同时，本文复用 [72] 中的 GMW 实现以构造 ssPEQT 协议。

- **ROT.** 随机不经意传输采用 libOTe 中实现的 SoftSpokenOT [58], 并将有限域大小设置为 5 比特以在计算与通信之间取得平衡。
- **Share Correlation.** 本文复用了 [73] 中实现的 Permute+Share 方法 [74, 70] 来构造 SK-MPSU 与 LG 协议所需的 share correlation generation 过程。
- **MKR-PKE.** 在 PK-MPSU 协议中, 多密钥重随机化公钥加密 (MKR-PKE) 基于 openssl [75] 提供的椭圆曲线实现进行构建, 并采用曲线 NIST P-256 (亦称 secp256r1 或 prime256v1)。
- 此外, 本文使用 cryptoTools [76] 实现哈希函数与伪随机数生成, 并通过 Coproto [77] 实现网络通信。

4.5.2 实验结果分析

我们测试了参与方数量 $m \in \{3, \dots, 10\}$ 以及集合规模 $n \in \{2^6, \dots, 2^{20}\}$ 的多种组合。结果显示, 我们的协议在所有测试场景下均优于 LG 协议。

- **线性复杂度的优越性:** 理论对比 (表 4.1) 显示, PK-MPSU 是首个在标准半诚实模型下同时实现线性计算和通信复杂度的协议。实验数据进一步验证了这种优势, 尤其是在参与方数量 m 较多时, PK-MPSU 的开销增长远比非线性协议平缓。
- **广域网环境下的表现:** PK-MPSU 在带宽受限的情况下表现尤为突出。在 50 Mbps 的 WAN 网络中, 对于 9 个参与方、每方 2^{18} 个元素的大规模场景, LG 协议耗时长达 6.6 小时, 而本章 PK-MPSU 仅需 47 分钟。
- **扩展性:** 由于具备线性复杂度, PK-MPSU 是实验中唯一能够支持 10 个参与方、每方 2^{18} 个元素规模运行的协议, 充分证明了其“双线性”特征的实用价值。

4.5.3 实验结果与分析

本文在不同参与方数量 $m \in \{3, 4, 5, 7, 9, 10\}$ 以及不同集合规模 $n \in \{2^6, 2^8, 2^{10}, 2^{12}, 2^{14}, 2^{16}, 2^{18}, 2^{20}\}$ 的设置下对上述 MPSU 协议进行了大量的对比实验, 并分别在 LAN 与 WAN 网络环境中测试协议性能。实验从四个维度评估协议效率: 在线运行时间、总运行时间、在线通信开销以及总通信开销。实验结果如表 4.3 和 4.4 所示, 其中 m 表示参与方数量。本文中的 SK / PK 分别表示 SK-MPSU 协议/ PK-MPSU 协议。带有 * 的单元格表示在该实验中原始 LG 代码会产生错误结果。带有 - 的单元格表示实验运行时内存耗尽。在同一实验设置中性能最优的协议用加粗字体标记。

从实验结果可以观察到, 在所有测试场景中, 本文提出的 SK-MPSU 与 PK-MPSU 协议均优于 LG 协议。具体性能提升如下:

参数设置	m	协议	集合大小 n															
			在线阶段									总协议 = 离线阶段 + 在线阶段						
			2^6	2^8	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}	2^6	2^8	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
运行时间 (s) LAN	3	LG	0.050	0.058	0.069	0.143	0.425	1.582	6.219	25.16*	0.982	1.016	1.098	1.767	5.650	32.42	153.8	610.8*
		SK-MPSU	0.005	0.007	0.009	0.017	0.050	0.213	1.005	4.352	0.361	0.386	0.395	0.489	1.170	4.534	19.77	96.23
		PK-MPSU	0.092	0.237	0.822	3.176	12.69	51.17	205.7	829.1	0.117	0.273	0.868	3.270	13.08	53.05	215.0	872.6
	4	LG	0.069	0.076	0.093	0.162	0.478	1.637	6.375	25.79*	1.292	1.358	1.514	2.432	8.716	45.37	197.2	762.9*
		SK-MPSU	0.008	0.010	0.013	0.023	0.071	0.286	1.393	5.645	0.639	0.665	0.696	0.917	2.597	10.94	50.25	237.8
		PK-MPSU	0.159	0.465	1.570	6.136	24.60	44.91	398.4	1604	0.196	0.505	1.643	6.299	25.45	48.95	417.5	1694
	5	LG	0.107	0.109	0.126	0.190	0.541	1.998	7.588	31.33*	1.939	1.993	2.202	3.408	12.54	65.04	289.9	1152*
		SK-MPSU	0.012	0.013	0.017	0.030	0.087	0.368	1.714	7.003	0.914	0.964	1.023	1.558	4.551	18.71	85.79	373.4
		PK-MPSU	0.232	0.693	2.487	9.858	39.33	158.1	637.5	2816	0.292	0.796	2.547	10.09	40.25	162.2	655.0	2979
	7	LG	0.154	0.165	0.174	0.281	0.795	2.894	10.92	—	3.484	3.583	3.921	5.84	23.42	111.2	484.5	—
		SK-MPSU	0.019	0.021	0.028	0.048	0.156	0.607	2.817	—	2.051	2.079	2.214	3.470	12.92	56.50	245.6	—
		PK-MPSU	0.463	1.365	5.030	19.43	77.22	310.0	1247	—	0.550	1.469	5.158	19.73	78.59	316.3	1276	—
	9	LG	0.222	0.226	0.234	0.417	1.216	4.449	17.29	—	5.501	5.612	6.249	10.27	41.69	182.7	793.3	—
		SK-MPSU	0.027	0.031	0.039	0.075	0.230	0.970	4.293	—	3.363	3.402	3.895	7.161	30.09	126.3	678.8	—
		PK-MPSU	0.764	2.271	8.074	31.64	126.7	507.0	2039	—	0.880	2.375	8.238	32.19	128.7	515.5	2080	—
	10	LG	0.228	0.243	0.276	0.514	1.479	5.467	—	—	6.736	6.846	8.335	13.56	55.58	238.3	—	—
		SK-MPSU	0.031	0.036	0.043	0.088	0.286	1.183	—	—	3.965	4.232	4.821	9.334	41.97	175.9	—	—
		PK-MPSU	0.865	2.800	9.944	39.09	155.9	622.7	2503	—	0.970	2.912	10.11	39.58	158.4	634.6	2556	—
运行时间 (s) 400Mbps	3	LG	4.502	4.505	4.522	4.914	6.272	8.744	17.78	54.13*	12.46	13.59	15.76	19.52	30.73	78.74	282.8	1188*
		SK-MPSU	2.165	2.166	2.332	3.157	3.734	4.444	9.705	33.10	8.403	9.710	12.62	16.30	25.02	56.88	194.7	801.2
		PK-MPSU	4.419	4.555	5.553	7.984	18.21	59.77	226.3	900.3	5.168	5.306	6.472	8.968	19.65	62.73	237.3	946.1
	4	LG	5.696	5.880	6.540	7.094	7.323	11.36	23.13	78.81*	17.94	20.99	27.63	32.69	50.59	145.7	554.7	2167*
		SK-MPSU	2.967	2.969	3.298	3.976	4.618	6.507	17.10	59.21	11.46	13.93	20.21	26.77	47.49	133.2	520.0	2141
		PK-MPSU	5.622	5.899	7.929	12.17	32.40	113.8	440.9	1761	6.773	7.052	9.25	13.53	34.27	118.9	461.3	1857
	5	LG	7.385	7.708	8.621	9.198	9.687	14.62	32.83	126.3*	23.64	26.82	37.39	48.48	88.32	263.3	1053	4394*
		SK-MPSU	3.768	3.733	4.471	4.800	5.521	8.938	25.95	95.40	17.53	21.10	30.28	44.30	88.15	278.9	1119	4820
		PK-MPSU	6.849	7.928	10.50	17.08	49.75	179.8	703.3	2873	8.424	9.483	12.22	18.86	52.07	185.4	724.5	2980
	7	LG	9.312	9.833	10.55	11.35	12.14	21.29	66.93	—	34.92	45.69	66.26	94.89	203.4	705.8	2898	—
		SK-MPSU	5.373	5.381	6.207	6.644	8.164	17.67	56.894	—	34.00	44.95	61.34	95.03	228.7	817.4	3506	—
		PK-MPSU	9.504	12.32	15.14	29.73	92.66	348.5	1377.4	—	11.88	14.71	17.72	32.35	95.86	356.5	1409	—
	9	LG	11.41	12.21	13.34	14.41	15.09	33.84	115.5	—	56.65	75.24	104.1	169.1	406.0	1503	6387	—
		SK-MPSU	6.977	7.068	7.830	8.387	12.24	29.20	105.7	—	58.81	84.80	107.6	182.0	502.5	1915	8137	—
		PK-MPSU	13.67	18.84	22.96	45.54	148.2	570.9	—	—	16.87	22.04	26.31	48.98	152.3	581.5	2034	—
	10	LG	11.77	12.24	15.80	16.49	17.48	45.20	—	—	66.19	92.22	125.1	219.8	582.1	2179	—	—
		SK-MPSU	7.780	8.032	8.635	9.203	14.54	38.31	—	—	71.58	109.2	132.7	242.7	684.0	2687	—	—
		PK-MPSU	16.32	23.05	28.66	59.79	184.8	708.5	2792	—	19.90	26.66	32.44	63.66	189.3	720.9	2846	—
运行时间 (s) 50Mbps	3	LG	4.596	4.736	4.834	5.287	7.476	19.29	60.58	234.4*	13.05	14.54	18.58	25.31	46.49	149.4	610.0	2549*
		SK-MPSU	2.169	2.339	2.379	3.333	4.537	9.940	31.60	121.1	8.689	10.33	13.38	20.15	46.80	148.6	611.4	2766
		PK-MPSU	4.849	5.004	6.001	8.936	21.35	71.57	322.7	1120	5.603	5.762	6.929	9.904	22.71	74.20	332.1	1163
	4	LG	5.653	5.900	6.205	6.932	11.88	32.10	119.4	470.2*	18.05	21.43	29.09	42.78	102.2	367.5	1544	6812*
		SK-MPSU	3.131	3.147	3.372	4.271	6.979	18.83	69.46	269.7	12.26	15.16	22.27	39.99	110.1	411.1	1768	8078
		PK-MPSU	6.393	6.664	8.348	14.20	38.71	138.4	536.9	2136	7.549	7.832	9.683	15.57	40.48	142.76	554.6	2215
	5	LG	7.183	7.916	8.283	10.51	15.59	51.78	198.8	—	24.56	29.00	40.27	72.24	194.9	763.6	3308	—
		SK-MPSU	3.859	3.961	4.587	5.462	10.46	32.89	127.7	—	18.60	23.31	36.40	75.50	236.7	945.9	4105	—
		PK-MPSU	7.966	9.237	11.69	20.91	60.92	220.9	864.3	—	9.523	10.82	13.45	22.69	63.12	225.8	864.9	—
	7	LG	9.08	9.415	10.68	14.18	29.47	108.4	418.0	—	38.05	51.56	88.41	182.2	585.0	2423	10444	—
		SK-MPSU	5.471	5.594	6.468	8.883	22.92	80.96	318.9	—	36.57	49.54	83.742	201.8	714.3	2962	13265	—
		PK-MPSU	12.87	14.95	19.03	38.14	115.5	429.6	1693	—	15.25	17.38	21.60	40.75	118.6	437.0	1721	—
	9	LG	10.39	10.55	12.76	17.29	48.80	184.8	718.5	—	66.62	90.76	156.9	371.7	1314	5504	23814	—
		SK-MPSU	7.094	7.23	8.429	14.12	42.34	160.3	636.8	—	62.51	95.48	159.0	452.3	1708	7294	32303	—
		PK-MPSU	19.41	25.02	30.69	61.53	189.8	704.8	2793	—	22.67	28.34	34.13	65.06	193.8	714.4	2831	—
	10	LG	11.04	11.37	15.05	20.73	60.73	230.4	—	—	81.15	124.3	208.7	530.5	1921	8083	—	—
		SK-MPSU	7.826	8.222	9.636	17.56	55.45	214.5	—	—	76.53	122.1	211.7	620.8	2415	10418	—	—
		PK-MPSU	23.9	28.84	37.78	77.75	232.7	866.9	3440	—	27.61	32.53	41.67	81.74	237.2	878.9	3489	—

表 4.3 SK-MPSU、PK-MPSU 和 LG 协议在 LAN 与 WAN 环境下的在线 / 总运行时间

参数设置	m	协议	集合大小 n															
			在线阶段								总协议 = 离线阶段 + 在线阶段							
			2^6	2^8	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}	2^6	2^8	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
通信量 (MB)	3	LG	0.157	0.284	0.962	3.662	14.43	57.58	229.8	917.0*	6.311	7.904	13.37	32.58	114.6	474.3	2052	8973*
		SK-MPSU	0.032	0.111	0.426	1.690	6.788	27.87	112.7	455.6	2.542	3.582	8.529	30.91	132.7	588.8	2614	11529
		PK-MPSU	1.815	1.983	2.655	5.418	16.36	60.78	239.3	959.5	2.084	2.325	3.073	5.908	16.92	61.41	240.0	960.3
	4	LG	0.242	0.449	1.536	5.868	23.15	92.38	368.6	1471*	9.591	12.44	23.89	67.25	253.7	1074	4674	20419*
		SK-MPSU	0.058	0.204	0.791	3.145	12.81	51.69	208.8	843.7	3.941	6.385	16.96	66.89	293.9	1312	5834	25751
		PK-MPSU	2.723	2.975	3.983	8.126	24.54	101.9	359.0	1439	3.127	3.488	4.610	8.861	25.38	102.8	360.1	1440
	5	LG	0.330	0.630	2.173	8.325	32.86	131.2	523.5	2090*	12.92	17.75	36.48	110.7	435.2	1868	8228	35737*
		SK-MPSU	0.090	0.323	1.257	5.007	20.36	82.11	331.5	1339	5.504	10.18	30.44	124.7	548.9	2445	10832	47635
		PK-MPSU	3.630	3.967	5.311	10.84	32.72	121.6	478.7	1919	4.169	4.650	6.147	11.82	33.84	122.8	480.1	1921
	7	LG	0.519	1.038	3.635	13.99	55.29	220.8	881.3	—	19.92	29.90	41.59	243.3	997.9	4326	18924	—
		SK-MPSU	0.172	0.634	2.489	10.03	40.31	162.5	655.4	—	8.827	18.80	64.76	275.4	1226	5474	24275	—
		PK-MPSU	5.445	5.950	7.966	16.25	49.07	182.3	718.0	—	6.253	6.98	9.220	17.72	50.76	184.2	720.1	—
	9	LG	0.723	1.509	5.347	20.65	81.72	326.3	1303	—	28.15	44.74	115.0	415.33	1742	7609	33360	—
		SK-MPSU	0.279	1.046	4.122	16.60	66.76	269.0	1084	—	13.99	33.04	119.9	516.3	2295	10207	45079	—
		PK-MPSU	7.260	7.933	10.62	21.67	65.43	243.1	957.3	—	8.338	9.300	12.29	23.63	67.68	245.6	960.1	—
	10	LG	0.831	1.768	6.296	24.36	96.43	385.1	—	—	32.32	54.34	148.7	549.4	2314	10081	—	—
		SK-MPSU	0.341	1.288	5.086	20.48	82.39	332.0	—	—	16.21	40.56	149.9	651.6	2901	12908	—	—
		PK-MPSU	8.168	8.925	11.95	24.38	73.61	273.5	1077	—	9.380	10.46	13.83	26.59	76.14	276.3	1080	—

表 4.4 SK-MPSU、PK-MPSU 和 LG 协议在 LAN 与 WAN 环境下的在线 / 总通信开销

在线计算性能提升。在 LAN 环境下, SK-MPSU 协议相比 LG 实现了 $3.9 - 10.0\times$ 的加速; 在 400 / 50 Mbps 网络环境下, 相比 LG 分别实现了 $1.1 - 2.1\times$ / $1.1 - 2.6\times$ 的加速。例如, 在在线阶段计算 3 个参与方、每方集合规模为 2^{20} 的并集时, SK-MPSU 协议仅需 4.4 秒 (LAN), 以及 33.1 / 121.1 秒 (400 / 50 Mbps), 而 LG 分别需要 25.2 秒 (LAN) 和 54.1 / 234.4 秒 (400 / 50 Mbps), 因此分别实现了 $5.8\times$ 和 $1.64\times$ / $1.94\times$ 的性能提升。

总计算性能提升。在 LAN 环境下, SK-MPSU 协议相比 LG 实现了 $1.2 - 7.8\times$ 的加速。在 400 / 50 Mbps 网络环境下, PK-MPSU 协议相比 LG 分别实现了 $1.2 - 4.0\times$ / $1.8 - 9.2\times$ 的加速。例如, 在 3 个参与方、每方集合规模为 2^{20} 的并集计算中, SK-MPSU 协议在 LAN 环境下总运行时间为 96.2 秒, 而 LG 需要 610.8 秒, 因此实现了 $6.4\times$ 的加速。又例如, 在 9 个参与方、每方集合规模为 2^{18} 的并集计算中, PK-MPSU 协议在 400 / 50 Mbps 网络下的总运行时间为 38.4 / 47.2 分钟, 而 LG 分别需要 106.4 / 396.9 分钟, 因此分别实现了 $2.8\times$ / $8.5\times$ 的性能提升。

在线通信开销降低。SK-MPSU 协议的在线通信开销相比 LG 减少了 $1.2 - 4.9\times$ 。例如, 在在线阶段计算 3 个参与方、每方集合规模为 2^{20} 的并集时, SK-MPSU 协议需要 455.6 MB 的通信量, 而 LG 需要 917.0 MB, 因此通信开销降低了 $2.0\times$ 。

总通信开销降低。PK-MPSU 协议的总通信开销相比 LG 减少了 $3.0 - 36.5\times$ 。例如, 在计算 9 个参与方、每方集合规模为 2^{18} 的并集时, PK-MPSU 协议仅需要 960.1 MB 的通信量, 而 LG 需要 33360 MB, 因此通信开销降低了 $34.8\times$ 。

综上所述, 本文提出的协议在不同应用场景中均表现出明显优势, 并适用于不同的使用需求:

- **SK-MPSU。**该协议在 LAN 与 WAN 环境下均具有最优的在线性能。特别地, 在

LAN 环境中，当 3 个参与方各自持有 2^{20} 个元素的集合时，在线阶段仅需 4.4 秒；当 5 个参与方各自持有 2^{20} 个元素的集合时，在线阶段仅需 7 秒。在 WAN 环境下，由于在线通信开销更低，该协议在带宽较低的网络（50 Mbps）中相比 LG 取得了更显著的性能提升。此外，在 LAN 环境的大多数实验中，该协议也具有最优的总运行性能。因此，当在线性能是主要关注点，或者在高带宽网络中综合考虑总性能时，SK-MPSU 是最佳选择。

- **PK-MPSU**。该协议在 WAN 环境下具有最优的总运行性能，并且在带宽较低的网络（50 Mbps）中相比 LG 具有更显著的优势，这主要得益于其更低的总通信开销。例如，在 50 Mbps 网络环境中，当 9 个参与方各自持有 2^{18} 个元素的集合时，LG 完整执行协议需要约 6.6 小时，而 PK-MPSU 协议仅需 47 分钟。这表明 PK-MPSU 在带宽受限的网络环境中表现尤为突出。此外，从实验数据中可以看出，在参与方数量 m 较多时，PK-MPSU 的开销增长远比非线性协议平缓，并且 PK-MPSU 是实验中唯一能够支持 10 个参与方、每方 2^{18} 个元素规模运行的协议，充分证明了其“双线性”特征的实用价值。

4.6 本章小结

5 基于对称密钥的通用多方隐私集合运算框架

5.1 引言

在第三章和第四章中，本文重点研究了多方隐私集合求并（Multi-party Private Set Union, MPSU）这一特定功能，并分别基于对称密钥技术和公钥密码技术提出了满足标准半诚实安全的高效协议。然而，在实际应用中，多方隐私集合运算（MPSO）的需求远不止于单一的交集或并集计算。现实的数据协作场景通常需要对交、并、差等基本运算进行组合，从而形成更为复杂的集合公式计算。以下三个典型应用场景生动地说明了现实世界的数据共享与协作分析对集合复合公式计算的迫切需求：

- **公共卫生监测与决策支持.** 在传染病防控中，卫生部门需精准识别“已接种特定疫苗但仍被确诊感染”的人群。这要求首先联合多家疾控中心的接种名单计算并集 $\bigcup V_i$ ，随后与各医疗机构持有的确诊名单的并集 $\bigcup D_i$ 计算交集 $(\bigcup V_i) \cap (\bigcup D_i)$ ，其中 V_i 与 D_i 分别代表第 i 家疾控中心持有的接种名单与第 i 家医疗机构持有的确诊名单。此类分析对于评估疫苗有效性及制定后续防疫政策至关重要。
- **联合金融风控与反欺诈.** 在跨机构反欺诈场景中，银行联盟希望构建一个覆盖全行业的风险黑名单数据库，但为了保护高净值客户隐私，必须确保计算结果中排除了各行各业的白名单客户。该需求对应于计算复合公式 $(\bigcup B_i) \setminus (\bigcup W_i)$ ，其中 B_i 与 W_i 分别代表第 i 个银行持有的黑名单与白名单。
- **广告归因与转化效果评估.** 在广告归因分析中，广告商往往需要进行多维度的转化分析。例如，为衡量全渠道投放广告对线下购买的综合贡献，需计算所有投放平台浏览受众 V_i 的并集与购买列表 P 的交集 $(\bigcup V_i) \cap P$ ；为评估某新合作平台的独家拉新能力，这需要计算购买列表与新平台浏览该广告的受众的交集，并剔除所有其他平台的浏览受众的并集，即计算 $(P \cap V_{\text{new}}) \setminus (\bigcup V_{\text{others}})$ 。

针对上述需求中集合复合公式的安全计算问题，现有解决思路大体可分为两类：第一类思路是针对特定应用场景设计具体的 MPSO 协议。该类方法通常围绕特定功能进行深度优化，因此在目标场景下能够获得较高的执行效率。然而，该路线在实际应用中仍面临明显局限。一方面，学术界对复杂集合运算场景的关注相对有限，现有研究主要集中在交集与并集等基础且结构较为简单的功能上，导致许多实际应用需求（如前述三个典型应用场景）至今仍缺乏高效且可落地的具体解决方案。另一方面，由于不同协议往往基于异构的密码学技术路线独立设计，各协议之间缺乏统一的功能抽象与结构框架，整体表现出较差的灵活性与可扩展性。当应用需求发生变化时，系统往往需要重新设计或替换底层协议，从而显著增加系统部署、集成与运维成本，难以满足现实环境中

对敏捷部署与长期演进的需求；第二类思路是构建通用的 MPSO 框架，实现计算任意集合公式的通用 MPSO 功能。这一直是 MPSO 领域的重要研究目标，但实现该目标面临显著的技术挑战。早在该领域发展的早期，Kissner 与 Song 在其经典工作中 [31] 即尝试解决这一问题。然而，该框架受限于其采用的集合公式表示方式——将目标公式表示为交集、并集及元素规约 (element reduction) 操作的组合，仅能表达由交运算与并运算构成的集合表达式，无法支持差集运算。因此，该方法难以覆盖许多实际应用需求 (例如前述应用场景二与三中的相关实例)。此外，该框架高度依赖加法同态加密，计算成本极高，其效率根本无法满足现实世界的应用需求，因此仅具有理论价值。此后的二十年间，关于 MPSO 框架的研究进展十分有限，仅有 Blanton 和 Aguiar 提出的一项方案 [36] 基于通用 MPC 技术实现了完整的通用 MPSO 功能。尽管该方法在表达能力上具有完备性，但其依赖电路实现集合运算，导致计算和通信开销高。在现实规模数据场景下，其性能开销难以接受，从而限制了其在实际系统中的部署与应用。

综上所述，现有两类技术路线分别在灵活性与效率之间存在明显权衡：具体的 MPSO 协议效率较高但功能单一，而通用的 MPSO 框架表达能力完备却难以满足实际性能需求。这一矛盾表明，设计一种兼具通用性与高效率的任意集合公式计算框架，仍然是当前 MPSO 研究中亟待解决的关键问题。本章致力于解决这一长期悬而未决的开放性问题：

能否在标准半诚实模型下，构建出同时实现通用 MPSO 功能并且具备实际可用效率的 MPSO 框架？

本章围绕上述问题展开研究，从表示方法、密码学原语与系统框架三个层面提出了一套新的技术体系：

1. **表示层.** 本章引入了一种全新的任意集合公式表示方式——规范集合谓词公式 (Canonical Set Predicate Formula, CSPF)。该表示方式将目标集合公式表示为一类由原子集合谓词 (形如 $x \in X_i$ 或 $x \notin X_i$) 构成的析取范式，其中每个子公式满足特定的结构，子公式共同表示了目标集合的一个划分，从而将复杂的集合公式计算问题转化为若干个相互独立的子公式计算问题，为并行化与模块化计算提供基础。
2. **原语层.** 本章提出一种可组合的新型密码学原语——谓词零分享 (Predicative Zero-Sharing)，并在集合运算场景下给出其特化实例——成员零分享 (Membership Zero-Sharing)。该原语的核心功能是将指定逻辑谓词在各参与方输入上的真值“不经意地”编码为秘密分享的形式，并支持基于逻辑运算的组合计算。为实现成员零分享的高效构造，本文首先提出谓词零分享的松弛版本 (Relaxed Predicative Zero-Sharing) 的定义，并设计了相应的组合技术与转化技术，从而给出一种自底向上的高效构造范式：从关联于原子命题 (或其否定) 的松弛谓词零分享出发，逐步构造关联于任意复杂谓词公式、满足标准半诚实安全的谓词零分享协议。基于该

范式，本章通过分别针对成员原子集合谓词 $x \in X_i$ 和非成员原子集合谓词 $x \notin X_i$ 给出了成员零分享的松弛版本（Relaxed Membership Zero-Sharing）的高效实例化，最终实现了关联于任意集合谓词公式的标准安全成员零分享协议。该协议是本文提出的 MPSO 统一框架的核心底层组件，其高度的可组合性是本章提出的 MPSO 框架能够支持任意集合公式的核心依赖。

3. **框架层.** 基于 CSPF 表示法与成员零分享原语，并结合多方秘密分享洗牌协议，本章构建了支持任意集合公式计算的 MPSO 统一框架。该框架的基本思想是将输入集合中所有满足目标集合公式的元素转化为各方持有的秘密分享序列，这一思想赋予了该框架极强的扩展性，使得其能进一步实现任意集合公式结果求势（MPSO-card）及基于电路的通用 MPSO（Circuit-MPSO）功能，从而满足更复杂的数据分析需求。

本章后续内容安排如下：第 5.2 节提出规范集合谓词公式（CSPF）的定义，并证明任意集合公式转化为 CSPF 的等价性定理；第 5.3 节详细阐述谓词零分享原语的安全性定义、松弛定义以及组合与转化范式；第 5.4 节给出成员零分享协议的实例化及关联于任意集合谓词公式的标准安全成员零分享协议的高效构造；第 5.5 节给出 MPSO 统一框架的完整执行流程及安全性证明、复杂度分析；最后，第 5.6 节对本章内容进行总结。

5.2 集合公式的谓词公式表示

本节给出一种用于表示任意集合公式的统一形式——规范集合谓词公式（Canonical Set Predicate Formula, CSPF）。该表示方式是本章统一框架的表示层基础，通过把目标集合公式表示为若干个子公式的析取，将其对应的结果集合 Y 划分为若干个相互不相交的子集合 $\{Y_1, \dots, Y_s\}$ ，并且满足每个 Y_i 一定是某个输入集合的子集，从而为后续 MPSO 框架的设计和其中基于成员零分享的模块化计算提供结构支持。下面通过定义一系列概念来精确地刻画这些结构性质，从而形式化规范集合谓词公式的定义。

5.2.1 可构造集合与集合谓词公式

设 $X_1, \dots, X_m$ 为 m 个参与方的私有集合。我们首先形式化由这些集合通过任意集合公式的计算所能得到的结果集合，统称为 $X_1, \dots, X_m$ 上的可构造集合（constructible set）。

定义 5.1 (可构造集合). 给定集合 $X_1, \dots, X_m$ ，若集合 Y 可由 $X_1, \dots, X_m$ 经过有限次集合运算（包括求交、并、差集运算）导出，则称 Y 为 $X_1, \dots, X_m$ 上的可构造集合。¹

¹在上下文语境明确时，可简称为可构造集合。

特别地, 若对于某个 $1 \leq i \leq m$, 可构造集合 Y 满足 $Y \subseteq X_i$, 则称其为 $(X_1, \dots, X_m$ 上的) X_i -局部的可构造集合 (X_i -local constructible set)。

为了刻画集合元素级别的成员判定关系, 下面引入集合谓词公式的概念。

定义 5.2 (集合谓词公式). 若谓词公式 $\varphi(x, X_1, \dots, X_m)$ 由原子命题由形如 $M(x, X_i) : x \in X_i$ (M 称为集合成员谓词) 的原子命题组成, 则称其为集合谓词公式。

可构造集合和集合谓词公式之间存在如下对应关系。

定理 5.3. 设 Y 为 $X_1, \dots, X_m$ 上的可构造集合, 则存在一个集合谓词公式 $\varphi(x, X_1, \dots, X_m)$, 使得对于任意 x , 满足:

$$x \in Y \iff \varphi(x, X_1, \dots, X_m) = 1.$$

此时, 称集合谓词公式 φ 表示可构造集合 Y , 或称 φ 为 Y 的集合谓词公式表示。

证明. 下面通过对集合运算的次数进行数学归纳法构造 φ 。

- **基础情况.** 若存在某个 $i \in \{1, \dots, m\}$ 使得 $Y = X_i$, 显然 $\varphi(x, X_1, \dots, X_m) = M(x, X_i) : x \in X_i$ 。
- **归纳假设.** 假设对于由 $X_1, \dots, X_m$ 出发经过 k 次集合运算得到的任意集合 A 和 B , 均存在集合谓词公式 φ_A 和 φ_B , 使得:

$$x \in A \iff \varphi_A(x, X_1, \dots, X_m) = 1, \quad x \in B \iff \varphi_B(x, X_1, \dots, X_m) = 1.$$

- **归纳步骤.** 考虑由 A 和 B 经过一次额外的集合运算得到的集合 Y (即从 $X_1, \dots, X_m$ 出发需要 $k+1$ 次集合运算得到 Y), 根据集合运算的类型构造 φ 如下:

1. **并集.** 若 $Y = A \cup B$, 则 $\varphi(x, X_1, \dots, X_m) = \varphi_A(x, X_1, \dots, X_m) \vee \varphi_B(x, X_1, \dots, X_m)$ 。
2. **交集.** 若 $Y = A \cap B$, 则 $\varphi(x, X_1, \dots, X_m) = \varphi_A(x, X_1, \dots, X_m) \wedge \varphi_B(x, X_1, \dots, X_m)$ 。
3. **差集.** 若 $Y = A \setminus B$, 则 $\varphi(x, X_1, \dots, X_m) = \varphi_A(x, X_1, \dots, X_m) \wedge \neg \varphi_B(x, X_1, \dots, X_m)$ 。

以此类推, 即可为任意可构造集合 Y 构造相应的集合谓词公式 φ 。证毕。 $\square$

上述定理表明, 任意集合公式计算的算问题均可转化为元素级的集合谓词逻辑判定问题。但是为了适配后续 MPSO 框架的设计, 我们需要该表示具有特定的结构特性。下面先定义集合谓词公式相对于某一输入集合的“局部性”, 这一性质使得该公式在输入集合上的计算结果可通过执行一次成员零分享协议的实例来实现秘密分享。

定义 5.4 (局部性). 若集合谓词公式 $\varphi(x, X_1, \dots, X_m)$ 可以写成如下形式:

$$\varphi(x, X_1, \dots, X_m) = (x \in X_i) \wedge \psi(x, X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_m),$$

其中 $\psi(x, X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_m)$ 不包含关于 X_i 的原子命题, 则称 φ 相对于 X_i 是局部的 (*local to X_i*) 或称其为 X_i -局部的 (X_i -local) 集合谓词公式, 同时, 称 ψ 为 φ 的 X_i -局部剩余公式 (*residual formula*)。

推论 5.5. 若可构造集合 Y 的集合谓词公式表示为 X_i -局部的集合谓词公式, 则 Y 一定是 X_i -局部的可构造集合。

5.2.2 规范集合谓词公式表示

在上述概念的基础上, 下面正式给出规范集合谓词公式的定义。

定义 5.6 (规范集合谓词公式 (CSPF)). 设集合谓词公式 $\psi(x, X_1, \dots, X_m)$ 为一个或多个子公式的析取², 记作 $\psi = Q_1 \vee \dots \vee Q_s (s \geq 1)$ 。若 ψ 满足以下两个条件:

1. **局部性:** 每个子公式 Q_i ($1 \leq i \leq s$) 相对于某个输入集合 X_j ($1 \leq j \leq m$) 是局部的;
2. **划分性:** 令 Y 为 ψ 所表示的可构造集合, Y_i 为子公式 Q_i 所表示的可构造集合, $\{Y_1, \dots, Y_s\}$ 构成 Y 的一个划分, 即

$$Y_i \cap Y_k = \emptyset \ (i \neq k), \quad \bigcup_{i=1}^s Y_i = Y;$$

则称 $\psi(x, X_1, \dots, X_m)$ 为一个 (在 $X_1, \dots, X_m$ 上的) 规范集合谓词公式 (CSPF)。

以上 CSPF 的结构保证了其表示的目标集合可划分为若干个互不相交的局部子集, 从而可在 MPSO 框架下分别计算。下面的定理证明了这种表示法的普适性。

定理 5.7. 设 Y 为 $X_1, \dots, X_m$ 上的可构造集合, 则存在一个 CSPF $\psi(x, X_1, \dots, X_m)$, 使得对于任意 x , 满足:

$$x \in Y \iff \psi(x, X_1, \dots, X_m) = 1$$

证明. 根据定理 5.3, Y 可以表示为一个集合谓词公式。因为任何集合谓词公式都可以转化为析取范式 (Disjunctive Normal Form, DNF), Y 可以表示为一个 DNF 公式:

$$\varphi(x, X_1, \dots, X_m) = C_1 \vee \dots \vee C_n$$

²单个子公式的析取即为其自身。

其中每个 C_i ($1 \leq i \leq n$) 都是一个合取项。

首先, 利用反证法证明 φ 可以转化为另一个包含 s 个合取项 ($s > n$) 的 DNF 公式 ψ , 使得每个 C_i ($1 \leq i \leq s$) 至少包含一个形如 $x \in X_j$ 的原子命题, 即 C_i 可以写成 $C_i = (x \in X_j) \wedge D_i$ 的形式: 假设存在一个合取项 C_i 不包含任何形如 $x \in X_j$ 的原子命题, 即 C_i 是若干形如 $x \notin X_j$ 的原子命题的合取。考虑以下两种情况:

- **情况 1:** $C_i = (x \notin X_{i_1}) \wedge \cdots \wedge (x \notin X_{i_t})$, 其中 $t < m$ 。由于 C_i 是 φ 的一个合取项, 其对应的集合 Y'_i 是可构造集合 Y 的子集, 且 Y'_i 本身也是一个可构造集合。因此, 可以通过如下方式扩充 C_i :

$$C_i = C_i \wedge ((x \in X_1) \vee \cdots \vee (x \in X_m)) = (C_i \wedge (x \in X_1)) \vee \cdots \vee (C_i \wedge (x \in X_m)).$$

对于任何同时包含 $x \in X_{i_d}$ 及其否定 $x \notin X_{i_d}$ ($1 \leq d \leq t$) 的合取项, 其求值结果恒为 0, 可以舍弃。剩余的公式为:

$$C_i = (C_i \wedge (x \in X_{j_1})) \vee \cdots \vee (C_i \wedge (x \in X_{j_{m-t}})) = C_{i,1} \vee \cdots \vee C_{i,m-t}.$$

这将每个 C_i 拆分为 $m - t$ 个合取项, 其中每个合取项至少包含一个形如 $x \in X_j$ 的原子命题。将每个 C_i 替换为上述等式, 可得到新的 DNF 公式:

$$\psi(x, X_1, \cdots, X_m) = C_1 \vee \cdots \vee C_s,$$

其中每个 C_i ($1 \leq i \leq s$) 表示某个 X_j -局部的可构造集合, 但此时 C_i 尚未满足 X_j -局部的集合谓词公式的定义, 因为 D_i 中可能仍包含与 X_j 相关的原子命题。

- **情况 2:** $C_i = (x \notin X_1) \wedge \cdots \wedge (x \notin X_m)$ 。这与 C_i 表示的集合 Y_i 可由 $X_1, \cdots, X_m$ 通过集合运算得到相矛盾 (即元素不可能不属于任何输入集合, 却在可构造集合中), 因此这种情况不成立。

接下来, 将新的 DNF 公式 ψ 转化为表示互不相交集 $\{Y_1, \cdots, Y_s\}$ 的子公式析取形式: 由于 ψ 的析取形式蕴含 $Y_1 \cup \cdots \cup Y_s = Y$, 若能证明不相交性, 则证明了 $\{Y_1, \cdots, Y_s\}$ 构成 Y 的一个划分。令 $\psi_1(x, X_1, \cdots, X_m) = C_2 \vee \cdots \vee C_s$, 则有 $\psi(x, X_1, \cdots, X_m) = C_1 \vee \psi_1(x, X_1, \cdots, X_m)$ 。我们将此式扩充为 $C_1 \vee ((C_1 \wedge \neg C_1) \vee \psi_1(x, X_1, \cdots, X_m))$, 展开得到 $C_1 \vee (C_1 \wedge \psi_1(x, X_1, \cdots, X_m)) \vee (\neg C_1 \wedge \psi_1(x, X_1, \cdots, X_m))$ 。鉴于 $C_1 \wedge \psi_1 = 1$ 必然要求 $C_1 = 1$, 故 $C_1 \vee (C_1 \wedge \psi_1) = C_1$, 因此:

$$\psi(x, X_1, \cdots, X_m) = C_1 \vee (\neg C_1 \wedge \psi_1(x, X_1, \cdots, X_m)).$$

对所有的 C_i ($1 \leq i \leq s$) 重复此过程, 我们得到:

$$\psi(x, X_1, \cdots, X_m) = C_1 \vee (\neg C_1 \wedge C_2) \vee \cdots \vee (\neg C_1 \wedge \neg C_2 \wedge \cdots \wedge \neg C_{s-1} \wedge C_s).$$

记作 $\psi(x, X_1, \dots, X_m) = C'_1 \vee C'_2 \cdots \vee C'_s$ 。可以看出任意两个不同的 C'_i 和 C'_k 均满足 $C'_i \wedge C'_k = 0$ ，因此 C'_i 和 C'_k 表示的集合 Y_i 和 Y_k 满足 $Y_i \cap Y_k = \emptyset$ 。因此，每个 C'_i 表示一个互不相交的集合。

最后，证明每个 C'_i 都可以化简为 X_j -局部的集合谓词公式：根据定义， C'_i 是先前合取项 C_k ($1 \leq k < i$) 的否定与 C_i 的合取：

$$C'_i = \bigwedge_{k=1}^{i-1} \neg C_k \wedge C_i.$$

由于 C_i 可以写成 $(x \in X_j) \wedge D_i$ 的形式（其中 D_i 是合取项的剩余部分），代入 C'_i 得到：

$$C'_i = \bigwedge_{j=1}^{i-1} \neg C_j \wedge (x \in X_j) \wedge D_i = (x \in X_j) \wedge D'_i.$$

此时， D'_i 可能仍包含与 X_j 相关的原子命题。要消除这些冗余项从而满足定义 5.4，一个关键观察是：要使 $C'_i = 1$ 成立，条件 $x \in X_j$ 必须为真。因此，可以将 C'_i 中所有形如 $x \in X_j$ 的项赋值为真，所有形如 $x \notin X_j$ 的项赋值为假，从而得到一个等价公式 $Q_i = (x \in X_j) \wedge Q'_i$ 。因为 Q'_i 中不再包含任何与 X_j 相关的原子命题，因此 Q_i 满足 X_j -局部的集合谓词公式的定义。证毕。 $\square$

为了更直观地理解定理 5.7，下面给出三方场景下的 3 个可构造集合：交集 $Y = X_1 \cap X_2 \cap X_3$ 、并集 $Y = X_1 \cup X_2 \cup X_3$ 以及一个复合集合公式 $Y = ((X_1 \cap X_2) \cup (X_1 \cap X_3)) \setminus (X_1 \cap X_2 \cap X_3)$ ，并给出了每个可构造集合对应的 CSPF 表示 $\psi(x, X_1, X_2, X_3)$ ：

交集. $\psi(x, X_1, X_2, X_3) = (x \in X_1) \wedge (x \in X_2) \wedge (x \in X_3)$ 。在该情况下， ψ 是单个子公式 $Q_1 = \psi$ 的析取，其表示的集合 $Y_1 = Y$ 。 Q_1 相对于 X_1, X_2, X_3 均是局部的，并且 Y_1 本身即为 Y 的一个划分，因此 ψ 满足定义 5.6。

并集. $\psi(x, X_1, X_2, X_3) = (x \in X_1) \vee ((x \notin X_1) \wedge (x \in X_2)) \vee ((x \notin X_1) \wedge (x \notin X_2) \wedge (x \in X_3))$ 。 ψ 是三个子公式 Q_1, Q_2, Q_3 的析取，其中每个 $Q_i = (x \notin X_1) \wedge \cdots \wedge (x \notin X_{i-1}) \wedge (x \in X_i)$ 表示集合 $Y_i = X_i \setminus (X_1 \cup \cdots \cup X_{i-1})$ ，并且是 X_i -局部的集合谓词公式， $\{Y_1, Y_2, Y_3\}$ 是 Y 的一个划分，因此 ψ 满足定义 5.6。

复合集合公式. 在该情况下， Y 可表示为下面两种不同的 CSPF：

- $\psi(x, X_1, X_2, X_3) = ((x \in X_1) \wedge (x \in X_2) \wedge (x \notin X_3)) \vee ((x \in X_1) \wedge (x \in X_3) \wedge (x \notin X_2))$ 。 ψ 是两个子公式 Q_1, Q_2 的析取，其中 Q_1 和 Q_2 分别表示集合 $Y_1 = X_1 \cap X_2 \setminus X_3$ 和 $Y_2 = X_1 \cap X_3 \setminus X_2$ 。 Q_1 相对于 X_1 和 X_2 是局部的，而 Q_2 相对于 X_1 和 X_3 是局部的，并且 $\{Y_1, Y_2\}$ 是 Y 的一个划分，因此 ψ 满足定义 5.6。
- $\psi(x, X_1, X_2, X_3) = (x \in X_1) \wedge [((x \in X_2) \wedge (x \notin X_3)) \vee ((x \in X_3) \wedge (x \notin X_2))]$ 。将 ψ 看作单个子公式 $Q_1 = \psi$ 的析取，其中 Q_1 相对于 X_1 是局部的，因此 ψ 满足定义 5.6。

第三个例子表明，给定可构造集合 Y ，其 CSPF 表示通常并不唯一。不同的 CSPF 表示中对应的子公式数量将直接影响后续计算 Y 的 MPSO 协议的计算与通信开销，因此，在实际构造中应尽可能减少子公式数量以提升整体效率。

5.3 谓词零分享

本节引入一种新型密码学原语——谓词零分享 (Predicative Zero-Sharing)。该原语本质上是一类协议族，其中每个协议都与一个谓词公式相关联，其核心功能是将该公式在参与方输入上的真值“不经意地”编码为参与方之间的一个零分享（当真值结果为真时）或随机分享（当真值结果为假时）。该原语可以看作是多种现有 MPC 协议的统一抽象，并为后续 MPSO 框架的设计提供统一的组合接口。

5.3.1 功能定义

谓词零分享协议允许 m ($m \geq 2$) 个持有私有输入的参与方获得一组在有限域 $\mathbb{F}$ 上的秘密分享：如果与协议相关联的谓词公式 Q 在所有参与方的输入上的取值为真，那么参与方获得 0 的秘密分享；否则，参与方获得一个独立且均匀采样的随机值的秘密分享。该概率性功能 $\mathcal{F}_{\text{PZS}}^Q$ 的形式化定义如图 5.1 所示。

参数： m 个参与方 $P_1, \dots, P_m$ ，输入向量为 $\mathbf{x} = (x_1, \dots, x_m)$ ；有限域 $\mathbb{F}$ ；指定谓词公式 Q 。

功能： 收到每个参与方 P_i 的输入 x_i 后，随机采样分享值 $s_i \leftarrow \mathbb{F}$ ，且满足：若 $Q(\mathbf{x}) = 1$ ，则 $s_1 + \dots + s_m = 0$ 。将分享份额 s_i 发送给 P_i 。

图 5.1 谓词零分享功能 $\mathcal{F}_{\text{PZS}}^Q$

5.3.2 安全性松弛

谓词零分享的提出源于对许多现有 MPC 协议的高度抽象。鉴于该功能的概率性本质，这些协议必须满足定义 2.1 才能安全地计算谓词零分享。该定义蕴含以下结论：当 $Q(x) = 0$ 时，参与方之间分享的随机秘密值独立于任意 $m-1$ 个参与方的联合视图。在这些协议中，一些协议，例如第三章提出的 mss-ROT，其安全性严格遵循标准定义，实现了标准的谓词零分享功能；然而，我们注意到，更广泛的一类协议，例如 ROT 和等值条件随机数生成 (Equality-Conditional Randomness Generation, ECRG) [24] 实现的是

一种类似的功能，其不同之处在于：若 $Q(x) = 0$ ，则参与方之间分享的均匀随机秘密值并不独立于任意 $m - 1$ 个参与方的联合视图。下面以 ROT 为例说明 ROT 协议实现了该类松弛版本的谓词零分享 (Relaxed Predicative Zero-Sharing) 功能（其关联谓词公式为用于测试选择比特是否为 1 的简单谓词：设发送方 $\mathcal{S}$ 将其分享份额设置为 $s_1 = -r_0$ ，其中 r_0 是来自 ROT 的第一条消息；设接收方 $\mathcal{R}$ 将其分享份额设置为 $s_2 = r_e$ ，即接收到的消息。鉴于 ROT 的功能可以表示为 $-r_0 + r_e = e \cdot (-r_0 + r_1)$ ，若 $e = 0$ ，则 $s_1 + s_2 = 0$ ；否则， $s_1 + s_2 = -r_0 + r_1$ ，该值是均匀随机分布的，但依赖于发送方 $\mathcal{S}$ 视图中 ROT 的输出消息。

由于上述协议安全性的松弛难以基于定义 2.1 精准刻画，下面为谓词零分享引入一种等价的安全性定义，该定义由三个核心要求组成：正确性、隐私性和独立性。以下定理不仅给出了该定义的详细描述，并且严格声明了其标准安全性定义的等价性。

定理 5.8. 考虑一个概率多项式的 m 元功能 $\mathcal{F}^f$ ，其接收各参与方的输入 $\mathbf{x} = (x_1, \dots, x_m)$ ，并向各参与方输出 $f(\mathbf{x})$ 的秘密分享。令 Π 为用于计算 $\mathcal{F}^f$ 的 m 方协议， s_i 和 s_i^Π 分别用来表示参与方 P_i 在理想世界中从 $\mathcal{F}^f$ 获得的输出以及在真实世界中实际执行 Π 获得的输出 ($1 \leq i \leq m$)。若协议 Π 满足：

- **正确性.** Π 的输出构成 $f(\mathbf{x})$ 的秘密分享，即：

$$\{s_1, \dots, s_m\}_x \stackrel{s}{\approx} \{s_1^\Pi, \dots, s_m^\Pi\}_x$$

- **隐私性.** 存在一个 PPT 的算法 Sim ，使得对于任意规模为 $t < m$ 个的参与方子集 $\mathbf{P}_A = \{P_{i_1}, \dots, P_{i_t}\}$ ，满足：

$$\{\text{Sim}(\mathbf{P}_A, \mathbf{x}_A, \mathbf{s}_A)\}_x \stackrel{c}{\approx} \{\text{View}_A^\Pi(\mathbf{x})\}_x$$

- **独立性.** 在协议 Π 的实际执行过程中，对于任意规模为 $t < m$ 个的参与方子集 $\mathbf{P}_A = \{P_{i_1}, \dots, P_{i_t}\}$ ， $f(\mathbf{x})$ 所依赖的随机性独立于敌手的视图 $\text{View}_A^\Pi(\mathbf{x})$ 。

则存在一个 PPT 算法 Sim ，使得对于任意 $\mathbf{P}_A = \{P_{i_1}, \dots, P_{i_t}\}$ ，满足：

$$\{\text{Sim}(\mathbf{P}_A, \mathbf{x}_A, \mathbf{s}_A), s_1, \dots, s_m\}_x \stackrel{c}{\approx} \{\text{View}_A^\Pi(\mathbf{x}), s_1^\Pi, \dots, s_m^\Pi\}_x$$

即模拟器在理想世界中模拟的敌手视图和所有参与方从理想功能 $\mathcal{F}^f$ 获得的输出的联合分布与敌手在真实世界中的视图和实际执行协议 Π 获得的输出的联合分布是计算不可区分的。

证明. 我们通过对敌手 $\mathcal{A}$ 腐化的参与方数量 t 进行归纳来证明该定理。

– 基础情况：t = m - 1

假设 $\mathcal{A}$ 腐化了 $t = m - 1$ 个参与方。记被腐化参与方的集合为 $\mathbf{P}_A = \{P_{i_1}, \dots, P_{i_{m-1}}\}$ ，则仅剩一名诚实参与方 P_h 。根据隐私性要求，存在模拟器 Sim 满足：

$$\{\text{Sim}(\mathbf{P}_A, \mathbf{x}_A, \mathbf{s}_A)\}_x \stackrel{c}{\approx} \{\text{View}_A^\Pi(\mathbf{x})\}_x$$

令 $\mathbf{r}$ 和 $\mathbf{r}^\Pi$ 分别表示理想世界和真实世界中 $f(\mathbf{x})$ 依赖的随机性。显然，在理想世界中， $\mathbf{r}$ 独立于 $\text{Sim}(\mathbf{P}_A, \mathbf{x}_A, \mathbf{s}_A)$ 。同时，根据独立性要求，真实世界中的 $\mathbf{r}^\Pi$ 独立于 $\text{View}_A^\Pi(\mathbf{x})$ ，因此可以得出：

$$\{\text{Sim}(\mathbf{P}_A, \mathbf{x}_A, \mathbf{s}_A), \mathbf{r}\}_{\mathbf{x}} \stackrel{c}{\approx} \{\text{View}_A^\Pi(\mathbf{x}), \mathbf{r}^\Pi\}_{\mathbf{x}}$$

由于每个被腐化参与方 P_i 从理想功能获得的输出 s_i 和从协议实际执行获得的 s_i^Π 均可分别从其在理想世界和真实世界中的视图计算得出（其中 $i \in \{i_1, \dots, i_{m-1}\}$ ），因此可将上述不可区分性扩展为：

$$\{\text{Sim}(\mathbf{P}_A, \mathbf{x}_A, \mathbf{s}_A), \mathbf{s}_A, \mathbf{r}\}_{\mathbf{x}} \stackrel{c}{\approx} \{\text{View}_A^\Pi(\mathbf{x}), \mathbf{s}_A^\Pi, \mathbf{r}^\Pi\}_{\mathbf{x}}$$

其中 $\mathbf{s}_A^\Pi = (s_{i_1}^\Pi, \dots, s_{i_{m-1}}^\Pi)$ 。

根据功能定义，诚实参与方 P_h 的输出满足 $s_h = -(\sum_{s_i \in \mathbf{s}_A} s_i) + f(\mathbf{x})$ 。由正确性要求可知， $s_h^\Pi = -(\sum_{s_i^\Pi \in \mathbf{s}_A^\Pi} s_i^\Pi) + f(\mathbf{x})$ 。将不可区分符号两边的分布分别包含 s_h 和 s_h^Π ：

$$\{\text{Sim}(\mathbf{P}_A, \mathbf{x}_A, \mathbf{s}_A), \mathbf{s}_A, s_h, \mathbf{r}\}_{\mathbf{x}} \stackrel{c}{\approx} \{\text{View}_A^\Pi(\mathbf{x}), \mathbf{s}_A^\Pi, s_h^\Pi, \mathbf{r}^\Pi\}_{\mathbf{x}}$$

上述不可区分性仍成立，是因为 s_h （对应地， s_h^Π ）由 $\mathbf{s}_A$ （对应地， $\mathbf{s}_A^\Pi$ ）、随机性 $\mathbf{r}$ （对应地， $\mathbf{r}^\Pi$ ）以及各方输入 $\mathbf{x}$ 共同唯一确定。由此可推导出：

$$\{\text{Sim}(\mathbf{P}_A, \mathbf{x}_A, \mathbf{s}_A), \mathbf{s}_A, s_h\}_{\mathbf{x}} \stackrel{c}{\approx} \{\text{View}_A^\Pi(\mathbf{x}), \mathbf{s}_A^\Pi, s_h^\Pi\}_{\mathbf{x}}$$

即当敌手 $\mathcal{A}$ 腐化 $t = m - 1$ 个参与方时，协议 Π 能够安全地计算 f 。请注意，此时的独立性要求意味着 $\mathbf{r}^\Pi$ 独立于实际执行中任意 $m - 1$ 个参与方的联合视图，这将在后续证明中使用。

– 归纳假设： $t = m - k$

假设对于任何腐化 $t = m - k$ 个参与方（ $1 \leq k < m - 1$ ）的敌手 $\mathcal{A}$ ， Π 均能安全计算 f 。即存在模拟器 Sim 满足：

$$\begin{aligned} & \{\text{Sim}(\{P_1, \dots, P_m\} \setminus \mathbf{P}_H, \{x_1, \dots, x_m\} \setminus \mathbf{x}_H, \{s_1, \dots, s_m\} \setminus \mathbf{s}_H), \{s_1, \dots, s_m\} \setminus \mathbf{s}_H, \mathbf{s}_H\}_{\mathbf{x}} \\ & \stackrel{c}{\approx} \{\{\text{View}_1^\Pi(\mathbf{x}), \dots, \text{View}_m^\Pi(\mathbf{x})\} \setminus \{\text{View}_H^\Pi(\mathbf{x})\}, \{s_1^\Pi, \dots, s_m^\Pi\} \setminus \mathbf{s}_H^\Pi, \mathbf{s}_H^\Pi\}_{\mathbf{x}} \end{aligned}$$

其中 $\mathbf{P}_H$ 是诚实参与方集合， $\text{View}_H^\Pi(\mathbf{x})$ 表示 $\mathbf{P}_H$ 的联合视图，而 $\mathbf{s}_H$ 和 $\mathbf{s}_H^\Pi$ 分别是理想世界和真实世界中的对应输出。

显然，在理想世界中， $\mathbf{s}_H$ 独立于联合分布 $\{\text{Sim}(\dots), \{s_1, \dots, s_m\} \setminus \mathbf{s}_H\}$ 。因此，可以得出结论：在真实世界中，对于任意规模为 k 的参与方子集 $\mathbf{P}_H \subset \{P_1, \dots, P_m\}$ ， $\mathbf{s}_H^\Pi$ 独立于联合分布 $\{\{\text{View}_i^\Pi(\mathbf{x})\} \setminus \text{View}_H^\Pi(\mathbf{x}), \{s_i^\Pi\} \setminus \mathbf{s}_H^\Pi\}$ 。

– 归纳步骤： $t = m - k - 1$

我们继续证明敌手 $\mathcal{A}$ 腐化 $t = m - k - 1$ 个参与方的情况：

令 $\mathbf{P}_{\mathcal{H}'}$ 代表 $\{P_1, \dots, P_m\}$ 中规模为 $k+1$ 的子集。我们将其分解为 $\mathbf{P}_{\mathcal{H}'} = \{\mathbf{P}_{\mathcal{H}}, P_h\}$, 其中 $\mathbf{P}_{\mathcal{H}}$ 包含恰好 k 个参与方, P_h 为剩余的一个参与方。根据隐私性, 我们有:

$$\{\text{Sim}(\mathbf{P}_{\mathcal{A}}, \mathbf{x}_{\mathcal{A}}, \mathbf{s}_{\mathcal{A}}), \mathbf{s}_{\mathcal{A}}\}_{\mathbf{x}} \stackrel{c}{\approx} \{\text{View}_{\mathcal{A}}^{\Pi}(\mathbf{x}), \mathbf{s}_{\mathcal{A}}^{\Pi}\}_{\mathbf{x}}$$

根据正确性, 我们同样有 $\{\mathbf{s}_{\mathcal{H}}\}_{\mathbf{x}} \stackrel{c}{\approx} \{\mathbf{s}_{\mathcal{H}}^{\Pi}\}_{\mathbf{x}}$ 。

由归纳假设可知, $\mathbf{s}_{\mathcal{H}}$ 独立于联合分布 $\{\text{View}_{\mathcal{A}}^{\Pi}(\mathbf{x}), \mathbf{s}_{\mathcal{A}}^{\Pi}\}$ 。同时, $\mathbf{s}_{\mathcal{H}}$ 显然独立于理想世界中的联合分布 $\{\text{Sim}(\mathbf{P}_{\mathcal{A}}, \mathbf{x}_{\mathcal{A}}, \mathbf{s}_{\mathcal{A}}), \mathbf{s}_{\mathcal{A}}\}_{\mathbf{x}}$ 。结合上述条件:

$$\{\text{Sim}(\mathbf{P}_{\mathcal{A}}, \mathbf{x}_{\mathcal{A}}, \mathbf{s}_{\mathcal{A}}), \mathbf{s}_{\mathcal{A}}, \mathbf{s}_{\mathcal{H}}\}_{\mathbf{x}} \stackrel{c}{\approx} \{\text{View}_{\mathcal{A}}^{\Pi}(\mathbf{x}), \mathbf{s}_{\mathcal{A}}^{\Pi}, \mathbf{s}_{\mathcal{H}}^{\Pi}\}_{\mathbf{x}}$$

回顾基础情况中的推导, $\mathbf{r}^{\Pi}$ 独立于实际执行中任意 $m-1$ 个参与方的联合视图, 因此 $\mathbf{r}^{\Pi}$ 独立于 $\{\text{View}_{\mathcal{A}}^{\Pi}(\mathbf{x}), \text{View}_{\mathcal{H}}^{\Pi}(\mathbf{x})\}$ 。由于 $\mathbf{s}_{\mathcal{A}}^{\Pi}$ 和 $\mathbf{s}_{\mathcal{H}}^{\Pi}$ 可分别由 $\text{View}_{\mathcal{A}}^{\Pi}(\mathbf{x})$ 和 $\text{View}_{\mathcal{H}}^{\Pi}(\mathbf{x})$ 确定, 故 $\mathbf{r}^{\Pi}$ 独立于 $\{\text{View}_{\mathcal{A}}^{\Pi}(\mathbf{x}), \mathbf{s}_{\mathcal{A}}^{\Pi}, \mathbf{s}_{\mathcal{H}}^{\Pi}\}$ 。此外, 理想世界中 $\mathbf{r}$ 独立于 $\{\text{Sim}(\mathbf{P}_{\mathcal{A}}, \mathbf{x}_{\mathcal{A}}, \mathbf{s}_{\mathcal{A}}), \mathbf{s}_{\mathcal{A}}, \mathbf{s}_{\mathcal{H}}\}$, 从而可将不可区分性扩展为:

$$\{\text{Sim}(\mathbf{P}_{\mathcal{A}}, \mathbf{x}_{\mathcal{A}}, \mathbf{s}_{\mathcal{A}}), \mathbf{s}_{\mathcal{A}}, \mathbf{s}_{\mathcal{H}}, \mathbf{r}\}_{\mathbf{x}} \stackrel{c}{\approx} \{\text{View}_{\mathcal{A}}^{\Pi}(\mathbf{x}), \mathbf{s}_{\mathcal{A}}^{\Pi}, \mathbf{s}_{\mathcal{H}}^{\Pi}, \mathbf{r}^{\Pi}\}_{\mathbf{x}}$$

根据功能定义, 参与方 P_h 的输出满足 $s_h = -(\sum_{s_i \in \mathbf{s}_{\mathcal{A}}} s_i + \sum_{s_i \in \mathbf{s}_{\mathcal{H}}} s_i) + f(\mathbf{x})$ 。由正确性可知, $s_h^{\Pi} = -(\sum_{s_i^{\Pi} \in \mathbf{s}_{\mathcal{A}}^{\Pi}} s_i^{\Pi} + \sum_{s_i^{\Pi} \in \mathbf{s}_{\mathcal{H}}^{\Pi}} s_i^{\Pi}) + f(\mathbf{x})$ 。最后, 通过包含 s_h 和 s_h^{Π} 得到:

$$\{\text{Sim}(\mathbf{P}_{\mathcal{A}}, \mathbf{x}_{\mathcal{A}}, \mathbf{s}_{\mathcal{A}}), \mathbf{s}_{\mathcal{A}}, \mathbf{s}_{\mathcal{H}}, s_h, \mathbf{r}\}_{\mathbf{x}} \stackrel{c}{\approx} \{\text{View}_{\mathcal{A}}^{\Pi}(\mathbf{x}), \mathbf{s}_{\mathcal{A}}^{\Pi}, \mathbf{s}_{\mathcal{H}}^{\Pi}, s_h^{\Pi}, \mathbf{r}^{\Pi}\}_{\mathbf{x}}$$

该不可区分性成立是因为 s_h (对应地, s_h^{Π}) 由 $\mathbf{s}_{\mathcal{A}}$ 、 $\mathbf{s}_{\mathcal{H}}$ (对应地, $\mathbf{s}_{\mathcal{A}}^{\Pi}$ 、 $\mathbf{s}_{\mathcal{H}}^{\Pi}$)、随机性 $\mathbf{r}$ (对应地, $\mathbf{r}^{\Pi}$) 以及输入 $\mathbf{x}$ 共同唯一确定。由此导出:

$$\{\text{Sim}(\mathbf{P}_{\mathcal{A}}, \mathbf{x}_{\mathcal{A}}, \mathbf{s}_{\mathcal{A}}), \mathbf{s}_{\mathcal{A}}, s_h\}_{\mathbf{x}} \stackrel{c}{\approx} \{\text{View}_{\mathcal{A}}^{\Pi}(\mathbf{x}), \mathbf{s}_{\mathcal{A}}^{\Pi}, s_h^{\Pi}\}_{\mathbf{x}}$$

即在敌手 $\mathcal{A}$ 腐化 $t = m - k - 1$ 个参与方的情况下, Π 能够安全地计算 f 。归纳步骤完成。

□

谓词零分享功能 $\mathcal{F}_{\text{PZS}}^Q$ 是 $\mathcal{F}^f$ 的一个特例, 其中

$$f(\mathbf{x}) = \begin{cases} 0 & \text{若 } Q(\mathbf{x}) = 1 \\ s & \text{若 } Q(\mathbf{x}) = 0 \end{cases}$$

且 s 是一个随机值。可知, s 是 f 所依赖的随机性, 将其在协议实际执行中的值记为 s^{Π} 。在谓词零分享这一特例下, 独立性要求具体为: 若 $Q(\mathbf{x}) = 0$, 则在协议 Π 的执行过程中, 秘密值 $s^{\Pi} = s_1^{\Pi} + \dots + s_m^{\Pi}$ 的分布独立于 $\text{View}_{\mathcal{A}}^{\Pi}(\mathbf{x})$ 。综上, 谓词零分享协议的正确性和独立性要求共同保证了: 若 $Q(\mathbf{x}) = 0$, 则各参与方执行协议 Π 获得的输出构成的秘密分享中的秘密值 s^{Π} 是均匀随机分布的, 且独立于任意 $t \leq m - 1$ 个参与方的联合视图。

注 5.9. 后文中将使用上述给出的等价的安全性定义——通过分别证明满足正确性、隐私性和独立性，来证明本工作中所有谓词零分享协议的安全性。在该安全性定义下，我们将仅满足正确性和隐私性要求的谓词零分享协议称为**松弛谓词零分享**，并将与谓词公式 Q 相关联的松弛谓词零分享功能记作 $\mathcal{F}_{\text{rPZS}}^Q$ 。

5.3.3 从松弛到标准的转化技术

本节给出一种将松弛谓词零分享转化为关联相同谓词公式的满足标准半诚实安全的谓词零分享的高效方法：假设所有参与方已调用松弛谓词零分享协议 Π_{rPZS}^Q 获得了一组秘密分享 $[r] = (r_1, \dots, r_m)$ ，其目标是生成一组满足标准谓词零分享安全性定义的秘密分享 $[s] = (s_1, \dots, s_m)$ 。为此，参与方只需在离线阶段准备一组随机值的秘密分享 $[b]$ （由每个参与方 P_i 采样一个均匀随机的分享份额 b_i ），并在在线阶段协作执行一次安全乘法计算 $[s] = [r] \cdot [b]$ 。

定理 5.10. 在 $\mathcal{F}_{\text{rPZS}}^Q$ 和安全乘法的混合模型下，上述协议安全地实现了功能 $\mathcal{F}_{\text{PZS}}^Q$ 。

正确性与独立性分析. 设有限域大小 $|\mathbb{F}| \geq 2^\sigma$ ，由松弛谓词零分享协议 Π_{rPZS}^Q 的正确性可知：

- 若 $Q(\mathbf{x}) = 1$ ， $r = 0$ ，则 $s = 0$ 。
- 若 $Q(\mathbf{x}) = 0$ ， r 是均匀随机分布的。用 E 表示事件秘密值 s 是均匀随机且独立于任意 $t \leq m-1$ 个参与方的联合视图。用 E_0 和 E_1 分别表示事件 $r = 0$ 和 $r \neq 0$ 。由于 b 是均匀随机且独立分布的，我们有事件 E 发生的概率为 $Pr[E] = Pr[E|E_0] \cdot Pr[E_0] + Pr[E|E_1] \cdot Pr[E_1] = 0 \cdot Pr[E_0] + 1 \cdot Pr[E_1] = Pr[E_1] = 1 - Pr[E_0] = 1 - \frac{1}{|\mathbb{F}|} \geq 1 - 2^{-\sigma}$ 。

隐私性分析. 该协议的隐私性直接继承自底层原语松弛谓词零分享和安全乘法计算协议的隐私性分析。具体地，在理想世界中，给定协议输出的前提下，模拟器依次调用松弛谓词零分享与安全乘法计算协议的子模拟器。只要松弛谓词零分享与安全乘法计算协议安全地实现了各自的理想功能，则模拟器在理想世界中模拟的敌手视图（即两个子模拟器在理想世界中模拟的敌手视图的联合分布）与敌手在真实世界中的视图（敌手分别在两个子协议的调用过程中视图的联合分布）是计算不可区分的。

鉴于松弛谓词零分享输出的秘密分享中的均匀随机秘密值依赖于某些 $t \leq m-1$ 个参与方的联合视图，在调用谓词零分享后对这些秘密值进行重构时，若这 t 个参与方发生共谋，则可能会泄露信息。该转化技术通过消除这种依赖性来防止此类泄露。这一步骤对于确保我们的 MPSO 框架在抵御任意共谋攻击时的安全性至关重要。该转化技术可以利用 Beaver 三元组 [78] 对在线阶段进行如下优化：

首先回顾 Beaver 三元组技术。一个 Beaver 三元组由三个秘密分享 $([a], [b], [c])$ 组成，其中 $[a]$ 和 $[b]$ 是随机的秘密分享，且满足 $c = a \cdot b$ 。通常情况下，Beaver 三元组

用于在在线阶段将一次乘法运算规约为两次重构过程；而在此处，由于乘数 b 本身是随机的，Beaver 三元组可用于在在线阶段将一次乘法运算规约为仅一次重构过程。具体而言：

$$s = r \cdot b = (r + a - a) \cdot b = (r + a) \cdot b - a \cdot b$$

因此我们可以计算：

$$[s] = (r + a) \cdot [b] - [a \cdot b] = (r + a) \cdot [b] - [c]$$

上述等式表明，一旦 $u = r + a$ 公开，我们就可以在本地计算 $[s]$ 。因此，生成 $[s]$ 的任务便简化为重构 $[u] = [r] + [a]$ 。令每个参与方 P_i 在本地计算其分享份额 $u_i = r_i + a_i$ 并将 u_i 发送给 Leader 节点 P_1 ，随后 P_1 计算 $u = u_1 + \dots + u_m$ 并将其向所有参与方公开。由此可见，该转化技术仅消耗一个在离线阶段生成的 Beaver 三元组；在在线阶段，Leader 节点的通信开销为 $2(m-1)\log_2|\mathbb{F}|$ ，每个 Client 节点的通信开销为 $2\log_2|\mathbb{F}|$ 。

上述优化后的转化技术采用星型通信拓扑，这使其能够集成到后续将介绍的 MPSO 协议中而不改变其原有拓扑结构——从而在保证其标准半诚实安全性的同时，实现优越的复杂度性能。

5.3.4 从简单到复合的组合技术

根据所关联的谓词公式 Q 的类型，可将谓词零分享协议分为两类：若 Q 是一个简单谓词，则称之为简单谓词零分享 (Simple Predicative Zero-Sharing)；若 Q 是一个复合谓词，即 Q 由 $q > 1$ 个文字（原子命题或其否定）和逻辑联结词 $\wedge$ 与 $\vee$ 构成，其中每个文字 Q_i 是一个简单谓词或其否定 ($1 \leq i \leq q$)，则称对应的谓词零分享为复合谓词零分享 (Compound Predicative Zero-Sharing)。本节将介绍将简单谓词零分享协议组合成复合谓词零分享协议的高效方法——对于任何复合谓词 Q ，只要为其中每个文字 Q_i 构造对应的松弛版本的简单谓词零分享协议，就可以构造与 Q 相关联的满足标准半诚实安全的复合谓词零分享协议。

总的来说，构造与 Q 相关联的复合谓词零分享协议可分为三个阶段：首先，参与方为每个文字 Q_i 执行对应的松弛简单谓词零分享协议。对于仅涉及参与方子集的文字，未参与的参与方将其缺失的分享份额设置为 0；其次，参与方参照由所有文字通过逻辑与 AND、逻辑或 OR 操作符组合成公式 Q 的计算过程，逐步对持有的秘密分享进行协同处理，将其组合成一个满足与 Q 相关联的松弛版本的复合谓词零分享定义的输出秘密分享。其中，在每一步结束时，参与方都会获得一个与当前计算的中间公式相对应的秘密分享；最后，参与方将松弛复合谓词零分享转化为足标准半诚实安全的复合谓词零分享。完整的构造过程详见图 5.2。

定理 5.11. 在 $(\mathcal{F}_{\text{rPZS}}^{Q_1}, \dots, \mathcal{F}_{\text{rPZS}}^{Q_q})$ -混合模型下，协议 Π_{PZS}^Q 在任意半诚实敌手腐化不超过 $t < m$ 个参与方的情况下，安全地实现了功能 $\mathcal{F}_{\text{PZS}}^Q$ 。

参数： m 个参与方 $P_1, \dots, P_m$ ，其输入为 $\mathbf{x} = (x_1, \dots, x_m)$ 。有限域 $\mathbb{F}$ 。一个由 q 个文字 $Q_1, \dots, Q_q$ ($q \geq 1$) 和逻辑联结词 AND 和 OR 构成的简单/复合谓词 Q 。离线阶段生成的 Beaver 三元组 $([a], [b], [c])$ 。

协议：

1. **简单谓词分享阶段。** 参与方为每个文字 Q_i 调用 $\mathcal{F}_{\text{rPZS}}^{Q_i}$ ($1 \leq i \leq q$)。若 Q_i 未涉及所有参与方，则未参与的参与方将对应 Q_i 的分享份额设置为 0。由此，每个 Q_i 在参与方之间都有一个对应的秘密分享。
2. **模拟公式计算阶段。** 若 $q > 1$ ，参与方按照操作符的优先级，逐步模拟 Q 的计算。在其中的每一步中，参与方模拟两个谓词 Q'_i 和 Q'_j 通过逻辑操作符连接成一个谓词公式 Q' 的计算——参与方已经从前一步获得了与 Q'_i 和 Q'_j 对应的秘密分享，目标是生成一组满足与 Q' 相关联的松弛谓词零分享输出的秘密分享。具体的操作取决于所计算的操作符类型：

- **AND：** 假设参与方持有分别与 Q'_i 和 Q'_j 对应的两个秘密分享 $[r_i]$ 和 $[r_j]$ 。为了计算 Q' 的松弛谓词零分享（其中 $Q'(\mathbf{x}) = Q'_i(\mathbf{x}) \wedge Q'_j(\mathbf{x})$ ），参与方在本地将两个秘密分享对应的份额相加，得到秘密分享 $[r_i + r_j]$ 。
- **OR：** 假设参与方持有分别与 Q'_i 和 Q'_j 对应的两个秘密分享 $[r_i]$ 和 $[r_j]$ 。为了计算 Q' 的松弛谓词零分享（其中 $Q'(\mathbf{x}) = Q'_i(\mathbf{x}) \vee Q'_j(\mathbf{x})$ ），参与方执行一次安全乘法计算，得到 $[r_i \cdot r_j] = [r_i] \cdot [r_j]$ 。

在每一步结束后，参与方将 Q' 视为一个新的文字，并重复上述过程，直到 Q 中仅剩一个文字。与最终文字对应的秘密分享 $[r]$ 即为与 Q 相关联的松弛谓词零分享协议的输出秘密分享。

3. **从松弛到标准的转化。** 所有参与方利用 Beaver 三元组执行一次安全乘法计算 $[s] = [r] \cdot [b]$ 来计算 $[s]$ （参见第 5.3.3 节）。

图 5.2 谓词零分享协议 Π_{PZS}^Q

正确性与独立性分析. 在模拟公式计算阶段的每一步中：

- 若 $Q'(\mathbf{x}) = Q'_i(\mathbf{x}) \wedge Q'_j(\mathbf{x})$, 参与方计算 $[r_i + r_j] = [r_i] + [r_j]$ 。若 $Q'(\mathbf{x}) = 1$, 即 $Q'_i(\mathbf{x}) = 1$ 且 $Q'_j(\mathbf{x}) = 1$, 根据 $\mathcal{F}_{\text{rPZS}}^{Q'_i}$ 和 $\mathcal{F}_{\text{rPZS}}^{Q'_j}$ 的功能可知 $r_i = 0$ 且 $r_j = 0$, 因此 $r_i + r_j = 0$; 否则, $Q'_i(\mathbf{x}) = 0$ 或 $Q'_j(\mathbf{x}) = 0$, 根据 $\mathcal{F}_{\text{rPZS}}^{Q'_i}$ 和 $\mathcal{F}_{\text{rPZS}}^{Q'_j}$ 的功能 r_i 和 r_j 中至少有一个是随机值, 故 $r_i + r_j$ 也是随机值。
- 若 $Q'(\mathbf{x}) = Q'_i(\mathbf{x}) \vee Q'_j(\mathbf{x})$, 参与方计算 $[r_i \cdot r_j] = [r_i] \cdot [r_j]$ 。若 $Q'(\mathbf{x}) = 1$, 即 $Q'_i(\mathbf{x}) = 1$ 或 $Q'_j(\mathbf{x}) = 1$, 根据 $\mathcal{F}_{\text{rPZS}}^{Q'_i}$ 和 $\mathcal{F}_{\text{rPZS}}^{Q'_j}$ 的功能 $r_i = 0$ 或 $r_j = 0$, 因此 $r_i \cdot r_j = 0$; 否则, $Q'_i(\mathbf{x}) = 0$ 且 $Q'_j(\mathbf{x}) = 0$, 则 r_i 和 r_j 均为随机值。用 $E^{i,j}$ 表示事件 $r_i \cdot r_j$ 是随机值。用 E_0^i 表示事件 $r_i = 0$, E_1^i 表示事件 $r_i \neq 0$ 。我们有事件 $E^{i,j}$ 发生的概率为 $Pr[E^{i,j}] = Pr[E^{i,j}|E_0^i] \cdot Pr[E_0^i] + Pr[E^{i,j}|E_1^i] \cdot Pr[E_1^i] = 0 \cdot Pr[E_0^i] + 1 \cdot Pr[E_1^i] = Pr[E_1^i] = 1 - Pr[E_0^i]$ 。为了将正确性误差限制在 $2^{-\sigma}$ 以内, 要求任何 E_0^i 发生的概率是可忽略的。通过联合界 (Union Bound), $Pr[\bigvee_i E_0^i] \leq \sum_i Pr[E_0^i] = \frac{|\text{OR}|}{|\mathbb{F}|}$ 。因此, 需设置有限域大小 $|\mathbb{F}| \geq |\text{OR}| \cdot 2^\sigma$, 其中 $|\text{OR}|$ 是 Q 中 OR 操作符的数量。

上面模拟公式计算阶段中每一步模拟 AND 和 OR 操作符计算的正确性, 保证了该阶段实现与 Q 相关联的松弛谓词零分享协议的正确性。根据第 5.3.3 节中的正确性和独立性分析, 最终协议满足与 Q 相关联的标准谓词零分享的正确性和独立性要求。

隐私性分析. 谓词零分享的隐私性易于验证: 由于所有交互均发生在对构建模块 (即所有松弛简单谓词零分享协议、安全乘法计算及转化技术) 的调用内部, 因此给定理想功能的输出, 模拟器只需以反向链式的方式调用这些构建模块的子模拟器。最终, 只要所有松弛简单谓词零分享协议、安全乘法计算和转化技术的隐私性成立, 那么敌手的视图在理想世界和真实世界中就是不可区分的。

5.4 成员零分享

谓词零分享刻画了一类 MPC 协议的统一抽象。只有指定相关联的谓词公式, 谓词零分享才可被实例化为具体协议。为了在 MPSO 语境下实例化谓词零分享, 本节引入了一类更具体的 MPC 协议——成员零分享 (Membership Zero-Sharing), 作为谓词零分享的一个子集, 每个成员零分享协议都与一个集合谓词公式相关联, 并构成后续 MPSO 框架的技术核心。

本节的目标是为任意一阶集合谓词公式构造完整的成员零分享协议。从宏观来看, 我们的构造遵循图 5.2 中给出的构建谓词零分享的技术路线。要构造具体协议, 我们还需要实例化其中的松弛谓词零分享组件——更具体地, 在本章集合谓词公式的语境下, 松弛成员零分享。鉴于任何集合谓词公式仅由两类文字组成——集合成员谓词 $x \in Y$ 及其否定 $x \notin Y$, 因此问题可归约为在两方场景下分别构造与这两个简单谓词相关联的松弛成员零分享协议。

5.4.1 功能定义

成员零分享协议允许 m ($m \geq 2$) 个参与方参与，其中一个参与方（记为 P_{pivot} ）持有一个元素 x 作为输入，而其他每个参与方 P_j ($j \in \{1, \dots, m\} \setminus \{\text{pivot}\}$) 持有一个集合 X_j 作为输入。如果关联的集合谓词公式 $Q(x, X_1, \dots, X_{\text{pivot}-1}, X_{\text{pivot}+1}, \dots, X_m) = 1$ ，参与方将获得 0 的秘密分享；否则，他们将获得一个随机值的秘密分享。

图 5.3 定义了成员零分享的批量版本，其中 P_{pivot} 持有一个向量 $\mathbf{x} = (x_1, \dots, x_n)$ ，而每个 P_j 持有 n 个集合序列作为输入。参与方获得 n 组秘密分享，其中第 i 组秘密分享表示对第 i 个输入计算相同公式 Q 得到的真值。

参数： m 个参与方 $P_1, \dots, P_m$ ，其中 P_{pivot} 是唯一持有 n 个元素（而非 n 个集合）的参与方。一个集合成员谓词公式 Q 。批量大小 n 。有限域 $\mathbb{F}$ 。

功能： 收到来自 P_{pivot} 的输入 $\mathbf{x} = (x_1, \dots, x_n)$ 以及来自每个 P_j ($j \in \{1, \dots, m\} \setminus \{\text{pivot}\}$) 的输入 $\mathbf{X}_j = (X_{j,1}, \dots, X_{j,n})$ 后，为 $1 \leq i \leq m$ 随机采样 $\mathbf{s}_i = (s_{i,1}, \dots, s_{i,n}) \leftarrow \mathbb{F}^n$ ，且满足：对于 $1 \leq d \leq n$ ，若 $Q(x_d, X_{1,d}, X_{\text{pivot}-1,d}, X_{\text{pivot}+1,d}, \dots, X_{m,d}) = 1$ ，则 $\sum_{1 \leq i \leq m} s_{i,d} = 0$ 。将 $\mathbf{s}_i$ 发送给 P_i 。

图 5.3 批量成员零分享功能 $\mathcal{F}_{\text{bMZS}}^Q$

5.4.2 针对集合成员谓词的松弛成员零分享

针对集合成员谓词 $x \in Y$ 的一类松弛成员零分享可以定义为如下两方功能：包含两个参与方，即持有集合 Y 的发送方 $\mathcal{S}$ 和持有元素 x 的接收方 $\mathcal{R}$ 。该功能采样 $s, r \leftarrow \mathbb{F}$ ，若 $x \in Y$ ，则设 $u = -r$ ；否则 $u = s - r$ 。同时，它生成一个包含关于 s 的部分信息的中间消息 leak 。最后，该功能向 $\mathcal{S}$ 输出 r, leak ，向 $\mathcal{R}$ 输出 u 。其中的 leak 允许在 $x \notin Y$ 时向 $\mathcal{S}$ 泄露关于秘密值 s 的部分信息，从而体现了松弛谓词零分享中的安全性松弛。

从 $\mathcal{R}$ 的视角看，若 $x \in Y$ ，则 $u = -r$ ；否则由于 s 是由该功能均匀采样的，且 $\mathcal{R}$ 未获得关于 s 的额外信息，故 $u = s - r$ 是随机的。因此， u 可以看作是一个“可编程”随机函数 f 在 x 处的取值，其中 f 在集合 Y 的元素上的取值均被编程为相同的均匀随机值 $-r$ ，而其他位置的取值为随机值。为了高效实例化该功能，令 f 为一个可编程 PRF (OPPRF)。向 $\mathcal{S}$ 泄露的 leak 即为 PRF 密钥，而 $\mathcal{R}$ 通过与 $\mathcal{S}$ 调用 OPPRF 来不经意地计算 $u = f(\text{leak}, x)$ 。具体构造如下： $\mathcal{S}$ 采样一个均匀随机值 r ，并将 Y 设置为关键字集合，将 n 个重复的值 $-r$ 设置为对应的值集合。随后 $\mathcal{S}$ 和 $\mathcal{R}$ 调用 OPPRF，其中 $\mathcal{R}$ 输入 x 并获得 u 。最后， $\mathcal{S}$ 和 $\mathcal{R}$ 分别输出 r 和 u 。根据 OPPRF 的功能，若 $x \in Y$ ，则 $u = -r$ ；否则 u 是伪随机的。向 $\mathcal{S}$ 输出的 leak 即为来自 OPPRF 的 PRF 密钥。该协议可以利用 batch OPPRF 自然地扩展到批量版本。

5.4.3 针对集合非成员谓词的松弛成员零分享

针对集合非成员谓词 $x \notin Y$ 的一类松弛成员零分享协议可以定义为如下两方功能：发送方 $\mathcal{S}$ 输入集合 Y ，接收方 $\mathcal{R}$ 输入元素 x 。该功能采样 $s, r \leftarrow \mathbb{F}$ ，若 $x \notin Y$ ，则设 $u = -r$ ；否则 $u = s - r$ 。它同样生成一个包含关于 s 的部分信息的中间消息 leak 。最后，该功能向 $\mathcal{S}$ 输出 r, leak ，向 $\mathcal{R}$ 输出 u 。

该功能在直觉上与秘密分享隐私成员测试 (ssPMT) 高度相似：——两者在 $x \notin Y$ 时均产生 0 的秘密分享。核心区别在于，本协议输出的是有限域 $\mathbb{F}$ 上的零分享，其中输出 0 的秘密分享的对立事件是输出 $\mathbb{F}$ 中一个随机值的秘密分享；而 ssPMT 输出的是 $\mathbb{F}_2$ 上的比特秘密分享，其中输出 0 的秘密分享的对立事件是输出 1 的秘密分享。鉴于第三章中给出了高效的批量秘密分享隐私成员测试 (batch ssPMT) 构造，我们的目标是高效地将比特秘密分享转化为零分享（批量版本首先由参与方调用 batch ssPMT，随后执行 n 次转化）。回顾第 5.3.1 节中，ROT 被视为一种与谓词 $e = 0$ 相关联的松弛简单谓词零分享。ROT 的一种变体涉及由 $\mathcal{S}$ 和 $\mathcal{R}$ 分别持有的两个选择比特 e_0, e_1 [1, 24, 79]，它是一种关联谓词为 $e_0 \oplus e_1 = 0$ 的松弛简单谓词零分享。这种“双选择比特”的 ROT 实际上可以看作是从比特秘密分享到零分享的一种转化：执行协议后， $\mathcal{S}$ 获得 $r_0, r_1 \in \mathbb{F}$ ，而 $\mathcal{R}$ 获得 $r_{e_0 \oplus e_1} \in \mathbb{F}$ ³。令 $\mathcal{S}$ 设置 $r = -r_0$ ， $\mathcal{R}$ 设置 $u = r_{e_0 \oplus e_1}$ ，则若 $e_0 \oplus e_1 = 0$ ，有 $r + u = 0$ ；否则 $r + u = r_1 - r_0 = s$ 是随机的。向 $\mathcal{S}$ 泄露的 s 即为 r_1 （因为 $\mathcal{S}$ 获知了 r_0, r_1 ，故可计算 s ）。

5.4.4 针对任意集合谓词公式的成员零分享

利用上述针对两方松弛成员零分享协议的实例化，我们在图 5.4 中给出了针对任意集合谓词公式 Q 的完整批量成员零分享协议。

定理 5.12. 在 $(\mathcal{F}_{\text{bOPPRF}}, \mathcal{F}_{\text{bssPMT}}, \mathcal{F}_{\text{ROT}})$ -混合模型下，协议 Π_{bMZS}^Q 在任意半诚实敌手腐化不超过 $t < m$ 个参与方的情况下，安全地实现了 $\mathcal{F}_{\text{bMZS}}^Q$ 。

正确性与独立性分析. 成员零分享的正确性和独立性继承自谓词零分享，仅需针对批量场景调整正确性参数： $|\mathbb{F}| \geq |\text{OR}| \cdot n \cdot 2^\sigma$ ，其中 $|\text{OR}|$ 是 Q 中 OR 操作符的数量。

隐私性分析. 成员零分享的隐私性易于验证：所有交互均发生在两个松弛批量成员零分享协议的调用内部，这些协议可进一步分解为三个基础模块——batch OPPrF、batch ssPMT 和 ROT。因此，给定理想功能的输出，模拟器只需以反向推导的方式调用这些基础模块的子模拟器。只要 batch OPPrF、batch ssPMT 和 ROT 协议是安全的，敌手的视图在理想世界和真实世界中是不可区分的，从而满足隐私性定义。

³这种双选择比特 ROT 与标准的 2 选 1 ROT 等价，其中 e_0 由 $\mathcal{S}$ 采样，用于指示是否交换 r_0 和 r_1 的顺序，如图 5.4 所示。

参数： m 个参与方 $P_1, \dots, P_m$, 其中 P_{pivot} 持有 n 个元素而非 n 个集合。集合谓词公式 Q 由 $q \geq 1$ 个文字 $Q_1, \dots, Q_q$ 组成, 每个 Q_i ($1 \leq i \leq q$) 的形式为 $x \in X_j$ 或 $x \notin X_j$ ($j \neq \text{pivot}$)。 n 个 Beaver 三元组 $([a], [b], [c])$ 。批量大小 n 。域 $\mathbb{F}$ 。

协议：

1. **简单谓词分享阶段。** 在此阶段, 对于每个 Q_i , P_{pivot} 和 P_j 根据 Q_i 的形式调用批量大小为 n 的针对 $x \in X_j$ 或 $x \notin X_j$ 的松弛批量成员零分享协议, 其余参与方将其秘密分享设为 0。最终, 参与方持有一个与 Q_i 相关联的包含 n 组秘密分享的向量。具体地:

- $x \in X_j$: 对于第 k 个实例 ($1 \leq k \leq n$), P_j 采样 $r_{i,k}$ 并设置 $K_{i,k} = X_{j,k}$ 和 $V_{i,k} = \{-r_{i,k}, \dots, -r_{i,k}\}$ 。然后 P_{pivot} 和 P_j 调用 $\mathcal{F}_{\text{bOPPRF}}$, 其中 P_j 作为 $\mathcal{S}$, 输入为 $(K_{i,1}, \dots, K_{i,n})$ 和 $(V_{i,1}, \dots, V_{i,n})$; P_{pivot} 作为 $\mathcal{R}$ 输入为 $\mathbf{x}$, 并接收 $\mathbf{u}_i$ 。 P_{pivot} 设置其份额 $\mathbf{r}_{i,\text{pivot}} = \mathbf{u}_i$ 。 P_j 设置其份额 $\mathbf{r}_{i,j} = (r_{i,1}, \dots, r_{i,n})$ 。对于其余每个 P_d , 设置其份额 $\mathbf{r}_{i,d} = \mathbf{0}$ 。
- $x \notin X_j$: P_{pivot} 和 P_j 调用 $\mathcal{F}_{\text{bssPMT}}$, 在第 k 个实例 ($1 \leq k \leq n$) 中, P_j 输入 $X_{j,k}$ 并接收 $e_{i,k}^0$, P_{pivot} 输入 x_k 并接收 $e_{i,k}^1$ 。然后他们调用 n 个 ROT 实例, 在第 k 个实例中, P_j 作为 $\mathcal{S}$ 接收 $r_{i,k}^0, r_{i,k}^1$, P_{pivot} 作为 $\mathcal{R}$ 输入 $e_{i,k}^1$ 并接收 $r_{i,k}^{e_{i,k}^1}$ 。 P_{pivot} 设置其份额 $\mathbf{r}_{i,\text{pivot}} = (r_{i,1}^{e_{i,1}^1}, \dots, r_{i,n}^{e_{i,n}^1})$ 。 P_j 设置其份额 $\mathbf{r}_{i,j} = (-r_{i,1}^{e_{i,1}^0}, \dots, -r_{i,n}^{e_{i,n}^0})$ 。对于其余每个 P_d , 设置其份额 $\mathbf{r}_{i,d} = \mathbf{0}$ 。

Q_i 的 n 组秘密分享向量记作 $[\mathbf{r}_i] = (\mathbf{r}_{i,1}, \dots, \mathbf{r}_{i,m})$ 。

2. **公式模拟阶段。** 参与方遵循图 5.2 中的相同阶段, 但在每一步中生成包含 n 组秘密分享的向量, 操作取决于操作符类型:

- AND 操作符: 参与方本地将两个向量的对应分量相加。
- OR 操作符: 参与方在两个向量的对应分量之间执行 n 次安全乘法计算, 即 $[\mathbf{r}_i \cdot \mathbf{r}_j] = [\mathbf{r}_i] \cdot [\mathbf{r}_j]$ 。

最终向量 $[\mathbf{r}]$ 包含 n 组来自针对 Q 的松弛批量成员零分享的输出秘密分享。

3. **从松弛到标准的转化。** 所有参与方使用 $([a], [b], [c])$ 计算 $[\mathbf{s}] = [\mathbf{r}] \cdot [\mathbf{b}]$ 。

图 5.4 批量成员零分享协议 Π_{bMZS}^Q

5.5 实现通用功能及其扩展的 MPSO 框架

5.5.1 技术路线概述

本章提出的 MPSO 框架可分为两个阶段：第一阶段基于任何可构造集合 Y 的 CSPF 表示 $\psi(x, X_1, \dots, X_m)$ ，目标是各方获得与 Y 的一个划分相对应的秘密分享。回顾 $\psi = Q_1 \vee \dots \vee Q_s$ ，对于每个子公式 Q_i ($1 \leq i \leq s$)，设 $X_{i_1}, \dots, X_{i_q}$ ($1 \leq q \leq m$) 为 Q_i 中所有原子命题涉及的集合，并用 X_j ($j \in \{i_1, \dots, i_q\}$) 表示 Q_i 相对于其满足局部性的集合。即：

$$Q_i(x, X_{i_1}, \dots, X_{i_q}) = (x \in X_j) \wedge Q'_i(x, X_{i_1}, \dots, X_{j-1}, X_{j+1}, \dots, X_{i_q})$$

其中 Q'_i 是 Q_i 的 X_j -局部剩余公式。设 Y_i 为由 Q_i 表示的集合，则 Q'_i 本质上是针对 X_j 中元素关于 Y_i 的成员测试集合谓词公式。我们先给出第一阶段的初步构造：对于每个 Q_i ， $P_{i_1}, \dots, P_{i_q}$ 调用 n 个与 Q'_i 相关联的成员零分享实例。在每个实例中， P_j 作为 P_{pivot} 输入 X_j 中的一个元素。对于每个 $x \in X_j$ ，如果 $Q'_i = 1$ ，参与方收到 0 的秘密分享，否则收到随机值的秘密分享。 P_j 将 x 加到其对应的分享份额上，使得对于每个 $x \in X_j$ ，如果 $Q_i = 1$ （即 $Q'_i = 1$ ），各方收到 x 的秘密分享，否则还是收到随机值的秘密分享。

在上述初步构造中，针对 X_j 的成员测试需要在大小为 n 的相同输入集合上，重复执行 n 次与 Q'_i 相关联的成员零分享。因此，每个实例的均摊开销相对于 n 的复杂度是 $O(n)$ 。为了实现 $O(1)$ 的均摊开销，利用哈希分桶 (Hashing-to-bins) 技术来优化该过程，具体而言：令 P_j 通过布谷鸟哈希预处理 X_j ，其他参与方通过简单哈希预处理各自的集合。该预处理过程使得 P_j 的每个元素 $x \in X_j$ 仅需与其他参与方拥有的具有相同索引的桶进行比较，同时保持成员测试的正确性。因此，在每个成员零分享实例中，当 P_j 作为 P_{pivot} 输入其布谷鸟哈希桶的单个元素时，其他参与方只需输入具有相同索引的简单哈希桶中所有元素组成的集合，该集合是输入集合的子集而非整个集合。在 n 次成员零分享调用中，每个输入集合的参与方输入到 n 个成员零分享实例中的集合大小总和等于其简单哈希表的大小，相对于 n 的复杂度是 $O(n)$ ，因此每个实例的摊销开销被降到 $O(1)$ 。优化后的第一阶段具体构造如下。

对于每个 Q_i ， P_j 使用哈希函数 h_1, h_2, h_3 通过布谷鸟哈希将元素分配到 $B = O(n)$ 个桶中，使得每个桶 C_j^b ($1 \leq b \leq B$) 最多包含一个元素。同时，每个 $P_{j'}$ ($j' \in \{i_1, \dots, i_q\} \setminus \{j\}$) 将其每个元素 $y \in X_{j'}$ 分配到桶 $\mathcal{T}_{j'}^{h_1(y)}, \mathcal{T}_{j'}^{h_2(y)}, \mathcal{T}_{j'}^{h_3(y)}$ 中。如果 P_j 将元素 $x \in X_j$ 映射到 C_j^b 中，则 $b \in \{h_1(x), h_2(x), h_3(x)\}$ 。如果 $P_{j'}$ 也持有 x ，则 x 必定也被映射到 $\mathcal{T}_{j'}^b$ 。这使得除了 X_j 以外的其他参与方能够将其输入集合相对于 X_j 在布谷鸟哈希桶里的排列对齐，从而保证对于 C_j^b 中的每个 x ， $x \in X_j$ 当且仅当 x 在 $\mathcal{T}_{j'}^b$ 中。由此可推导出：

$$Q'_i(x, X_{i_1}, \dots, X_{j-1}, X_{j+1}, \dots, X_{i_q}) = Q'_i(x, \mathcal{T}_{i_1}^b, \dots, \mathcal{T}_{j-1}^b, \mathcal{T}_{j+1}^b, \dots, \mathcal{T}_{i_q}^b).$$

根据上述关系, 让 $P_{i_1}, \dots, P_{i_q}$ 调用与 Q'_i 相关联的批量成员零分享, 其中 P_j 作为 P_{pivot} 输入 $C_j^1, \dots, C_j^B$, 每个 $P_{j'}$ 输入 $\mathcal{T}_{j'}^1, \dots, \mathcal{T}_{j'}^B$ 。协议结束后, 他们收到 B 组秘密分享, 使得如果每个桶 C_j^b 中的元素 x 满足 $Q_i(x, X_{i_1}, \dots, X_{i_q}) = 1$ (即 $x \in Y_i$), 参与方收到 0 的秘密分享, 否则收到随机值的秘密分享。最后, P_j 将 x 拼接上全零字符串 (用于区分元素与随机值), 然后将该值加到自己的第 b 个分享份额上, 从而满足: 如果 $x \in Y_i$, 参与方持有 x 的秘密分享。

鉴于 $\{Y_1, \dots, Y_s\}$ 构成 Y 的一个划分, 如果参与方对所有 $Q_1, \dots, Q_s$ 执行上述过程, 他们将持有 Y 中所有元素的秘密分享 (每个元素的秘密分享出现一次, 其中混杂着不在 Y 中的元素及重复元素对应的随机秘密分享), 这些秘密分享按 $Y_1, \dots, Y_s$ 的主序排列。在每个 Y_i 内部, 对应 Y_i 中元素的秘密分享按 P_j 的布谷鸟哈希位置的次序排列, 该顺序取决于整个集合 X_j 。如果参与方执行上述过程但不执行最后 P_j 将元素加入分享的步骤, 各方将在该元素对应的位置持有 0 的秘密分享。是否执行最后的追加元素步骤由 MPSO 框架的目标功能决定, 该框架可实现的功能可分为三类:

MPSO. 为了实现该功能, 参与方必须向 P_1 重构 Y 中的所有元素, 因此他们必须执行所有的追加元素步骤来对元素进行秘密分享。然而, 直接重构秘密分享会导致两类信息泄露: 1) 重构出的元素的主序揭示了每个元素所属的子集 Y_i 。2) 重构出的元素的次序会泄露 X_j 的信息。因此, 所有参与方在重构阶段前需调用多方秘密分享洗牌协议, 以打乱所有秘密分享的顺序并重新分享所有秘密。在重构阶段后, P_1 识别出所有重构秘密中拼接有全零字符串的值, 截取出其中的元素值并输出。

MPSO-card. 为了实现该功能, 参与方必须在不泄露实际元素的情况下重构秘密以获取基数。为此, 参与方跳过所有的追加元素步骤, 使得对于 Y 中的每个元素, 参与方持有 0 的秘密分享; 而对于不在 Y 中的元素或重复元素, 参与方持有随机秘密分享。如上所述, 这些秘密分享也按照相同的顺序排列, 因此直接重构会导致与实现 MPSO 构造中相同的信息泄露。所以, 参与方同样需要在重构阶段前调用多方秘密分享洗牌协议。随后, 他们向 P_1 重构秘密, 然后由 P_1 统计所有重构秘密中 0 的数量作为 Y 的基数并输出。

Circuit-MPSO. 实现 Circuit-MPSO 功能有两种方案:

- **方案 1:** 参与方跳过所有的追加元素步骤。他们将所有秘密分享连同其对应位置的布谷鸟哈希桶中的元素按顺序输入到一个通用 MPC 协议中。该通用 MPC 协议实现如下电路: 用于从所有秘密中识别出 0, 将所有等于 0 的秘密对应位置的元素组成集合 Y , 最后计算 Y 上的任意函数。
- **方案 2:** 参与方执行所有的追加元素步骤。他们将秘密分享输入到一个通用 MPC 协议中。该通用 MPC 协议实现如下电路: 首先通过是否拼接有全零字符串区分元素与随机值, 然后截取出所有的元素值组成集合 Y , 最后计算 Y 上的任意函数。

5.5.2 框架构造及安全性证明

令 $C_{s,B,l}^1$ 和 $C_{N,l'}^2$ 为两个电路：

- $C_{s,B,l}^1$ 具有 $sB(m \log_2 |\mathbb{F}| + l)$ 条输入线，分为 s 个部分，每部分有 $B(m \log_2 |\mathbb{F}| + l)$ 条输入线，其中每 B 个。在第 i 个部分 ($1 \leq i \leq s$) 中，第 k 组的 B 个在 $\mathbb{F}$ 上的输入与参与方 P_k ($1 \leq k \leq m$) 相关联，将该组中的第 b 个输入 ($1 \leq b \leq B$) 记为 $u_{i,k,b} \in \mathbb{F}$ ；最后 B 个长度为 l 的输入与特定参与方 P_j ($1 \leq j \leq m$) 相关联，记其中的第 b 个输入 ($1 \leq b \leq B$) 为 $z_{i,b} \in \{0,1\}^l$ 。该电路首先实现下面功能：如果 $u_{i,1,b} + \dots + u_{i,m,b} = 0$ ，则产生比特 $w_{i,b} = 1$ ，否则为 0（针对 $1 \leq i \leq s, 1 \leq b \leq B$ ）。然后，该电路计算并输出 $f(Z)$ ，其中 $Z = \{z_{i,b} | w_{i,b} = 1\}_{1 \leq i \leq s, 1 \leq b \leq B}$ ， f 是要在可构造集合 Y 上计算的函数。
- $C_{N,l'}^2$ 具有 m 组，每组 N 个在 $\mathbb{F}$ 上的输入。第 k 组与 P_k ($1 \leq k \leq m$) 相关联，其中第 i 个输入记为 $z_{k,i}$ ($1 \leq i \leq N$)。电路计算并输出 $f(Z)$ ，其中 $Z = \{z_i | z_{1,i} + \dots + z_{m,i} = z_i || 0^{l'}\}_{1 \leq i \leq N}$ ， f 是要在 Y 上计算的函数。

实现计算 Y 的通用 MPSO 功能及其扩展的 MPSO-card 和 Circuit-MPSO 功能的具体协议构造见图 5.5。

定理 5.13. 在 $(\mathcal{F}_{\text{bMZS}}^Q, \mathcal{F}_{\text{shuffle}})$ -混合模型下，图 5.5 中的协议在任意半诚实敌手腐化不超过 $t < m$ 个参与方的情况下，安全地实现了通用 MPSO、MPSO-card 和 Circuit-MPSO 功能。

正确性分析. 协议的正确性源于定理 5.7 及定义 5.6 中 CSPF 的性质，以及批量成员零分享的正确性。为确保所有批量成员零分享协议的正确性，有限域大小需满足 $|\mathbb{F}| \geq |\text{OR}| \cdot B \cdot 2^\sigma$ ，其中 $|\text{OR}|$ 是所有子公式 Q_i ($1 \leq i \leq s$) 中 OR 操作的总数。

设 Y_i 为 Q_i 表示的集合。在 MPSO 和 Circuit-MPSO 协议（方案 2）中，对于每个 Q_i ，参与方持有 $|Y_i|$ 个 Y_i 元素的秘密分享和 $B - |Y_i|$ 个随机值的秘密分享。由于 $\{Y_1, \dots, Y_s\}$ 是 Y 的一个划分，各方总计持有 $|Y|$ 个 Y 元素的秘密分享和 $sB - |Y|$ 个随机值的秘密分享。 P_1 或电路 $C_{s,B,l}^2$ 通过检查最后 l' 位是否全为零来识别并截取所有集合元素。当某个随机值与 $0^{l'}$ 发生碰撞时会产生错误（假阳性），总的假阳性错误概率至多为 $sB \cdot 2^{-l'}$ 。为了使该失败概率可忽略，设置 $l' \geq \sigma + \log s + \log B$ 。在 MPSO-card 和 Circuit-MPSO（方案 1）中，参与方持有 $|Y|$ 个 0 的秘密分享，以及 $B - |Y|$ 个随机值的秘密分享。为了将总的假阳性错误概率限制在 $2^{-\sigma}$ 以内，我们设置 $|\mathbb{F}| \geq sB \cdot 2^\sigma$ 。

安全性分析. 与之前对谓词零分享的隐私性证明类似，除 MPSO 与 MPSO-card 协议中最后发送给 P_1 的重构消息外，所有交互均在构建模块（即所有批量成员零分享协议和多方秘密分享洗牌协议）的调用内部完成。因此给定输出 Y ，模拟器只需以反向链式的方式调用构建模块的子模拟器，即可模拟与诚实方之间的交互。模拟的核心难点在于当 P_1 被腐化时对最后重构消息的模拟。在从通用 MPSO 理想功能获得 Y 之后，模拟器利用 Y 中元素生成 $|Y|$ 个秘密分享，然后均匀采样 $sB - |Y|$ 个随机秘密分享。最后，

参数: m 个参与方 $P_1, \dots, P_m$ 。集合大小 n 。元素长度 l 。全零字符串长度 l' 。有限域 $\mathbb{F}$ 。编码函数 $\text{code} : \mathbb{F} \rightarrow \{0, 1\}^{l+l'}$ 。可构造集合 Y 的 CSPF 表示 $\psi(x, X_1, \dots, X_m)$, $\psi = Q_1 \vee \dots \vee Q_s$ 。布谷鸟哈希参数: 哈希函数 h_1, h_2, h_3 及桶数量 B 。

输入: 每个参与方 P_i 持有私有集合 $X_i = \{x_i^1, \dots, x_i^n\} \subseteq \{0, 1\}^l$ 。

1. **哈希分桶阶段.** 对于 $1 \leq i \leq m$, P_i 执行布谷鸟哈希分桶 $\mathcal{C}_i^1, \dots, \mathcal{C}_i^B \leftarrow \text{Cuckoo}_{h_1, h_2, h_3}^B(X_i)$ 以及简单哈希分桶 $\mathcal{T}_i^1, \dots, \mathcal{T}_i^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_i)$ 。
2. **成员零分享阶段.** 对于 ψ 中的第 i 个子公式 $Q_i(x, X_{i_1}, \dots, X_{i_q})$ ($1 \leq i \leq s$):
 1. 若 $q = 1$ (设 $i_1 = j$), 则 $Q_i(x, X_j) = (x \in X_j)$ 。对于 $1 \leq b \leq B$, 若 $\mathcal{C}_j^b$ 非空, P_j 设置 $s_{i,j,b} = 0$, 否则 P_j 随机选取 $s_{i,j,b}$ 。 $P_{j'} (j' \neq j)$ 设置 $s_{i,j',b} = 0$ 。
 2. 若 $q > 1$, 设 Q_i 相对于 X_j 是局部的, 并且对应的 X_j -局部剩余公式为 Q'_i 。所有输入集合涉及在 Q_i 内的参与方调用 $\mathcal{F}_{\text{bMZS}}^{Q'_i}$, 其中 P_j 作为 P_{pivot} 输入 $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B$, 其余参与方 $P_{j'} (j' \in \{i_1, \dots, i_q\} \setminus \{j\})$ 输入 $\mathcal{T}_{j'}^1, \dots, \mathcal{T}_{j'}^B$ 。在协议执行结束后, 每个涉及的参与方 $P_{i'} (i' \in \{i_1, \dots, i_q\})$ 收到向量 $\mathbf{s}_{i,i'} = (s_{i,i',1}, \dots, s_{i,i',B})$, 其中每个分量作为自己对应的分享份额。其余未涉及的参与方 $P_k (k \in \{1, \dots, m\} \setminus \{i_1, \dots, i_q\})$ 对于每个 $1 \leq b \leq B$ 设置对应的分享份额 $s_{i,k,b} = 0$ 。

参与方的后续动作取决于具体的目标功能:

- MPSO: 3.** 对于 $1 \leq i \leq s, 1 \leq b \leq B$, P_j (步骤 2 中的 j) 检查 $\mathcal{C}_j^b$: 若非空, 计算 $s'_{i,j,b} = \text{code}(s_{i,j,b}) \oplus (x \| 0^{l'})$, 其中 x 为桶中元素; 若为空, 随机采样 $s'_{i,j,b} \leftarrow \{0, 1\}^{l+l'}$ 。
- 4.** 各参与方 $P_k (1 \leq k \leq m)$ 构造向量 $\mathbf{u}_k \in (\{0, 1\}^{l+l'})^{sB}$, 其中对于 $1 \leq i \leq s, 1 \leq b \leq B$, $u_{k,(i-1)B+b} = s'_{i,k,b}$ 。
- 5.** 各参与方 $P_k (1 \leq k \leq m)$ 将 $\mathbf{u}_k$ 输入到 $\mathcal{F}_{\text{shuffle}}$, 得到随机置换后的分享份额 $\mathbf{u}'_k$ 。
- 6.** $P_j (1 < j \leq m)$ 将 $\mathbf{u}'_j$ 发给 P_1 。 P_1 计算 $\mathbf{u}' = \sum \mathbf{u}'_j$, 识别出所有满足 $y \| 0^{l'}$ 形式的分量, 截取 y 组成 Y 并输出。

MPSO-card: 3. 各参与方 $P_k (1 \leq k \leq m)$ 构造向量 $\mathbf{u}_k \in \mathbb{F}^{sB}$, 其中对于 $1 \leq i \leq s, 1 \leq b \leq B$, $u_{k,(i-1)B+b} = s_{i,k,b}$ 。

- 4.** 各参与方 $P_k (1 \leq k \leq m)$ 将 $\mathbf{u}_k$ 输入到 $\mathcal{F}_{\text{shuffle}}$, 得到随机置换后的分享份额 $\mathbf{u}'_k$ 。
- 5.** $P_j (1 < j \leq m)$ 将 $\mathbf{u}'_j$ 发送给 P_1 。 P_1 统计重构结果 $\sum_{1 \leq j \leq m} \mathbf{u}'_j$ 中 0 的数量, 将其作为 Y 的基数并输出。

Circuit-MPSO (方案 1): 3. 所有参与方调用电路为 $C_{s,B,l}^1$ 的 m 方通用 MPC 协议。对于 $1 \leq i \leq s, 1 \leq k \leq m$, P_k 将分享份额 $s_{i,k,1}, \dots, s_{i,k,B}$ 输入到第 i 部分电路; 此外, P_j (与步骤 2 中的 j 对应) 输入 $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B$ 中的元素。

Circuit-MPSO (方案 2): 3. 对于 $1 \leq i \leq s, 1 \leq b \leq B$, P_j (步骤 2 中的 j) 检查 $\mathcal{C}_j^b$: 若非空, 计算 $s'_{i,j,b} = \text{code}(s_{i,j,b}) \oplus (x \| 0^{l'})$, 其中 x 为桶中元素; 若为空, 随机采样 $s'_{i,j,b} \leftarrow \{0, 1\}^{l+l'}$ 。

- 4.** 所有参与方调用电路为 $C_{N,l'}^2$ 的 m 方通用 MPC 协议。对于 $1 \leq i \leq s, 1 \leq b \leq B, 1 \leq k \leq m$, P_k 将 $s'_{i,k,b}$ 作为其第 $((i-1)B+b)$ 个输入。

对全部 sB 个秘密分享进行随机置换。由于成员零分享满足独立性要求，各随机秘密分享中的秘密独立于任意 $t \leq m - 1$ 个参与方的联合视图。此外，多方秘密分享洗牌功能确保了任何 $t \leq m - 1$ 个参与方的联盟都无法获知该置换。因此，模拟视图与真实视图是不可区分的。下面是完整的安全性证明：

令 $\mathbf{P}_A$ 表示由敌手 $\mathcal{A}$ 控制的被腐化方集合。在 MPSO 协议中，模拟器从每个被腐化方 $P_c \in \mathbf{P}_A$ 处接收其输入 X_c ，并且如果 $P_1 \in \mathbf{P}_A$ ，模拟器还会接收结果集 Y 。对于每个 P_c ，其视图由其输入 X_c 、来自每个 $\mathcal{F}_{\text{bMZS}}^{Q'_i}$ ($1 \leq i \leq s$) 的 B 个秘密分享份额 $\mathbf{s}_{i,c}$ (如果 P_c 属于 Q_i 的涉及方集合 $\{P_{i_1}, \dots, P_{i_q}\}$)、来自 $\mathcal{F}_{\text{shuffle}}$ 的洗牌后的秘密分享份额 $\mathbf{u}'_c$ 组成；此外如果 $P_1 \in \mathbf{P}_A$ ，还包括来自 P_j ($1 < j \leq m$) 的重构消息 $\mathbf{u}'_j$ 。

假设有 s' 个子公式 $\mathbf{Q}_A = \{Q_{j_1}, \dots, Q_{j_{s'}}\} \subseteq \{Q_1, \dots, Q_s\}$ 不涉及诚实方，而 $\mathbf{Q}_H = \{Q_1, \dots, Q_s\} \setminus \mathbf{Q}_A$ 包含了至少涉及一个诚实方的子公式。模拟器通过诚实执行协议来模拟每个 P_c 的视图，但进行以下修改：

- 对于每个 $Q_h \in \mathbf{Q}_H$ ，模拟来自 $\mathcal{F}_{\text{bMZS}}^{Q'_h}$ 的均匀秘密分享份额。
- **情况** $P_1 \notin \mathbf{P}_A$ 。从 $\mathcal{F}_{\text{shuffle}}$ 均匀采样秘密分享份额 $\mathbf{u}'_c$ 。
- **情况** $P_1 \in \mathbf{P}_A$ 。在被腐化方诚实地调用了所有 $\mathbf{Q}_A$ 中子公式的批量成员零分享协议后，各方持有 $s'B$ 个秘密分享。将其对应的所有元素秘密值（拼接全零字符串后）记为集合 $Y_A \subseteq Y$ ，将 $s'B - |Y_A|$ 个随机秘密值记为集合 R_A 。令 $Y_H = Y \setminus Y_A$ 。模拟器采样 $(s - s')B - |Y_H|$ 个随机值作为集合 R_H ，使用随机置换 π 对并集 $Y \cup R_H \cup R_A$ 进行洗牌，并将洗牌后的并集秘密分享为 $\mathbf{u}'_1, \dots, \mathbf{u}'_m$ ，其中 $\mathbf{u}'_c$ 作为来自 $\mathcal{F}_{\text{shuffle}}$ 的秘密分享份额输出给每个 $P_c \in \{P_{i_1}, \dots, P_{i_q}\}$ 。

在 $P_1 \notin \mathbf{P}_A$ 的情况下，可以看出， P_c 从每个 $\mathcal{F}_{\text{bMZS}}^{Q'_h}$ 和 $\mathcal{F}_{\text{shuffle}}$ 获得的秘密分享份额在真实执行中均匀分布且独立于任何其他分布（因为至少存在一个诚实方持有其中一个份额），这与模拟完全一致。

在 $P_1 \in \mathbf{P}_A$ 的情况下，对于每个 $Q_h \in \mathbf{Q}_H$ ， P_c 从 $\mathcal{F}_{\text{bMZS}}^{Q'_h}$ 获得的秘密分享份额同样是均匀分布且独立于真实执行中的任何其他分布的，因此：

$$\begin{aligned} & \{\text{Sim}^{Q_h}(\mathbf{P}_A^h, \mathbf{X}_A^h, \{\mathbf{s}_{h,c}\}_{P_c \in \mathbf{P}_A^h})_{Q_h \in \mathbf{Q}_H}, \{\mathbf{s}_{h,c}\}_{Q_h \in \mathbf{Q}_H, P_c \in \mathbf{P}_A^h}\} \mathbf{X} \\ & \stackrel{c}{\approx} \{\text{View}_A^{Q_h}(\mathbf{X}^h)_{Q_h \in \mathbf{Q}_H}, \{\mathbf{s}_{h,c}^\Pi\}_{Q_h \in \mathbf{Q}_H, P_c \in \mathbf{P}_A^h}\} \mathbf{X}, \end{aligned}$$

其中 $\mathbf{X} = \{X_1, \dots, X_m\}$ 。 $\mathbf{P}_A^h$ 表示参与 Q_h 的被腐化方组成的集合， $\mathbf{X}_A^h$ 包含其所有的输入集合， $\mathbf{X}^h$ 包含 Q_h 中涉及的所有参与方的输入集合。 Sim^{Q_h} 是由模拟器模拟出的 $\mathcal{F}_{\text{bMZS}}^{Q'_h}$ 的视图，而 $\text{View}_A^{Q_h}$ 是批量成员零分享协议中敌手在 Q_h 上的真实视图。带有上标 Π 的项代表真实执行中的区分项，否则代表模拟中的项。由于被腐化方诚实地调用了 $\mathbf{Q}_A$ 中所有子公式的批量成员零分享协议，我们得到：

$$\begin{aligned} & \{\text{Sim}^{Q_i}(\mathbf{P}_A^i, \mathbf{X}_A^i, \{\mathbf{s}_{i,c}\}_{P_c \in \mathbf{P}_A^i})_{1 \leq i \leq s}, \{\mathbf{s}_{i,c}\}_{1 \leq i \leq s, P_c \in \mathbf{P}_A^i}\} \mathbf{X} \\ & \stackrel{c}{\approx} \{\text{View}_A^{Q_i}(X_{i_1}, \dots, X_{i_q})_{1 \leq i \leq s}, \{\mathbf{s}_{i,c}^\Pi\}_{1 \leq i \leq s, P_c \in \mathbf{P}_A^i}\} \mathbf{X}. \end{aligned}$$

根据正确性, 在为每个 $Q_h \in \mathbf{Q}_{\mathcal{H}}$ 调用所有 $\mathcal{F}_{\text{bMZS}}^{Q'_h}$ 之后, 各方持有 $|Y_{\mathcal{H}}|$ 个 $Y_{\mathcal{H}}$ 中元素的秘密分享份额, 以及 $(s - s')B - |Y_{\mathcal{H}}|$ 个随机值的秘密分享份额 (这些随机秘密值的集合记为 $R_{\mathcal{H}}^{\Pi}$)。根据 $\mathcal{F}_{\text{bMZS}}^{Q'_h}$ 的独立性要求, 在批量成员零分享协议的真实执行中, $R_{\mathcal{H}}^{\Pi}$ 中的所有随机值均独立于任何 $m - 1$ 个参与方的联合视图 (即敌手视图)。在模拟中, $R_{\mathcal{H}}$ 中的随机值是使用独立的随机源采样的, 因此它们也独立于为 $\mathcal{F}_{\text{bMZS}}^{Q'_h}$ 模拟出的视图。在执行多方秘密分享洗牌后, $Y \cup R_{\mathcal{H}}^{\Pi} \cup R_{\mathcal{A}}^{\Pi}$ 中元素的顺序被随机打乱。根据 $\mathcal{F}_{\text{shuffle}}$ 的功能, 随机置换 π^{Π} 是独立采样的。因此:

$$\begin{aligned} & \{\text{Sim}^{Q_i}(\mathbf{P}_{\mathcal{A}}^i, \mathbf{X}_{\mathcal{A}}^i, \{\mathbf{s}_{i,c}\}_{P_c \in \mathbf{P}_{\mathcal{A}}^i})_{1 \leq i \leq s}, \{\mathbf{s}_{i,c}\}_{1 \leq i \leq s, P_c \in \mathbf{P}_{\mathcal{A}}}, \pi(Y \cup R_{\mathcal{H}} \cup R_{\mathcal{A}})\}_{\mathbf{X}} \\ & \stackrel{c}{\approx} \{\text{View}_{\mathcal{A}}^{Q_i}(X_{i_1}, \dots, X_{i_q})_{1 \leq i \leq s}, \{\mathbf{s}_{i,c}^{\Pi}\}_{1 \leq i \leq s, P_c \in \mathbf{P}_{\mathcal{A}}}, \pi^{\Pi}(Y \cup R_{\mathcal{H}}^{\Pi} \cup R_{\mathcal{A}}^{\Pi})\}_{\mathbf{X}}. \end{aligned}$$

由于 $\mathbf{u}'_1, \dots, \mathbf{u}'_m$ 和 $\mathbf{u}^{\Pi}_1, \dots, \mathbf{u}^{\Pi}_m$ 分别是 $\pi(Y \cup R_{\mathcal{H}} \cup R_{\mathcal{A}})$ 与 $\pi^{\Pi}(Y \cup R_{\mathcal{H}}^{\Pi} \cup R_{\mathcal{A}}^{\Pi})$ 的秘密分享, 可推导出:

$$\begin{aligned} & \{\text{Sim}^{Q_i}(\mathbf{P}_{\mathcal{A}}^i, \mathbf{X}_{\mathcal{A}}^i, \{\mathbf{s}_{i,c}\}_{P_c \in \mathbf{P}_{\mathcal{A}}^i})_{1 \leq i \leq s}, \{\mathbf{s}_{i,c}\}_{1 \leq i \leq s, P_c \in \mathbf{P}_{\mathcal{A}}}, \mathbf{u}'_1, \dots, \mathbf{u}'_m\}_{\mathbf{X}} \\ & \stackrel{c}{\approx} \{\text{View}_{\mathcal{A}}^{Q_i}(X_{i_1}, \dots, X_{i_q})_{1 \leq i \leq s}, \{\mathbf{s}_{i,c}^{\Pi}\}_{1 \leq i \leq s, P_c \in \mathbf{P}_{\mathcal{A}}}, \mathbf{u}^{\Pi}_1, \dots, \mathbf{u}^{\Pi}_m\}_{\mathbf{X}}. \end{aligned}$$

通过调用多方秘密分享洗牌的模拟器 Sim^{sh} , 得到

$$\begin{aligned} & \{\text{Sim}^{Q_i}(\mathbf{P}_{\mathcal{A}}^i, \mathbf{X}_{\mathcal{A}}^i, \{\mathbf{s}_{i,c}\}_{P_c \in \mathbf{P}_{\mathcal{A}}^i})_{1 \leq i \leq s}, \{\mathbf{s}_{i,c}\}_{1 \leq i \leq s, P_c \in \mathbf{P}_{\mathcal{A}}}, \text{Sim}^{sh}(\mathbf{P}_{\mathcal{A}}, \{\mathbf{u}_c, \mathbf{u}'_c\}_{P_c \in \mathbf{P}_{\mathcal{A}}}), \mathbf{u}'_1, \dots, \mathbf{u}'_m\}_{\mathbf{X}} \\ & \stackrel{c}{\approx} \{\text{View}_{\mathcal{A}}^{Q_i}(X_{i_1}, \dots, X_{i_q})_{1 \leq i \leq s}, \{\mathbf{s}_{i,c}^{\Pi}\}_{1 \leq i \leq s, P_c \in \mathbf{P}_{\mathcal{A}}}, \text{View}_{\mathcal{A}}^{sh}(\mathbf{u}^{\Pi}_1, \dots, \mathbf{u}^{\Pi}_m), \mathbf{u}^{\Pi}_1, \dots, \mathbf{u}^{\Pi}_m\}_{\mathbf{X}}, \end{aligned}$$

其中 $\mathbf{u}_k$ 由 $\{\mathbf{s}_{i,k}\}_{1 \leq i \leq s}$ 计算得到。由此得出结论: 敌手在真实执行中的视图与在模拟中的视图是不可区分的。

Circuit-MPSO (方案 2) 的安全性证明相同。协议的安全性证明与之相同。MPSO-card 和 Circuit-MPSO (方案 1) 协议的安全性证明与之类似, 不同之处在于模拟器将模拟中的所有元素替换为 0, 因为在 $P_1 \in \mathbf{P}_{\mathcal{A}}$ 的情况下, 模拟器只能获得基数而非集合本身。

5.5.3 复杂度分析

图 5.5 中的计算和通信复杂度均由所计算的可构造集合 Y 的 CPSF 表示 $\psi(x, X_1, \dots, X_m)$ 形式决定, 其中 $\psi = Q_1 \vee \dots \vee Q_s$ 。

在成员零分享阶段, 每个参与方 P_j 的计算复杂度包括两部分: (1) $O(\sum_{1 \leq i \leq s} (i_q \cdot |\text{AND}_i + \text{OR}_i| \cdot n))$, 其中 Q_i 相对于 X_j 是局部的 (我们用 Q'_i 表示 Q_i 的 X_j -局部剩余公式), i_q 是文字的数量, $|\text{AND}_i + \text{OR}_i|$ 是 Q'_i 中 AND 和 OR 操作的总数; (2) $O(\sum_{1 \leq i \leq s} (|\text{AND}_i + \text{OR}_i| \cdot n))$, 其中 Q_i 相对于 X_j 不是局部的但包含 X_j , 且 $|\text{AND}_i + \text{OR}_i|$ 是 Q_i 相对于其他某个集合的局部剩余公式中 AND 和 OR 操作的总数。

P_j 的通信复杂度同样可以计算为两部分：(1) $O(\sum_{1 \leq i \leq s} (i_q \cdot |\text{OR}_i| \cdot n))$ ，其中 Q_i 相对于 X_j 是局部的（我们用 Q'_i 表示 Q_i 的 X_j -局部剩余公式）， i_q 是文字的数量， $|\text{OR}_i|$ 是 Q'_i 中 OR 操作的数量；(2) $O(\sum_{1 \leq i \leq s} (|\text{OR}_i| \cdot n))$ ，其中 Q_i 相对于 X_j 不是局部的但包含 X_j ，且 $|\text{OR}_i|$ 是 Q_i 相对于其他某个集合的局部剩余公式中 OR 操作的数量。

在多方秘密分享洗牌与重构阶段，Leader 的计算复杂度和通信复杂度均为 $O(smn)$ ，而每个 Client 的计算复杂度和通信复杂度均为 $O(sn)$ ，其中 s 为 CSPF 表示 ψ 中子公式的数量。

5.6 本章小结

6 基于对称密钥的实例化协议与综合性能评估

6.1 引言

在第五章中, 本文构建了一个基于对称密钥技术的多方隐私集合运算 (MPSO) 统一框架。该框架通过首先引入规范集合谓词公式 (Canonical Set Predicate Formula, CSPF) 对目标集合公式进行统一表示, 然后提出谓词零分享 (Predicative Zero-Sharing) 这一新型密码学原语及其子类——成员零分享 (Membership Zero-Sharing) 作为核心组件, 成功实现了计算任意集合公式的通用 MPSO 功能, 并打破了以往研究中的“功能孤岛”, 解决了现有协议功能局限和缺乏统一性的问题。

然而, 在安全多方计算 (MPC) 领域, 协议的“通用性”往往意味着在处理某些具体场景时可能存在性能冗余。例如, 在上述通用 MPSO 框架的设计中, 为了消除按照 CSPF 中子公式的顺序生成秘密分享的位置与重构出的元素所属局部子集和在布谷鸟哈希表中的位置的之间的关联, 以防止秘密分享重构阶段造成信息泄露, 必须强制要求所有参与方在重构前执行一个多方秘密分享洗牌协议。然而, 在某些特殊情况下, 例如 MPSI 协议中, 由于交集必然是 Leader 输入集合的子集, 因此重构出的秘密的位置完全由其输入集合决定, 这对其而言不再构成额外的信息泄露。基于这一子集属性, 可以直接在 MPSI 协议中省去多方秘密分享洗牌这一步骤, 优化实例化出的 MPSI 协议的效率。

为了在实际应用中实现极致的效率, 本章针对 MPSO 中的典型功能, 在第五章提出的通用 MPSO 框架下实例化的各具体协议进行深度的协议级优化, 通过**结合特定集合公式的结构特性, 裁剪通用流程中的冗余步骤**, 给出了高效的 MPSI、MPCSI 和 MPSU 协议, 其中的 MPSI 协议是首个在标准半诚实模型下实现**最优渐进复杂度** (即达到与不考虑隐私的明文传输方案相同量级的复杂度——Leader 的计算与通信复杂度为 $O(mn)$, Client 的计算与通信复杂度为 $O(n)$) 的基于对称密钥技术的构造; MPSI-card 和 MPSI-card-sum 协议是首个在标准半诚实模型下实现**最优渐进复杂度**的 MPSI-card 和 MPSI-card-sum 协议; Circuit-MPSI 协议是首个在**不诚实大多数 (Dishonest Majority)** 设定下安全的 Circuit-MPSI 构造; MPSU 协议是标准半诚实模型下基于对称密钥的另一 MPSU 方案, 相比第三章提出的 MPSU 协议, 在理论上实现了**更优的渐进复杂度**。

为了验证第五章提出的通用 MPSO 框架的实际可用性, 我们使用 C++ 语言系统实现了该框架下实例化出的具体协议, 并在相同的硬件平台和网络环境 (涵盖局域网 LAN 和广域网 WAN) 下, 对所有协议, 包括第三章和第四章提出的 MPSU 协议与当前标准半诚实模型下各个具体子领域最先进 (State-of-the-Art, SOTA) 的方案进行了统一的基准测试 (Benchmark) 与性能对比。我们的实验数据充分证明, 该框架在保证了功能通用性的同时, 其实际在线效率已达到甚至超越了现有的各 SOTA 协议。

本章后续结构如下：6.2 节和 6.3 节首先介绍框架底层组件成员零分享的两个特例——全成员零分享和全非成员零分享的具体构造；6.4 节给出针对 MPSI 及其变体 (MPSI, MPSI-card, MPSI-card-sum, Circuit-MPSI) 的框架实例化协议的优化版本；6.5 节探讨针对 MPSU 及其变体 (MPSU, MPSU-card, Circuit-MPSU) 的框架实例化协议的优化版本；6.6 节详细分析了各类协议的复杂度并给出了各个具体子领域的相关工作的全面理论对比；6.7 节展示各类协议的具体实现细节及综合性能评估结果；最后，6.8 节对本章内容进行总结。

6.2 全成员零分享及其变体

6.2.1 全成员零分享

批量全成员零分享 (Batch Full Membership Zero-Sharing) 是成员零分享在特定谓词下的一个特例, 即当关联的公式 Q 是 $m-1$ 个集合成员谓词的合取时 (即 $\bigwedge_{j \in \{1, \dots, m\} \setminus \{\text{pivot}\}} x \in X_j$)。该功能专门为第 ?? 节中优化的 MPSI 及其变体设计。理想功能 $\mathcal{F}_{\text{bpMZS}}$ 在图 6.1 中给出, 其完整协议构造在图 6.2 中描述。

参数： m 个参与方 $P_1, \dots, P_m$, 其中仅枢纽方 P_{pivot} 持有 n 个独立元素作为输入, 其余参与方持有大小为 n 的集合。批量大小 n 。有限域 $\mathbb{F}$ 。

功能： 接收来自 P_{pivot} 的输入 $\mathbf{x} = (x_1, \dots, x_n)$ 以及来自每个 P_j ($j \in \{1, \dots, m\} \setminus \{\text{pivot}\}$) 的输入 $\mathbf{X}_j = (X_{j,1}, \dots, X_{j,n})$ 。对于 $1 \leq i \leq m$, 采样 $\mathbf{s}_i = (s_{i,1}, \dots, s_{i,n}) \leftarrow \mathbb{F}^n$, 且满足: 对于 $1 \leq d \leq n$, 如果 $\bigwedge_{j \in \{1, \dots, m\} \setminus \{\text{pivot}\}} (x_d \in X_{j,d}) = 1$, 则 $\sum_{1 \leq i \leq m} s_{i,d} = 0$ 。向 P_i 发送 $\mathbf{s}_i$ 。

图 6.1 批量全成员零分享功能 $\mathcal{F}_{\text{bpMZS}}$

复杂度分析. 在批量全成员零分享协议 (图 6.2) 中, 各阶段的开销计算如下:

- P_{pivot} 与每个 P_j 执行批量大小为 n 的 $\mathcal{F}_{\text{bOPPRF}}$ 。结合哈希分桶技术时, 每组集合大小 $|X_{j,i}| = O(1)$ 。我们遵循相关文献利用向量不经意线性求值 (Vector Oblivious Linear Evaluation, VOLE) 和不经意键值存储 (OKVS) 来实例化 $\mathcal{F}_{\text{bOPPRF}}$ 。利用通信均摊技术, 计算 n 个实例的总通信量与总元素数量 $3n$ 相当。这使得批量 OPPRF 的计算复杂度随 n 线性扩展。因此, 在此阶段, P_{pivot} 的计算和通信复杂度均为 $O(mn)$, 每个 P_j 的计算和通信复杂度均为 $O(n)$ 。
- 参与方利用 Beaver 三元组执行 n 次安全乘法, 并指定 P_{pivot} 作为领导者。这需要 P_{pivot} 产生 $O(mn)$ 的开销, 每个 P_j 产生 $O(n)$ 的开销。

参数: m 个参与方 $P_1, \dots, P_m$ 。批量大小 n 。有限域 $\mathbb{F}$ 。离线阶段生成的 n 个 Beaver 三元组 $([\mathbf{a}], [\mathbf{b}], [\mathbf{c}])$, 其中 $[\mathbf{a}] = ([a_1], \dots, [a_n])$, $[\mathbf{b}] = ([b_1], \dots, [b_n])$, $[\mathbf{c}] = ([c_1], \dots, [c_n])$ 且满足 $c_i = a_i \cdot b_i$ 。

输入: P_{pivot} 输入向量 $\mathbf{x} = (x_1, \dots, x_n)$ 。对于 $j \in \{1, \dots, m\} \setminus \{\text{pivot}\}$, P_j 输入 $\mathbf{X}_j = (X_{j,1}, \dots, X_{j,n})$ 。

协议流程:

1. 对于第 i 个实例 ($1 \leq i \leq n$), P_j 采样 $r_{j,i}$ 并设置键集合 $K_{j,i} = X_{j,i}$, 值集合 $V_{j,i} = \{-r_{j,i}, \dots, -r_{j,i}\}$, 其中 $|K_{j,i}| = |V_{j,i}|$ 。
2. P_{pivot} 和 P_j 调用 $\mathcal{F}_{\text{bOPPRF}}$ 。其中 P_j 作为发送方 $\mathcal{S}$ 输入 $(K_{j,1}, \dots, K_{j,n})$ 和 $(V_{j,1}, \dots, V_{j,n})$; P_{pivot} 作为接收方 $\mathcal{R}$ 输入 $\mathbf{x}$ 并接收输出 $\mathbf{u}_j$ 。
3. P_{pivot} 设置其秘密分享份额为 $\mathbf{r}_{\text{pivot}} = \sum_{j \in \{1, \dots, m\} \setminus \{\text{pivot}\}} \mathbf{u}_j$ 。 P_j 设置其份额为 $\mathbf{r}_j = (r_{j,1}, \dots, r_{j,n})$ 。此时, 所有参与方持有一个包含 n 个秘密分享的向量 $[\mathbf{r}] = ([r_1], \dots, [r_n])$ 。
4. 所有参与方使用 Beaver 三元组 $([\mathbf{a}], [\mathbf{b}], [\mathbf{c}])$ 执行 n 次安全乘法计算, 得到 $[\mathbf{s}]$, 即 $[s_i] = [r_i] \cdot [b_i]$ ($1 \leq i \leq n$)。

图 6.2 批量全成员零分享协议 Π_{bpMZS}

综上所述, 在在线阶段, P_{pivot} 的整体计算和通信复杂度为 $O(mn)$, 每个 P_j 的计算和通信复杂度为 $O(n)$ 。

6.2.2 带载荷的全成员零分享

为了支持 MPSI-card-sum 功能, 我们引入了一种称为“带载荷的批量全成员零分享 (Batch Full Membership Zero-Sharing with Payloads)”的变体。在此变体中, P_{pivot} 持有元素 x , 其他参与方持有元素集合及关联的载荷 (Payload) 集合。如果公式为真 (即 x 属于所有集合), 参与方将获得 0 的秘密分享, 以及 x 的所有关联载荷之和的秘密分享; 否则获得两个随机值的分享。

其构造思想与图 6.2 类似, 核心在于将载荷集合编码到 OPPrF 发送方的输入值集合中。具体而言, P_j 采样两个份额 r, w , 并设置值集合为一对元组 $(-r, v_i - w)$, 其中 v_i 是元素的关联载荷。通过 $\mathcal{F}_{\text{bOPPrF}}$, P_{pivot} 最终可以获得 (r', w') 。如果 x 匹配, 则 $(r', w') = (-r, v - w)$ 。该变体理想功能 $\mathcal{F}_{\text{bpMZSp}}$ 的定义详见图 6.3, 协议细节见图 6.4。其渐进复杂度与不带载荷的版本保持一致。

参数: m 个参与方 $P_1, \dots, P_m$ 。批量大小 n 。有限域 $\mathbb{F}$ 及载荷域 $\mathbb{F}'$ 。映射函数 $\text{payload}_j()$ 用于关联元素与载荷。

功能: 接收 P_{pivot} 的输入 $\mathbf{x}$, 以及 P_j 的集合 $\mathbf{X}_j$ 和载荷 $\mathbf{V}_j$ 。对于 $1 \leq i \leq m$, 采样 $\mathbf{s}_i \leftarrow \mathbb{F}^n, \mathbf{w}_i \leftarrow \mathbb{F}'^n$, 满足: 对于 $1 \leq d \leq n$, 如果 $\bigwedge (x_d \in X_{j,d}) = 1$, 则 $\sum s_{i,d} = 0$ 且 $\sum w_{i,d} = \sum v_{j,d}$ (其中 $v_{j,d} = \text{payload}_j(x_d)$)。向 P_i 输出 $(\mathbf{s}_i, \mathbf{w}_i)$ 。

图 6.3 带载荷的批量全成员零分享功能 $\mathcal{F}_{\text{bpMZSp}}$

参数: 同上, 另需 n 个 Beaver 三元组 $([\mathbf{a}], [\mathbf{b}], [\mathbf{c}])$ 。

协议流程:

1. 对于第 i 个实例, P_j 采样 $(r_{j,i}, w_{j,i})$ 。设置键集合 $K_{j,i} = X_{j,i} = (x_{j,i,1}, \dots)$, 值集合 $V'_{j,i} = \{(-r_{j,i}, v_{j,i,1} - w_{j,i}), \dots\}$ 。
2. P_{pivot} 和 P_j 调用 $\mathcal{F}_{\text{bOPPRF}}$ 。 P_j 作为 $\mathcal{S}$ 输入键值对, P_{pivot} 作为 $\mathcal{R}$ 接收 $(\mathbf{r}'_j, \mathbf{w}'_j)$ 。
3. P_{pivot} 设置份额 $\mathbf{r}_{\text{pivot}} = \sum \mathbf{r}'_j$, $\mathbf{w}_{\text{pivot}} = \sum \mathbf{w}'_j$ 。 P_j 设置份额 $\mathbf{r}_j$ 和 $\mathbf{w}_j$ 。
4. 所有参与方使用三元组计算 $[\mathbf{s}] = [\mathbf{r}] \cdot [\mathbf{b}]$ 。

图 6.4 带载荷的批量全成员零分享协议 Π_{bpMZSp}

6.3 全非成员零分享

批量全非成员零分享 (Batch Full Non-Membership Zero-Sharing) 是当关联公式 Q 是 $m - 1$ 个集合非成员谓词的合取时 (即 $\bigwedge_{j \in \{1, \dots, m\} \setminus \{\text{pivot}\}} x \notin X_j$) 的特例。该功能专门为第 ?? 节中的优化的 MPSU 及其变体设计。理想功能 $\mathcal{F}_{\text{bpNMZS}}$ 在图 6.5 中给出, 协议构造见图 6.6。

该协议的构造核心在于将底层的秘密分享隐私成员测试 (ssPMT, 输出布尔秘密分享) 高效地转化为算术域 $\mathbb{F}$ 上的零分享。我们利用双选择比特的 ROT 协议完成了这一转化, 实现了线性开销。

参数: m 个参与方 $P_1, \dots, P_m$ 。批量大小 n 。有限域 $\mathbb{F}$ 。

功能: 接收输入 $\mathbf{x}$ 和 $\mathbf{X}_j$ 。对于 $1 \leq i \leq m$, 采样 $\mathbf{s}_i \leftarrow \mathbb{F}^n$, 且满足: 如果 $\bigwedge_{j \in \{1, \dots, m\} \setminus \{\text{pivot}\}} x_d \notin X_{j,d} = 1$, 则 $\sum_{1 \leq i \leq m} s_{i,d} = 0$ 。向 P_i 发送 $\mathbf{s}_i$ 。

图 6.5 批量全非成员零分享功能 $\mathcal{F}_{\text{bpNMZS}}$

参数: m 个参与方, 批量大小 n 。Beaver 三元组 $([\mathbf{a}], [\mathbf{b}], [\mathbf{c}])$ 。

协议流程:

1. P_{pivot} 和 P_j 调用 $\mathcal{F}_{\text{bssPMT}}$ 。对于第 i 个实例, P_j 输入 $X_{j,i}$ 获得布尔份额 $e_{j,i}^0$, P_{pivot} 输入 x_i 获得布尔份额 $e_{j,i}^1$ 。
2. P_{pivot} 和 P_j 调用 n 个 ROT 实例。 P_j 作为发送方 $\mathcal{S}$ 接收随机数 $r_{j,i}^0, r_{j,i}^1$, P_{pivot} 作为接收方 $\mathcal{R}$ 输入选择比特 $e_{j,i}^1$ 并接收 $r_{j,i}^{e_{j,i}^1}$ 。 P_{pivot} 设置 $\mathbf{r}'_j = (r_{j,1}^{e_{j,1}^1}, \dots, r_{j,n}^{e_{j,n}^1})$ 。
3. P_{pivot} 设置份额 $\mathbf{r}_{\text{pivot}} = \sum_j \mathbf{r}'_j$ 。 P_j 设置份额 $\mathbf{r}_j = (-r_{j,1}^{e_{j,1}^0}, \dots, -r_{j,n}^{e_{j,n}^0})$ 。
4. 所有参与方使用三元组计算 $[\mathbf{s}] = [\mathbf{r}] \cdot [\mathbf{b}]$ 。

图 6.6 批量全非成员零分享协议 Π_{bpNMZS}

复杂度分析. 在协议中, P_{pivot} 与每个 P_j 执行 $\mathcal{F}_{\text{bssPMT}}$ 。基于本文第三章的结论, 该操作具有线性的计算和通信复杂度。后续的 ROT 操作也可在离线阶段预处理, 使得在线阶段 P_{pivot} 仅需发送掩码后的选择比特。最后的三元组乘法同样保持线性开销。因此, 在线阶段 P_{pivot} 的复杂度为 $O(mn)$, 每个 Client P_j 为 $O(n)$ 。

6.4 多方隐私集合求交与交集计算

考虑计算目标为一个可构造交集 Y ，其规范集合谓词公式 ψ 相对于 X_1 是局部的。例如 $X_1 \cap X_2 \cap X_3$ 相对于 X_1 的局部剩余公式为 $x \in X_2 \wedge x \in X_3$ 。在这种情况下， Y 的所有元素必然属于 P_1 （即 $Y \subseteq X_1$ ）。这意味着秘密分享的排列顺序完全由 X_1 决定。对于 MPSI 而言，由于 P_1 最终将获得结果集，这种原本会被视为泄露的“顺序信息”对 P_1 而言不再构成真实的隐私泄露。

因此，在 MPSI 协议中，我们可以省去**多方秘密分享洗牌 (Shuffle)** 步骤。当 P_1 （作为枢纽方 P_{pivot} ）与其他各方调用完批量全成员零分享后，各方可以直接向 P_1 重构秘密。如果重构的秘密为 0，则说明 P_1 对应的元素属于交集。这一优化极大地提升了在线效率。（注意：对于 MPSI-card，为了防止 P_1 知道具体哪些元素在交集内，仍然需要执行洗牌操作）。

此外，在这种针对性优化下，协议开销不再依赖于元素的比特长度 l ，因为参与方可以提前在本地对元素进行哈希压缩。为了保证正确性，必须确保哈希过程在 P_1 和其他参与方的元素之间不引入冲突，因此哈希函数的输出长度至少为 $\sigma + \log_2(m-1) + 2\log_2 n$ 。

6.4.1 多方隐私集合求交、交集求势和基于电路的 MPSI

MPSI、MPSI-card 以及 Circuit-MPSI 的完整优化协议描述在图 6.7 中给出。值得注意的是，现有的最先进 MPSI 协议 [2, 3] 通常需要参与方之间两两在线交互 (Pairwise Interaction)，这导致 Client 的复杂度依赖于参与方数量或腐化阈值 t 。而我们的协议引入了**星型通信拓扑 (Star Topology)**，使得 Leader 的计算和通信复杂度为 $O(mn)$ ，每个 Client 仅为 $O(n)$ 。这达到了与不安全的朴素哈希方案相同的最优复杂度。据我们所知，这是首次在标准半诚实模型下同时实现 MPSI 和 MPSI-card 最优复杂度的构造。

6.4.2 多方隐私交集求势与和

MPSI-card-sum 协议通过利用 6.2 节中“带载荷的批量全成员零分享”原语实现。在此过程中，各方获得指示交集的零分享以及载荷的秘密分享。随后经过两次洗牌操作（分别对零分享和载荷分享进行打乱）， P_1 统计零分享计算基数，并基于零分享构建的指示向量汇总载荷求和。其渐进复杂度同样保持在最优水平（即与 MPSI 一致）。该协议的完整流程在图 6.8 中给出。

6.5 多方隐私集合求并与并集计算

参数： m 个参与方。集合大小 n 。有限域 $\mathbb{F}$ 。哈希参数 B 。

输入： P_i 输入集合 X_i 。

1. **哈希分桶.** P_1 使用布谷鸟哈希 $\mathcal{C}_1^b \leftarrow \text{Cuckoo}(X_1)$ 。 P_j ($j > 1$) 使用简单哈希 $\mathcal{T}_j^b \leftarrow \text{Simple}(X_j)$ 。
2. **批量全成员零分享.** 所有参与方调用 $\mathcal{F}_{\text{bpMZS}}$ ， P_1 作为 P_{pivot} 输入 $\mathcal{C}_1$ 。 P_i 获得份额 $\mathbf{s}_i$ 。

后续操作依功能而定：

MPSI.

3. P_j 发送 $\mathbf{s}_j$ 给 P_1 。 P_1 重构秘密 $\mathbf{s} = \sum \mathbf{s}_i$ 。对于每个 b ，若 $s_b = 0$ ，则将 $\mathcal{C}_1^b$ 中的元素加入输出结果 Y 中。

MPSI-card.

3. P_i 将 $\mathbf{s}_i$ 输入 $\mathcal{F}_{\text{shuffle}}$ 获得打乱后的 $\mathbf{s}'_i$ 。
4. P_j 发送 $\mathbf{s}'_j$ 给 P_1 。 P_1 统计 $\sum \mathbf{s}'_i$ 中 0 的数量作为基数输出。

Circuit-MPSI.

3. 参与方将分享份额 $s_{i,b}$ 输入到通用 MPC 电路中，电路在内部进行相等性判断与后续逻辑运算。

图 6.7 MPSI / MPSI-card / Circuit-MPSI 协议

参数： m 个参与方 $P_1, \dots, P_m$ 。集合大小 n 。元素长度 l 。有限域 $\mathbb{F}$ 。布谷鸟哈希
参数： 哈希函数 h_1, h_2, h_3 及桶数 B 。

输入： 每个参与方 P_i 拥有输入集合 $X_i = \{x_i^1, \dots, x_i^n\} \subseteq \{0, 1\}^l$ 及载荷集合 $V_i = \{v_i^1, \dots, v_i^n\}$ ，并具有从元素到关联载荷的映射函数 $\text{payload}_i()$ 。

1. **哈希分桶。** P_1 执行布谷鸟哈希 $\mathcal{C}_1^1, \dots, \mathcal{C}_1^B \leftarrow \text{Cuckoo}_{h_1, h_2, h_3}^B(X_1)$ 。对于 $1 < j \leq m$ ， P_j 执行简单哈希 $\mathcal{T}_j^1, \dots, \mathcal{T}_j^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_j)$ 。 P_j 进一步定义载荷桶 $\mathcal{V}_j^1, \dots, \mathcal{V}_j^B$ ，其中对于 $1 \leq b \leq B$ ， $\mathcal{V}_j^b$ 包含 $\mathcal{T}_j^b$ 中元素的关联载荷。
2. **带载荷的批量全成员零分享。** 所有参与方调用批大小为 B 的带载荷的批量全成员零分享功能 $\mathcal{F}_{\text{bpMZSp}}$ ，其中 P_1 作为 P_{pivot} 输入 $\mathcal{C}_1^1, \dots, \mathcal{C}_1^B$ ，其余每个 P_j 输入 $(\mathcal{T}_j^1, \dots, \mathcal{T}_j^B)$ 和 $(\mathcal{V}_j^1, \dots, \mathcal{V}_j^B)$ 。对于 $1 \leq i \leq m$ ， P_i 获得输出份额 $(\mathbf{s}_i, \mathbf{w}_i)$ 。
3. 对于 $1 \leq b \leq B$ ，若 $\mathcal{C}_1^b$ 不是空桶， P_1 设置 $w_{1,b} = w_{1,b} + v$ ，其中 v 是 $\mathcal{C}_1^b$ 中元素的关联载荷。
4. 对于 $1 \leq i \leq m$ ， P_i 分别将 $\mathbf{s}_i$ 和 $\mathbf{w}_i$ 输入到多方秘密分享洗牌功能 $\mathcal{F}_{\text{shuffle}}$ 中。 P_i 接收到打乱后的份额 $\mathbf{s}'_i = (s'_1, \dots, s'_B)$ 和 $\mathbf{w}'_i = (w'_1, \dots, w'_B)$ 。
5. 对于 $1 < j \leq m$ ， P_j 将 $\mathbf{s}'_j$ 发送给 P_1 。 P_1 计算重构的指示向量 $\mathbf{s}' = \sum_{1 < j \leq m} \mathbf{s}'_j$ ，并定义一个布尔向量 $\mathbf{e} = (e_1, \dots, e_B)$ ，规则为：若 $s'_b = 0$ ，则 $e_b = 1$ ；否则 $e_b = 0$ 。 P_1 将 $\mathbf{e}$ 分发给所有 P_j 。对于 $1 \leq i \leq m$ ， P_i 输出向量 $\mathbf{e}$ 的汉明重量 $\text{HW}(\mathbf{e})$ 作为交集基数。
6. 对于 $1 \leq i \leq m$ ， P_i 在本地计算载荷份额之和 $u_i = \sum_{1 \leq b \leq B \text{ s.t. } e_b=1} w'_{i,b}$ 。对于 $1 < j \leq m$ ， P_j 将 u_j 发送给 P_1 。 P_1 计算并输出最终的载荷总和 $u = \sum_{1 \leq i \leq m} u_i$ 。

图 6.8 MPSI-card-sum 协议

6.5.1 多方隐私集合求并、并集求势和基于电路的 MPSU

考虑并集计算 $Y = X_1 \cup \dots \cup X_m$, 其对应的规范集合谓词公式 (CSPF) 为若干子公式的析取, 其中某些子公式直接就是某个原子命题 $x \in X_i$ (对于某个 $1 \leq i \leq m$)。例如, $X_1 \cup \dots \cup X_m$ 可以表示为 $(x \in X_1) \vee ((x \notin X_1) \wedge (x \in X_2)) \vee \dots \vee ((x \notin X_1) \wedge \dots \wedge (x \notin X_{m-1}) \wedge (x \in X_m))$ 。在这种情况下, 子公式 $x \in X_i$ 仅涉及 P_i 自身, 因此 P_i 可以直接在本地对其元素生成秘密分享。特别地, 如果 $i = 1$, 由于 P_1 是最终接收并集的一方, 该子公式甚至可以直接被忽略, 只要 P_1 在最终重建的集合中简单地追加自己原本的输入集合 X_1 即可, 这避免了大量冗余操作。

6.5.2 多方隐私集合求并、并集求势和基于电路的 MPSU

基于上述思路, MPSU 及相关变体的完整优化协议在图 6.9 中给出。

本文的 MPSU 协议遵循了 Liu 和 Gao [1] 提出的基于秘密分享的 MPSU 范式 (该范式的原始设计依赖非共谋假设)。虽然在第三章中, 我们通过引入多方秘密分享随机不经意传输 (mss-ROT) 消除了这一安全缺陷, 但 mss-ROT 引入了大量的参与方两两交互 (Pairwise Interaction), 使得协议的复杂度受到一定影响。

与之形成鲜明对比的是, 本章基于通用框架实例化的 MPSU 协议, 利用全非成员零分享及其转化技术 (结合 Beaver 三元组), 成功保留了星型通信拓扑 (Star Topology)。这使得本协议不仅能够抵御任意共谋攻击, 而且在维持 Leader 为 $O(m^2n)$ 复杂度的同时, 将每个 Client 的计算与通信复杂度压低至 $O(mn)$, 首次实现了该基于秘密分享的 MPSU 范式下的最优渐进复杂度。

6.6 各类协议的复杂度分析与对比

6.6.1 MPSI 及其变体的复杂度分析

在以下的渐近复杂度分析中, 我们仅考虑对集合大小 n 和参与方数量 m 的依赖, 忽略安全参数的影响。

在图 6.7 中, 各参与方 (P_1 作为 P_{pivot}) 调用批量大小为 $B = O(n)$ 的批量全成员零分享协议。在该阶段, Leader P_1 的计算和通信复杂度均为 $O(mn)$, 而每个 Client P_j ($1 < j \leq m$) 的计算和通信复杂度均为 $O(n)$ 。在 MPSI 与 Circuit-MPSI 中, 每个 P_j 直接将其分享份额发送给 P_1 , 因此, P_1 的总计算和通信复杂度为 $O(mn)$, 而每个 P_j 的总计算和通信复杂度为 $O(n)$ 。在 MPSI-card 中, 参与方在直接重构前调用了多方秘密分享洗牌协议。我们采用文献 [62] 中的多方秘密分享洗牌协议并指定 P_1 作为 Leader。在该阶段, P_1 的计算和通信复杂度为 $O(mn)$, 而每个 P_j 的计算和通信复杂度为 $O(n)$ 。综上所述, MPSI-card 的总体计算和通信复杂度与 MPSI / Circuit-MPSI 完全相同。

参数： m 个参与方 $P_1, \dots, P_m$ 。集合大小 n 。元素长度 l 。全零字符串长度 l' 。有限域 $\mathbb{F}$ 。编码函数 $\text{code} : \mathbb{F} \rightarrow \{0, 1\}^{l+l'}$ 。布谷鸟哈希参数：哈希函数 h_1, h_2, h_3 及桶数 B 。

输入： 每个参与方 P_i 持有输入集合 $X_i = \{x_i^1, \dots, x_i^n\} \subseteq \{0, 1\}^l$ 。

1. **哈希分桶.** P_1 执行简单哈希 $\mathcal{T}_1^1, \dots, \mathcal{T}_1^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_1)$ 。对于 $1 < j \leq m$, P_j 执行布谷鸟哈希 $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B \leftarrow \text{Cuckoo}_{h_1, h_2, h_3}^B(X_j)$ 以及简单哈希 $\mathcal{T}_j^1, \dots, \mathcal{T}_j^B \leftarrow \text{Simple}_{h_1, h_2, h_3}^B(X_j)$ 。
2. **批量全非成员零分享.** 对于 $1 < j \leq m$, 参与方 $P_1, \dots, P_j$ 调用批大小为 B 的批量全非成员零分享功能 $\mathcal{F}_{\text{bpNMZS}}$ 。其中 P_j 作为 P_{pivot} 输入 $\mathcal{C}_j^1, \dots, \mathcal{C}_j^B$, 其他参与方 $P_{j'}$ ($j' \in \{1, \dots, j-1\}$) 输入 $\mathcal{T}_{j'}^1, \dots, \mathcal{T}_{j'}^B$ 。对于 $1 \leq i \leq j$, P_i 获得输出份额 $\mathbf{s}_{j,i} = (s_{j,i,1}, \dots, s_{j,i,B})$ 。

后续操作依功能而定：

MPSU.

3. 对于 $1 < j \leq m$ 和 $1 \leq b \leq B$, 若 $\mathcal{C}_j^b$ 不是空桶, P_j 计算 $s_{j,j,b} = \text{code}(s_{j,j,b}) \oplus (x \| 0^{l'})$, 其中 x 是 $\mathcal{C}_j^b$ 中的元素; 否则 P_j 随机采样 $s_{j,j,b} \leftarrow \{0, 1\}^{l+l'}$ 。
4. 对于 $1 \leq i \leq m$, P_i 构造向量 $\mathbf{u}_i \in (\{0, 1\}^{l+l'})^{(m-1)B}$, 规则如下: 对于 $\max(2, i) \leq j \leq m$ 和 $1 \leq b \leq B$, $u_{i,(j-2)B+b} = s_{j,i,b}$ 。将其余位置设为 0。
5. 对于 $1 \leq i \leq m$, P_i 将 $\mathbf{u}_i$ 输入到 $\mathcal{F}_{\text{shuffle}}$ 中, 接收打乱后的份额 $\mathbf{u}'_i$ 。
6. 对于 $1 < j \leq m$, P_j 将 $\mathbf{u}'_j$ 发送给 P_1 。 P_1 计算重构向量 $\mathbf{u}' = \sum_{1 < j \leq m} \mathbf{u}'_j$ 并初始化 $Y = \emptyset$ 。对于 $1 \leq b \leq (m-1)B$, 如果 u'_b 满足 $y \| 0^{l'}$ 的格式 (其中 $y \in \{0, 1\}^l$), P_1 将 y 加入集合 $Y = Y \cup \{y\}$ 。最终输出 $Y \cup X_1$ 。

MPSU-card.

3. 对于 $1 < j \leq m$ 和 $1 \leq b \leq B$, 如果 $\mathcal{C}_j^b$ 是空桶, P_j 随机选择 $s_{j,j,b}$ 的值。
4. 对于 $1 \leq i \leq m$, P_i 构造向量 $\mathbf{u}_i \in \mathbb{F}^{(m-1)B}$, 规则如下: 对于 $\max(2, i) \leq j \leq m$ 和 $1 \leq b \leq B$, $u_{i,(j-2)B+b} = s_{j,i,b}$ 。将其余位置设为 0。
5. 对于 $1 \leq i \leq m$, P_i 将 $\mathbf{u}_i$ 输入到 $\mathcal{F}_{\text{shuffle}}$ 中, 接收 $\mathbf{u}'_i$ 。
6. 对于 $1 < j \leq m$, P_j 将 $\mathbf{u}'_j$ 发送给 P_1 。 P_1 输出并集基数为 $n + \text{Zero}(\sum_{1 < j \leq m} \mathbf{u}'_j)$, 其中 $\text{Zero}()$ 表示统计重构向量中 0 的数量。

Circuit-MPSU (采用通用电路方案).

3. 对于 $1 < j \leq m$ 和 $1 \leq b \leq B$, 若 $\mathcal{C}_j^b$ 不是空桶, P_j 计算 $s_{j,j,b} = \text{code}(s_{j,j,b}) \oplus (x \| 0^{l'})$, 其中 x 是 $\mathcal{C}_j^b$ 中的元素; 否则 P_j 随机采样 $s_{j,j,b} \leftarrow \{0, 1\}^{l+l'}$ 。
4. 对于 $1 \leq i \leq m$, P_i 构造向量 $\mathbf{u}_i \in (\{0, 1\}^{l+l'})^{(m-1)B}$, 规则同上, 其余位置设为 0。
5. 所有参与方调用包含电路 $C_{(m-1)B, l'}^2$ 的 m 方通用 MPC。对于 $1 \leq i \leq m, 1 \leq k \leq (m-1)B$, P_i 将 $u_{i,k}$ 输入到电路中进行联合计算。

图 6.9 MPSU / MPSU-card / Circuit-MPSU 协议

在图 6.8 中, 各参与方调用批量大小为 $B = O(n)$ 的带载荷的批量全成员零分享协议。在该阶段, P_1 的计算和通信复杂度为 $O(mn)$, 而每个 P_j ($1 < j \leq m$) 的计算和通信复杂度为 $O(n)$ 。随后, 参与方调用两次多方秘密分享洗牌协议, P_j 向 P_1 重构基数部分, 并且 P_1 广播针对洗牌后载荷的指示向量。总体而言, MPSI-card-sum 的计算和通信复杂度同样与 MPSI / MPSI-card / Circuit-MPSI 保持一致。

值得注意的是, 在不考虑隐私安全的朴素 (Naive) 解决方案中, 各 Client 直接将其输入集合发送给 Leader, 由 Leader 在本地计算结果。在该方案中, Leader 的计算和通信复杂度为 $O(mn)$, 每个 Client 的计算和通信复杂度为 $O(n)$ 。因此, 我们的 MPSI / MPSI-card / Circuit-MPSI / MPSI-card-sum 构造在确保安全性的同时, 成功实现了与朴素方案相匹配的 ** 最优复杂度 **。

6.6.2 MPSU 及其变体的复杂度分析

在以下的渐近复杂度分析中, 我们同样仅考虑对 n 和 m 的依赖, 省略安全参数。

在图 6.9 中, 对于 $1 < j \leq m$, 参与方 $P_1, \dots, P_j$ (其中 P_j 作为 P_{pivot}) 调用批量大小为 $B = O(n)$ 的批量全非成员零分享协议。每个 P_j 共参与 $m - j + 1$ 次批量全非成员零分享协议的调用, 并在第一次调用中担任 P_{pivot} 。 P_1 参与 $m - 1$ 次调用, 但从不担任 P_{pivot} 。在该阶段, 每个参与方的计算和通信复杂度均为 $O(mn)$ 。此后, 各参与方持有 $(m - 1)B$ 个秘密分享。接着, 他们将这 $(m - 1)B = O(mn)$ 个分享份额输入到多方秘密分享洗牌协议 (以 P_1 为 Leader) 中, 最后每个 P_j 将洗牌后的份额发送给 P_1 。因此, 在该步骤中, P_1 的计算和通信复杂度为 $O(m^2n)$, 而每个 P_j 的计算和通信复杂度为 $O(mn)$ 。综合来看, Leader P_1 的总体计算和通信复杂度为 $O(m^2n)$, 而每个 Client P_j 的总体计算和通信复杂度为 $O(mn)$ 。

我们的 MPSU 协议遵循了基于秘密分享的 MPSU 范式, 其中 Leader 的最优计算和通信复杂度为 $O(m^2n)$, 每个 Client 的最优计算和通信复杂度为 $O(mn)$ 。这一最优复杂度边界是由该范式下“在 m 个参与方之间对 $O(mn)$ 个元素进行秘密分享”的核心设计所决定的, 因为必不可少的重构步骤本身就需要达到这一复杂度。因此, 我们的 MPSU 构造成功达到了该 MPSU 范式下的 ** 最优计算和通信复杂度 **。

6.6.3 与现有工作的理论对比

表 6.1 和表 6.2 分别展示了各种 MPSI 协议在在线阶段与离线阶段的理论计算与通信复杂度对比, 其中 n 为集合大小, m 为参与方数量, t 为共谋参与方数量。 U 为元素定义域。“PK”表示基于公钥操作的协议, “SK”表示基于 OT 和对称密钥操作的协议。 λ, σ 分别为计算和统计安全参数。“增强半诚实/恶意”表示该恶意协议隐含了增强半诚实安全性, 但在标准半诚实模型下不安全。文献 [47] 是目前唯一区分了离线和在线阶段的现有 MPSI 工作, 因此我们省略了其他无离线阶段的方案。不同的 Beaver 三元组生

成方式导致了本文协议具有不同的离线复杂度：“BGV”表示各方使用带有分布式解密的加法同态加密（AHE）生成 Beaver 三元组，其中 AHE 采用 BGV 加密实例化；“OT”表示各方利用两两 OT 交互（使用 SoftSpokenOT 实例化）生成 Beaver 三元组。为简化表达，我们省略了部分小常数（如密文膨胀常数和域大小位数）。

表 6.3 和表 6.4 分别展示了相关的 MPSI-card / MPSI-card-sum 协议与本文协议在在线阶段与离线阶段的理论对比，其中 n 为集合大小， m 为参与方数量， t 为共谋参与方数量。“SK”表示基于 OT 和对称密钥操作的协议。 λ, σ 分别为计算和统计安全参数。Beaver 三元组与秘密分享相关性（Share correlation）生成方式的不同组合导致了文献 [3] 和本文协议在离线阶段具有不同的复杂度。“BGV + BGV”表示这两者均使用带有分布式解密的加法同态加密（基于 BGV 实例化）生成。“BGV + ST”表示三元组使用同态加密生成，而分享相关性通过运行两两交互的分享转换（Share Translation）协议生成。“OT + BGV/ST”表示三元组使用两两的 SoftSpokenOT 交互生成（在此情况下，采用不同的分享相关性生成方式复杂度相同）。

表 6.5 和表 6.6 分别展示了相关的 MPSU 协议与本文协议在在线阶段与离线阶段的理论对比，其中 n 为集合大小， m 为参与方数量， l 为元素的长度。“PK”表示基于公钥操作的协议。“SK”表示基于 OT 和对称密钥操作的协议。 λ, σ 分别为计算和统计安全参数。文献 [49] 中的协议无离线阶段。不同 Beaver 三元组与分享相关性生成方式的组合对应了本文协议的不同离线复杂度。符号释义与表 6.4 相同。

协议	计算复杂度		通信复杂度		安全性	底层操作
	Leader	Client	Leader	Client		
[41]	$O(m^2n^2)$	$O(m^2n^2)$	$O(m^2n^2\lambda)$	$O(m^2n^2\lambda)$	标准半诚实	PK
[31]	$O(mtn^2)$	$O(mtn^2)$	$O(mtn \log U \lambda)$	$O(mtn \log U \lambda)$	标准半诚实	PK
[80]	$O(mn^2)$	$O(n)$	$O(mn\lambda)$	$O(n\lambda)$	标准半诚实	PK
[43]	$O(mn)$	$O(n)$	$O(mn(\lambda + \sigma + \log n))$	$O(n(\lambda + \sigma + \log n))$	增强半诚实	SK
		$O(tn)$		$O(tn(\lambda + \sigma + \log n))$	标准半诚实	SK
[44]	$O(mn)$		$O(mn)$	$O(\log(m)n\sigma^2)$	增强半诚实	SK
				$O(mn\sigma^2)$	标准半诚实	SK
[81]	$O(mn \log n)$	$O(n \log n)$	$O((m^2 + mn)\lambda)$		恶意	SK
[47]	$O(mn)$		$O(mn\lambda(\log(n\lambda) + \lambda))$	$O(n\lambda(\log(n\lambda) + \lambda))$	增强半诚实/恶意	SK
[46]	$O(mn)$	$O(tn)$	$O(mn(\lambda + \sigma + \log n))$	$O(n(\lambda + \sigma + \log n))$	增强半诚实/恶意	SK
[2]	$O(mn)$	$O(tn)$	$O(mn(\lambda + \sigma + \log n))$	$O(tn(\lambda + \sigma + \log n))$	标准半诚实	SK
本文 MPSI	$O(mn)$	$O(n)$	$O(mn(\sigma + \log n))$	$O(n(\sigma + \log n))$	标准半诚实	SK

表 6.1 半诚实模型下 MPSI 协议的渐近在线通信与计算复杂度

协议	计算复杂度		通信复杂度		安全性	Beaver 三元组生成方式
	Leader	Client	Leader	Client		
[47]	$O(mn)$		$O(mn\lambda^2 + mn\lambda(\log(n\lambda) + \lambda))$	$O(m\lambda + n\lambda(\log(n\lambda) + \lambda))$	增强半诚实/恶意	—
本文 MPSI	$O(mn)$	$O(n)$	$O(mn(\lambda + \sigma + \log n))$	$O(n(\lambda + \sigma + \log n))$	标准半诚实	BGV
	$O(mn(\sigma + \log n))$		$O(mn(\sigma + \log n)\lambda)$		标准半诚实	OT

表 6.2 半诚实模型下 MPSI 协议的渐近离线通信与计算复杂度

协议	计算复杂度		通信复杂度		安全性	底层操作
	Leader	Client	Leader	Client		
[3]	$O((m-t)n + tn \log n)$	$O(tn)$	$O(((m-t)n + tn \log n)(\lambda + \sigma + \log n))$	$O(tn(\lambda + \sigma + \log n))$	标准半诚实	SK
本文协议	$O(mn)$	$O(n)$	$O(mn(\sigma + \log n))$	$O(n(\sigma + \log n))$	标准半诚实	SK

表 6.3 半诚实模型下 MPSI-card / MPSI-card-sum 协议的渐近在线通信与计算复杂度

协议	计算复杂度		通信复杂度		安全性	Beaver 三元组 + 分享相关性生成
	Leader	Client	Leader	Client		
[3]	$O(mn)$		$O(mn(\sigma + \log n))$		标准半诚实	BGV + BGV
	$O(mn \log n)$		$O(mn(\sigma + \lambda \log n))$		标准半诚实	BGV + ST
	$O(mn(\sigma + \log n))$		$O(mn(\sigma + \log n)\lambda)$		标准半诚实	OT + BGV/ST
本文协议	$O(mn)$	$O(n)$	$O(mn(\lambda + \sigma + \log n))$	$O(n(\lambda + \sigma + \log n))$	标准半诚实	BGV + BGV
	$O(mn \log n)$		$O(mn(\sigma + \lambda \log n))$		标准半诚实	BGV + ST
	$O(mn(\sigma + \log n))$		$O(mn(\sigma + \log n)\lambda)$		标准半诚实	OT + BGV/ST

表 6.4 半诚实模型下 MPSI-card / MPSI-card-sum 协议的渐近离线通信与计算复杂度

协议	计算复杂度		通信复杂度		安全性	底层操作
	Leader	Client	Leader	Client		
[49]	$O(mn(\log n / \log \log n))$		$\lambda mn(\log n / \log \log n)$		标准半诚实	PK
[79]	$O(m^2 n)$		$O(m^2 n(l + \sigma + \log m + \log n))$	$O(mn(l + \sigma + \log m + \log n) + m^2 n)$	标准半诚实	SK
本文 MPSU	$O(m^2 n)$	$O(mn)$	$O(m^2 n(l + \sigma + \log m + \log n))$	$O(mn(l + \sigma + \log m + \log n))$	标准半诚实	SK

表 6.5 半诚实模型下 MPSU 协议的渐近在线通信与计算复杂度

协议	计算复杂度		通信复杂度		Beaver 三元组 + 分享相关性生成
	Leader	Client	Leader	Client	
[79]	$O(m^2 n + mn(l + \sigma + \log m + \log n))$		$O(mn(l + \sigma + \log m + \log n)(m + \lambda))$		- + BGV
	$O(m^2 n(\log m + \log n) + mn(l + \sigma))$		$O(m^2 n\lambda(\log m + \log n) + mn(l + \sigma)(m + \lambda))$		- + ST
本文 MPSU	$O(m^2 n)$	$O(mn)$	$O(mn(l + \sigma + \log m + \log n)(m + \lambda))$	$O(mn\lambda(l + \sigma + \log m + \log n))$	BGV + BGV
	$O(m^2 n(\log m + \log n))$		$O(m^2 n\lambda(\log m + \log n) + mn(l + \sigma)(m + \lambda))$	$O(m^2 n\lambda(\log m + \log n) + mn\lambda(l + \sigma))$	BGV + ST
	$O(m^2 n(l + \sigma + \log m + \log n))$		$O(m^2 n(l + \sigma + \log m + \log n)\lambda)$		OT + BGV/ST

表 6.6 半诚实模型下 MPSU 协议的渐近离线通信与计算复杂度

6.7 各类协议的实现与综合性能评估

6.7.1 实现细节与参数设置

本文采用 C++ 语言实现了本文框架中的各实例化协议。其中，每个参与方分配 $m - 1$ 个工作线程，以便同时与其他所有参与方发起并发交互。在底层密码学原语的构建上，我们参考了文献 [69] 的方法，利用向量不经意线性求值 (VOLE) 与不经意键值存储 (OKVS) [16, 45, 18, 52] 来实例化批量 OPPRF；遵循文献 [79] 的策略，通过结合批量 OPPRF 与秘密分享等值测试 (ssPEQT) 来实例化批量 ssPMT。具体地，我们在系统实现中调用了以下开源密码学库：

- **VOLE**：采用 libOTe [71] 库中提供的 VOLE 实现，并通过 Expand-Convolute codes [68] 来实例化其编码族。
- **OKVS 与 GMW**：我们使用了文献 [18] 中提出的优化版 OKVS 构造（考虑到文献 [52] 提出的新型 OKVS 结构在小规模数据集 $n < 2^{10}$ 时参数适配性尚不明确，故作此选择），并直接复用了 [72] 中提供的 OKVS 代码实现。此外，我们同样基于 [72] 中封装的 GMW 实现来构建 ssPEQT 模块。
- **ROT**：采用 libOTe 库中封装的 SoftSpokenOT [58] 原语来实现随机不经意传输功能。
- **其他支持库**：我们在项目中使用 cryptoTools [76] 库完成各类哈希运算与伪随机数生成 (PRNG) 调用；同时采用 Coproto [77] 库来构建并管理网络层通信。

为了平衡系统的性能表现与安全性保障，我们将计算安全参数设定为 $\lambda = 128$ ，统计安全参数设定为 $\sigma = 40$ 。除此之外，核心模块的具体参数配置如下：

布谷鸟哈希 (Cuckoo Hashing) 参数. 为了确保批量 ssPMT 组件能够实现线性的通信复杂度，我们引入了无 stash 的布谷鸟哈希机制 [60]。为了将哈希失败的概率（即元素既无法存入哈希表又无 stash 缓存的冲突事件）严格控制在 2^{-40} 以下，我们针对 3-hash 方案设定了 $B = 1.27n$ 的桶数比例。

OKVS 参数. 我们采用了文献 [18] 中推荐的配置，即权重 $w = 3$ 、簇大小 (cluster size) 为 2^{14} 的方案体系。在这一配置下，OKVS 的膨胀率（即 OKVS 容量除以编码元素总数）为 1.28。

ROT 参数. 在调用 SoftSpokenOT 时，我们将其有限域位数 (field bits) 设定为 5，以此在计算开销与通信开销之间取得最佳平衡。

OPPRF 的输出长度. 根据文献 [79] 的证明，为确保批量 ssPMT 协议的正确执行，OPPRF 的最小输出长度应满足 $\sigma + \log_2 T + \log_2 B$ ，其中 T 表示批量 ssPMT 实例的调用总次数。在本文的 MPSU 协议中，调用总次数 $T = (m^2 - m)/2$ 。由此可推断，本文 MPSU 协议中 OPPRF 输出长度的理论下界应设定为 $\sigma + \log_2((m^2 - m)/2) + \log_2(1.27n)$ 。

有限域大小与全零字符串的长度. 有限域的规模与全零字符串的长度共同决定了本文各项协议中虚假碰撞 (Spurious Collision) 的发生概率。根据第 5.5 节中关于框架的正确性分析, 对于 MPSI、MPSI-card 以及 MPSI-card-sum 协议而言, 有限域规模只需满足 $|\mathbb{F}| \geq B \cdot 2^\sigma = 1.27n \cdot 2^\sigma$, 即可将全局虚假碰撞概率约束在 $2^{-\sigma}$ 内。而对于 MPSU 协议, 其有限域需要同时满足两个条件: $|\mathbb{F}| \geq B \cdot 2^\sigma$ 并且 $\mathbb{F}$ 中元素的比特长度需等于 $l + l'$ 。鉴于全零字符串的长度要求为 $l' \geq \sigma + \log(m-1) + \log B$, 这进一步要求 MPSU 中的有限域大小需满足 $|\mathbb{F}| \geq 2^l + (m-1)B \cdot 2^\sigma$ 。基于上述约束, 在实际实验中, 我们为 MPSI、MPSI-card 及 MPSI-card-sum 协议配置了 GF(64) 域, 而为 MPSU 协议配置了 GF(128) 域 (此时全零字符串 l' 设定为 64 比特)。

6.7.2 实验环境与对比方案

在多方安全计算 (MPC) 领域, 协议的通用性往往是以牺牲其实用性为代价的。例如, 相较于专门设计的专用 MPC 协议, 通用 MPC 协议的运行速度通常要慢上数个数量级。然而引人注目的是, 本文提出的 MPSO 框架成功打破了这一长期存在的性能瓶颈。通过对框架的典型实例 (包括 MPSI、MPSI-card、MPSI-card-sum 以及 MPSU) 进行具体实现, 全部代码已开源于以下链接: <https://github.com/real-world-cryptography/MPSO>。本文充分证明了: 该框架在保留对任意集合公式进行计算的完全通用性的同时, 其实际在线性能达到了甚至超越了现有的最先进 (State-of-the-Art, SOTA) 专用协议。

为了符合实际应用场景, 本文遵循了文献 [23, 1] 中的常用设定, 即假设 Beaver 三元组在离线阶段预先计算完毕并存储在本地。因此, 我们的评估将主要聚焦于框架及各对比协议的 **在线性能** 。**为了保证对比的公平性, 我们选择了当前标准半诚实模型下各自子领域中最先进的专用协议进行基准测试 (Benchmark):

- **最先进的 MPSI [2]:** 该工作提出了 O-Ring 和 K-Star 两个协议, 并提供了开源实现 [82]。鉴于 K-Star 协议在总通信开销相同的情况下运行速度快于 O-Ring, 我们在实验中选择 K-Star 进行对比。我们将腐化阈值设定为 $t = m - 1$, 在扣除随机向量不经意线性求值 (VOLE) 生成的离线开销后, 测量并报告其在线性能。对于文献 [2] 和本文的 MPSI 协议, 我们在表 6.7 中报告了 Leader 的运行时间以及所有参与方的总通信开销。
- **最先进的 MPSI-card [3]:** 由于该工作未提供开源代码, 我们直接引用其论文中的在线阶段实验结果。原论文实验在搭载 Intel i7-12700H 2.30GHz CPU 和 28GB 内存的环境下运行。我们在表 6.8 中列出了文献 [3] 和本文 MPSI-card 协议的 Leader 运行时间与通信开销。
- **最先进的 MPSU [79]:** 该工作提供了两个 MPSU 协议的开源实现 [83]。考虑到基于对称密钥的方案具有更好的在线性能, 我们选择其对称密钥版本进行对比。在 MPSU 基准测试中, 我们设定元素长度为 64 比特, 并在表 6.10 中记录 Leader 的在线运行

时间和所有参与方的总通信开销。

据我们所知，目前学术界尚无针对 MPSI-card-sum 协议的代码实现与实验数据。本文提供了首个开源实现，并在与 MPSI-card 相同的测试设定下，于表 6.9 中报告了其实验数据。

我们在一台配备 Intel(R) Xeon(R) 2.70GHz CPU (32 个物理核心) 和 128 GB 内存的云虚拟机上进行所有实验测试。在局域网 (LAN) 设定中，我们将带宽配置为 10 Gbps，往返时间 (RTT) 延迟设定为 0.1 ms。在广域网 (WAN) 设定中，我们测试了 400 Mbps 和 50 Mbps 两种网络带宽环境，两者的 RTT 延迟均设定为 80 ms。在所有协议的运行过程中，每个参与方均持有相等规模的输入集合，并利用 $m - 1$ 个线程同时与其他参与方进行并发交互。

6.7.3 实验结果与分析

实验结果如表 6.7 至 6.10 中所示，其中 m 为参与方数量， m 个输入集合的大小均相等。文献 [3] 的数据因无可代码直接引自原论文，表 6.8 中含有 — 的单元格表示原论文未报告该数据。表 6.10 中含有 — 的单元格表示内存耗尽的测试项。最佳结果以加粗标出。实验结果表明：

- **本文的 MPSI 及其变体在所有测试场景下的在线性能均显著优于现有的 SOTA 方案。**具体而言，与现有的 SOTA MPSI [2] 相比，本文的 MPSI 协议在 LAN 环境下实现了 $2.4 \sim 5.2\times$ 的速度提升，在 400 Mbps 和 50 Mbps 的 WAN 环境下分别实现了 $1.1 \sim 2.0\times$ 和 $1.1 \sim 2.6\times$ 的加速。与文献 [3] 报告的实验数据相比，本文的 MPSI-card 协议在 LAN 环境下甚至实现了惊人的 $18.0 \sim 63.4\times$ 加速。在通信开销方面，随着参与方数量 m 的增加，本文 MPSI 的性能优势愈发显著，最大可实现 $2.6\times$ 的通信量缩减；而本文的 MPSI-card 相比 [3] 更实现了高达 $14.0 \sim 20.3\times$ 的通信开销降低。此外，本文提出的 MPSI-card-sum 协议在实现更加丰富数据分析功能的同时，其计算和通信开销仅约为本文 MPSI 的两倍。
- **本文的 MPSU 展现出与 SOTA [79] 相当的在线运行时间，但在所有测试情况下均保持最低的通信开销。**值得注意的是，在 WAN 网络环境下，随着参与方数量 m 的增长，本文的 MPSU 协议展现出更强的扩展竞争力，最终成功反超 [79]，实现最高 $1.7\times$ 的加速比。由于广域网 (WAN) 环境更能真实反映跨机构 MPSU 应用的网络条件，本文提出的协议在现实世界的系统部署中无疑具备更强的实际应用优势。

6.8 本章小结

m	协议	时间 (秒)															通信量 (MB)				
		LAN					400Mbps					50Mbps									
		2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰
3	[2]	0.48	0.168	0.822	4.227	19.18	2.863	3.309	4.232	7.759	27.32	3.155	3.544	5.149	11.57	42.38	0.423	1.681	6.344	26.72	110.0
	本文协议	0.013	0.047	0.227	1.533	7.945	1.540	2.070	2.912	5.136	16.42	1.576	2.231	3.983	10.43	38.63	0.641	2.551	10.20	40.91	164.2
4	[2]	0.054	0.181	0.854	4.306	20.09	3.137	3.537	4.528	8.529	30.47	3.158	4.024	6.495	15.90	61.41	0.896	3.557	13.42	56.11	231.1
	本文协议	0.015	0.051	0.242	1.632	8.133	1.944	2.476	3.339	6.345	19.23	1.984	2.729	4.848	14.04	51.31	0.961	3.826	15.31	61.36	246.2
5	[2]	0.072	0.201	0.948	4.611	21.36	3.038	3.733	4.969	9.031	34.16	3.587	4.430	8.443	23.53	90.40	1.542	6.124	23.10	96.23	396.3
	本文协议	0.016	0.053	0.260	1.724	8.255	2.348	2.889	3.770	6.789	22.04	2.424	3.144	5.930	16.98	64.59	1.282	5.102	20.41	81.81	328.3
10	[2]	0.136	0.373	1.479	6.812	30.89	4.681	5.536	7.178	16.84	65.14	6.928	10.87	25.10	87.19	346.5	7.375	29.30	110.5	456.9	1883
	本文协议	0.026	0.075	0.354	1.910	8.898	4.363	4.915	5.769	9.423	33.38	4.496	5.766	11.01	34.13	133.3	2.884	11.48	45.92	184.1	738.7

表 6.7 各 MPSI 协议的运行时间与通信开销

m	协议	时间 (秒)															通信量 (MB)				
		LAN					400Mbps					50Mbps									
		2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰
3	本文协议	0.015	0.052	0.237	1.581	8.050	1.784	2.800	3.681	6.498	19.50	1.836	3.028	5.202	13.40	46.89	0.761	3.035	12.15	48.76	195.8
4	本文协议	0.016	0.054	0.252	1.684	8.300	2.267	3.780	5.005	8.565	24.88	2.331	4.109	6.905	18.56	64.89	1.122	4.472	17.91	71.83	288.4
5	[3]	0.670	1.789	6.289	31.24	—	—	—	—	—	—	—	—	—	—	—	20.70	94.49	425.6	1894	—
	本文协议	0.018	0.056	0.270	1.735	8.616	2.753	4.747	6.076	10.42	28.87	2.872	5.134	8.819	23.76	83.80	1.482	5.909	23.66	94.90	381.0
10	[3]	1.477	4.503	12.81	95.23	—	—	—	—	—	—	—	—	—	—	—	46.58	212.6	957.7	4262	—
	本文协议	0.026	0.071	0.375	2.001	9.226	5.174	9.578	11.93	18.43	50.33	5.346	10.56	18.07	49.45	174.6	3.285	13.09	52.42	210.2	844.0

表 6.8 各 MPSI-card 协议的运行时间与通信开销对比

m	时间 (秒)															通信量 (MB)				
	LAN					400Mbps					50Mbps									
	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰
3	0.023	0.088	0.417	2.810	14.67	2.519	3.541	4.686	9.417	31.88	2.620	3.987	7.237	20.83	78.99	1.283	5.107	20.43	81.89	328.6
4	0.025	0.091	0.436	3.044	15.16	3.084	4.680	5.948	12.40	38.17	3.207	5.268	9.564	28.56	104.4	1.885	7.499	29.99	120.2	482.4
5	0.027	0.094	0.474	3.150	15.49	3.650	5.729	7.288	13.17	43.06	3.843	6.526	12.10	36.71	133.6	2.486	9.890	39.56	158.6	636.2
10	0.039	0.120	0.632	3.610	16.65	6.474	10.93	13.79	23.56	71.36	6.781	12.76	24.96	78.64	287.1	5.493	21.85	87.37	350.2	1405

表 6.9 本文的 MPSI-card-sum 协议的运行时间与通信开销

m	协议	时间 (秒)															通信量 (MB)				
		LAN					400Mbps					50Mbps									
		2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰	2 ¹²	2 ¹⁴	2 ¹⁶	2 ¹⁸	2 ²⁰
3	[79]	0.017	0.050	0.215	1.005	4.352	3.157	3.734	4.444	9.705	33.10	3.340	4.495	9.779	32.11	118.7	2.416	9.702	39.87	161.2	652.1
	本文协议	0.022	0.068	0.298	1.892	9.607	3.327	3.774	4.878	11.04	37.77	3.535	4.606	10.25	32.50	120.97	2.414	9.695	39.84	161.1	651.6
4	[79]	0.023	0.071	0.286	1.393	5.645	3.976	4.618	6.507	17.10	59.21	4.270	6.924	19.00	69.23	267.2	5.653	23.06	93.06	376.0	1520
	本文协议	0.029	0.089	0.415	2.345	11.25	3.996	4.820	6.756	17.45	64.04	4.360	6.609	17.85	63.63	245.0	5.083	20.77	83.83	338.9	1370
5	[79]	0.030	0.087	0.368	1.714	7.003	4.800	5.521	8.938	25.95	95.40	5.419	10.70	32.81	126.5	500.1	10.69	43.52	175.6	709.1	2865
	本文协议	0.039	0.114	0.542	2.796	13.18	4.872	5.705	8.992	26.09	101.6	5.261	9.393	28.09	105.6	416.8	8.739	35.69	144.0	582.1	2358
10	[79]	0.088	0.286	1.183	—	—	9.203	14.54	38.31	—	—	18.01	56.11	213.2	—	—	74.68	300.2	1210	—	—
	本文协议	0.110	0.337	1.483	7.167	—	8.187	12.06	30.56	105.7	—	12.14	34.22	125.1	485.7	—	42.40	170.2	686.8	2775	—

表 6.10 各 MPSU 协议的运行时间与通信开销对比

7 基于公钥体制的通用多方隐私集合运算框架

7.1 引言

7.2 谓词零加密

7.3 成员零加密

7.4 框架构造

7.5 实例化协议

7.6 理论分析与对比

7.7 本章小结

8 总结与展望

8.1 全文总结

8.2 未来工作展望

参考文献

- [1] X. Liu and Y. Gao, “Scalable multi-party private set union from multi-query secret-shared private membership test,” in *ASIACRYPT 2023*, Springer, 2023.
- [2] M. Wu, T. H. Yuen, and K. Y. Chan, “O-ring and k-star: Efficient multi-party private set intersection,” in *USENIX Security 2024*, 2024.
- [3] Y. Chen, N. Ding, D. Gu, and Y. Bian, “Practical multi-party private set intersection cardinality and intersection-sum under arbitrary collusion,” in *Inscrypt 2022*, Lecture Notes in Computer Science, Springer, 2022.
- [4] A. C.-C. Yao, “Theory and applications of trapdoor functions (extended abstract),” in *23rd Annual Symposium on Foundations of Computer Science, FOCS 1982*, pp. 80–91, IEEE Computer Society, 1982.
- [5] O. Goldreich, S. Micali, and A. Wigderson, “How to play any mental game or A completeness theorem for protocols with honest majority,” in *Proceedings of the 19th Annual ACM Symposium on Theory of Computing, 1987*, pp. 218–229, ACM, 1987.
- [6] M. Ben-Or, S. Goldwasser, and A. Wigderson, “Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract),” in *Proceedings of the 20th Annual ACM Symposium on Theory of Computing, 1988*, pp. 1–10, ACM, 1988.
- [7] A. Narayanan, N. Thiagarajan, M. Lakhani, M. Hamburg, and D. Boneh, “Location privacy via private proximity testing,” in *NDSS 2011*, 2011.
- [8] M. Ion, B. Kreuter, A. E. Nergiz, S. Patel, S. Saxena, K. Seth, M. Raykova, D. Shana-han, and M. Yung, “On deploying secure computing: Private intersection-sum-with-cardinality,” in *IEEE European Symposium on Security and Privacy, EuroS&P 2020*, pp. 370–389, IEEE, 2020.
- [9] A. Bhowmick, D. Boneh, S. Myers, K. Talwar, and K. Tarbe, “The apple psi system,” 2021.
- [10] A. Berke, M. Bakker, P. Vepakomma, R. Raskar, K. Larson, and A. Pentland, “Assessing disease exposure risk with location histories and protecting privacy: A cryptographic approach in response to a global pandemic,” 2020.

- [11] K. Hogan, N. Luther, N. Schear, E. Shen, D. Stott, S. Yakoubov, and A. Yerukhimovich, “Secure multiparty computation for cooperative cyber risk assessment,” in *IEEE Cybersecurity Development, 2016*, pp. 75–76, IEEE Computer Society, 2016.
- [12] B. Pinkas, T. Schneider, and M. Zohner, “Faster private set intersection based on OT extension,” in *USENIX Security 2014*, pp. 797–812, 2014.
- [13] V. Kolesnikov, R. Kumaresan, M. Rosulek, and N. Trieu, “Efficient batched oblivious PRF with applications to private set intersection,” in *CCS 2016*, pp. 818–829, ACM, 2016.
- [14] B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, “Spot-light: Lightweight private set intersection from sparse OT extension,” in *Advances in Cryptology - CRYPTO 2019*, vol. 11694 of *Lecture Notes in Computer Science*, pp. 401–431, Springer, 2019.
- [15] M. Chase and P. Miao, “Private set intersection in the internet setting from lightweight oblivious PRF,” in *Advances in Cryptology - CRYPTO 2020*, vol. 12172 of *Lecture Notes in Computer Science*, pp. 34–63, Springer, 2020.
- [16] B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, “PSI from paxos: Fast, malicious private set intersection,” in *EUROCRYPT 2020*, Springer, 2020.
- [17] P. Rindal and P. Schoppmann, “VOLE-PSI: fast OPRF and circuit-psi from vector-ole,” in *EUROCRYPT 2021*, vol. 12697, pp. 901–930, Springer, 2021.
- [18] S. Raghuraman and P. Rindal, “Blazing fast PSI from improved OKVS and subfield VOLE,” in *ACM CCS 2022*, ACM, 2022.
- [19] G. Garimella, P. Mohassel, M. Rosulek, S. Sadeghian, and J. Singh, “Private set operations from oblivious switching,” in *Public-Key Cryptography - PKC 2021*, vol. 12711 of *Lecture Notes in Computer Science*, pp. 591–617, Springer, 2021.
- [20] Y. Chen, M. Zhang, C. Zhang, M. Dong, and W. Liu, “Private set operations from multi-query reverse private membership test,” in *Public-Key Cryptography - PKC 2024*, Springer, 2024.
- [21] V. Kolesnikov, M. Rosulek, N. Trieu, and X. Wang, “Scalable private set union from symmetric-key techniques,” in *Advances in Cryptology - ASIACRYPT 2019*, vol. 11922 of *Lecture Notes in Computer Science*, pp. 636–666, Springer, 2019.
- [22] Y. Jia, S. Sun, H. Zhou, J. Du, and D. Gu, “Shuffle-based private set union: Faster and more secure,” in *USENIX 2022*, 2022.

- [23] C. Zhang, Y. Chen, W. Liu, M. Zhang, and D. Lin, “Optimal private set union from multi-query reverse private membership test,” in *USENIX 2023*, pp. 337–354, USENIX Association, 2023.
- [24] Y. Jia, S. Sun, H. Zhou, and D. Gu, “Scalable private set union, with stronger security,” in *USENIX Security 2024*, 2024.
- [25] D. Demmler, P. Rindal, M. Rosulek, and N. Trieu, “PIR-PSI: scaling private contact discovery,” *Proc. Priv. Enhancing Technol.*, vol. 2018, no. 4, pp. 159–178, 2018.
- [26] J. R. Troncoso-Pastoriza, S. Katzenbeisser, and M. U. Celik, “Privacy preserving error resilient dna searching through oblivious automata,” in *ACM CCS 2007*, pp. 519–528, 2007.
- [27] T. Duong, D. H. Phan, and N. Trieu, “Catalic: Delegated psi cardinality with applications to contact tracing,” in *Advances in Cryptology – ASIACRYPT 2020*, Springer, 2020.
- [28] S. Dittmer, Y. Ishai, S. Lu, R. Ostrovsky, M. Elsabagh, N. Kiourtis, B. Schulte, and A. Stavrou, “Function secret sharing for PSI-CA: with applications to private contact tracing,” *IACR Cryptol. ePrint Arch.*, p. 1599, 2020.
- [29] A. K. Lenstra and T. Voss, “Information security risk assessment, aggregation, and mitigation,” in *Information Security and Privacy: 9th Australasian Conference, ACISP 2004.*, vol. 3108 of *Lecture Notes in Computer Science*, pp. 391–401, Springer, 2004.
- [30] J. Brickell and V. Shmatikov, “Privacy-preserving graph algorithms in the semi-honest model,” in *Advances in Cryptology - ASIACRYPT 2005, 11th International Conference on the Theory and Application of Cryptology and Information Security*, vol. 3788 of *Lecture Notes in Computer Science*, pp. 236–252, Springer, 2005.
- [31] L. Kissner and D. X. Song, “Privacy-preserving set operations,” in *Advances in Cryptology - CRYPTO 2005*, vol. 3621 of *Lecture Notes in Computer Science*, pp. 241–257, Springer, 2005.
- [32] S. Zhang, “Efficient VOLE based multi-party PSI with lower communication cost,” *IACR Cryptol. ePrint Arch.*, p. 1690, 2023.
- [33] J. Gao, N. Trieu, and A. Yanai, “Multiparty private set intersection cardinality and its applications,” *Proc. Priv. Enhancing Technol.*, vol. 2024, no. 2, 2024.

- [34] J. Su and Z. Chen, “Secure and scalable circuit-based protocol for multi-party private set intersection,” *CoRR*, vol. abs/2309.07406, 2023.
- [35] R. Li and C. Wu, “An unconditionally secure protocol for multi-party set intersection,” in *Applied Cryptography and Network Security - ACNS 2007*, vol. 4521 of *Lecture Notes in Computer Science*, pp. 226–236, Springer, 2007.
- [36] M. Blanton and E. Aguiar, “Private and oblivious set and multiset operations,” in *ASIACCS 2012*, pp. 40–41, ACM, 2012.
- [37] J. H. Seo, J. H. Cheon, and J. Katz, “Constant-round multi-party private set union using reversed laurent series,” in *Public Key Cryptography - PKC 2012*, pp. 398–412, Springer, 2012.
- [38] N. Chandran, N. Dasgupta, D. Gupta, S. L. B. Obbattu, S. Sekar, and A. Shah, “Efficient linear multiparty PSI and extensions to circuit/quorum PSI,” in *CCS ’21*, pp. 1182–1204, ACM, 2021.
- [39] A. Bay, Z. Erkin, J. Hoepman, S. Samardjiska, and J. Vos, “Practical multi-party private set intersection protocols,” *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1–15, 2022.
- [40] J. Vos, M. Conti, and Z. Erkin, “Fast multi-party private set operations in the star topology from secure ands and ors,” *IACR Cryptol. ePrint Arch.*, p. 721, 2022.
- [41] M. J. Freedman, K. Nissim, and B. Pinkas, “Efficient private matching and set intersection,” in *Advances in Cryptology - EUROCRYPT 2004*, vol. 3027 of *Lecture Notes in Computer Science*, pp. 1–19, Springer, 2004.
- [42] C. Hazay and M. Venkatasubramanian, “Scalable multi-party private set-intersection,” in *Public-Key Cryptography - PKC 2017*, vol. 10174 of *Lecture Notes in Computer Science*, pp. 175–203, Springer, 2017.
- [43] V. Kolesnikov, N. Matania, B. Pinkas, M. Rosulek, and N. Trieu, “Practical multi-party private set intersection from symmetric-key techniques,” in *CCS 2017*, pp. 1257–1272, ACM, 2017.
- [44] R. Inbar, E. Omri, and B. Pinkas, “Efficient scalable multiparty private set-intersection via garbled bloom filters,” in *SCN 2018*, vol. 11035 of *Lecture Notes in Computer Science*, pp. 235–252, Springer, 2018.

- [45] G. Garimella, B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, “Oblivious key-value stores and amplification for private set intersection,” in *Advances in Cryptology - CRYPTO 2021*, vol. 12826 of *Lecture Notes in Computer Science*, pp. 395–425, Springer, 2021.
- [46] O. Nevo, N. Trieu, and A. Yanai, “Simple, fast malicious multiparty private set intersection,” in *CCS 2021*, pp. 1151–1165, ACM, 2021.
- [47] A. Ben-Efraim, O. Nissenbaum, E. Omri, and A. Paskin-Cherniavsky, “Psimple: Practical multiparty maliciously-secure private set intersection,” in *ASIA CCS ’22*, pp. 1098–1112, ACM, 2022.
- [48] K. B. Frikken, “Privacy-preserving set union,” in *ACNS 2007*, vol. 4521 of *Lecture Notes in Computer Science*, pp. 237–252, Springer, 2007.
- [49] J. Gao, S. Nguyen, and N. Trieu, “Toward A practical multi-party private set union,” *Proc. Priv. Enhancing Technol.*, vol. 2024, no. 4, pp. 622–635, 2024.
- [50] P. Giorgi, F. Laguillaumie, L. Ottow, and D. Vergnaud, “Fast secure computations on shared polynomials and applications to private set operations,” in *ITC 2024*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
- [51] C. Hazay and Y. Lindell, *Efficient Secure Two-Party Protocols - Techniques and Constructions*. Information Security and Cryptography, Springer, 2010.
- [52] A. Bienstock, S. Patel, J. Y. Seo, and K. Yeo, “Near-Optimal oblivious Key-Value stores for efficient PSI, PSU and Volume-Hiding Multi-Maps,” in *USENIX Security 2023*, pp. 301–318, 2023.
- [53] T. ElGamal, “A public key cryptosystem and a signature scheme based on discrete logarithms,” *IEEE Transactions on Information Theory*, vol. 31, pp. 469–472, 1985.
- [54] J. Gao, S. Nguyen, and N. Trieu, “Toward a practical multi-party private set union.” Cryptology ePrint Archive, Paper 2023/1930, 2023. Version: 20240316:210303, <https://eprint.iacr.org/archive/2023/1930/20240316:210303>.
- [55] O. Goldreich, *The Foundations of Cryptography - Volume 2: Basic Applications*. Cambridge University Press, 2004.
- [56] R. Canetti, “Universally composable security: A new paradigm for cryptographic protocols,” in *FOCS 2001*, pp. 136–145, IEEE Computer Society, 2001.

-
- [57] M. O. Rabin, “How to exchange secrets with oblivious transfer,” *IACR Cryptol. ePrint Arch.*, p. 187, 2005.
 - [58] L. Roy, “Softspokenot: Quieter OT extension from small-field silent VOLE in the minicrypt model,” in *CRYPTO 2022*, Springer, 2022.
 - [59] M. J. Freedman, Y. Ishai, B. Pinkas, and O. Reingold, “Keyword search and oblivious pseudorandom functions,” in *TCC 2005*, vol. 3378 of *Lecture Notes in Computer Science*, pp. 303–324, Springer, 2005.
 - [60] B. Pinkas, T. Schneider, O. Tkachenko, and A. Yanai, “Efficient circuit-based PSI with linear communication,” in *EUROCRYPT 2019*, vol. 11478 of *Lecture Notes in Computer Science*, pp. 122–153, Springer, 2019.
 - [61] N. Chandran, D. Gupta, and A. Shah, “Circuit-psi with linear complexity via relaxed batch OPPRF,” *Proc. Priv. Enhancing Technol.*, 2022.
 - [62] S. Eskandarian and D. Boneh, “Clarion: Anonymous communication from multiparty shuffling protocols,” in *NDSS 2022*, The Internet Society, 2022.
 - [63] B. Pinkas, T. Schneider, G. Segev, and M. Zohner, “Phasing: Private set intersection using permutation-based hashing,” in *USENIX Security 2015*, pp. 515–530, USENIX Association, 2015.
 - [64] R. Pagh and F. F. Rodler, “Cuckoo hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
 - [65] N. Chandran, N. Dasgupta, D. Gupta, S. L. B. Obbattu, S. Sekar, and A. Shah, “Efficient linear multiparty PSI and extensions to circuit/quorum PSI,” in *CCS 2021*, pp. 1182–1204, ACM, 2021.
 - [66] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, P. Rindal, and P. Scholl, “Efficient two-round OT extension and silent non-interactive secure computation,” in *CCS 2019*, pp. 291–308, ACM, 2019.
 - [67] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, and P. Scholl, “Efficient pseudorandom correlation generators: Silent OT extension and more,” in *Advances in Cryptology - CRYPTO 2019*, Springer, 2019.
 - [68] S. Raghuraman, P. Rindal, and T. Tanguy, “Expand-convolute codes for pseudorandom correlation generators from LPN,” in *CRYPTO 2023*, vol. 14084 of *Lecture Notes in Computer Science*, pp. 602–632, Springer, 2023.

- [69] D. Bui and G. Couteau, “Improved private set intersection for sets with small entries,” in *Public-Key Cryptography - PKC 2023*, vol. 13941 of *Lecture Notes in Computer Science*, pp. 190–220, Springer, 2023.
- [70] M. Chase, E. Ghosh, and O. Poburinnaya, “Secret-shared shuffle,” in *Advances in Cryptology - ASIACRYPT 2020*, vol. 12493 of *Lecture Notes in Computer Science*, pp. 342–372, Springer, 2020.
- [71] P. Rindal, “libOTe: an efficient, portable, and easy to use Oblivious Transfer Library,” 2024. <https://github.com/osu-crypto/libOTe>.
- [72] P. Rindal, “Vole-PSI,” 2024. <https://github.com/Visa-Research/volepsi>.
- [73] J. Du, “PSU,” <https://github.com/dujiajun/PSU.git>.
- [74] P. Mohassel and S. S. Sadeghian, “How to hide circuits in MPC an efficient framework for private function evaluation,” in *Advances in Cryptology - EUROCRYPT 2013*, vol. 7881 of *Lecture Notes in Computer Science*, pp. 557–574, Springer, 2013.
- [75] “OpenSSL: TLS/SSL and crypto library.” <https://github.com/openssl/openssl.git>.
- [76] P. Rindal, “cryptoTools,” 2021. <https://github.com/ladnir/cryptoTools>.
- [77] P. Rindal, “Coproto,” 2024. <https://github.com/Visa-Research/coproto>.
- [78] D. Beaver, “Efficient multiparty protocols using circuit randomization,” in *CRYPTO 1991*, vol. 576, pp. 420–432, Springer, 1991.
- [79] M. Dong, C. Zhang, Y. Bai, and Y. Chen, “Efficient multi-party private set union without non-collusion assumptions,” in *34rd USENIX Security Symposium, USENIX Security 2025*, USENIX Association, 2025.
- [80] C. Hazay and M. Venkatasubramanian, “Scalable multi-party private set-intersection,” in *Public-Key Cryptography - PKC 2017*, vol. 10174 of *Lecture Notes in Computer Science*, pp. 175–203, Springer, 2017.
- [81] S. Ghosh and T. Nilges, “An algebraic approach to maliciously secure private set intersection,” in *Advances in Cryptology - EUROCRYPT 2019*, vol. 11478 of *Lecture Notes in Computer Science*, pp. 154–185, Springer, 2019.
- [82] M. Wu, T. H. Yuen, and K. Y. Chan, “oring,” 2024. <https://github.com/private-panda/oring>.

- [83] M. Dong, Y. Bai, and Y. Cao, “MPSU: An efficient c++ repo implementing MPSU”, 2025. <https://github.com/real-world-cryptography/MPSU>.

致 谢

攻读博士学位期间发表的学术论文

1. **Minglang Dong**, Yu Chen, Cong Zhang, Yujie Bai, and Yang Cao. Efficient multi-party private set union without non-collusion assumptions. In USENIX Security 2025.
2. **Minglang Dong**, Yu Chen, Cong Zhang, Yujie Bai, and Yang Cao. Multi-Party Private Set Operations from Predicative Zero-Sharing. In ACM CCS 2025.
3. Yu Chen, Min Zhang, Cong Zhang, **Minglang Dong**, Weiran Liu. Private Set Operations from Multi Query Reverse Private Membership Test. In PKC 2024.

攻读博士学位期间所获奖项

1. 2022 年“金融密码杯”全国密码应用和技术创新大赛, 创新赛, 团队特等奖, 2023.02.
2. 山东大学网络空间安全学院 2025 年度国家奖学金, 2025.10.